

User Manual

for S32K1_S32M24X SPI Driver

Document Number: UM2SPIASRR21-11 Rev0000R3.0.0 Rev. 1.0

1 Revision History	2
2 Introduction	3
2.1 Supported Derivatives	3
2.2 Overview	4
2.3 About This Manual	4
2.4 Acronyms and Definitions	6
2.5 Reference List	7
3 Driver	8
3.1 Requirements	8
3.2 Driver Design Summary	8
3.3 Hardware Resources	10
3.4 Deviations from Requirements	11
3.5 Driver Limitations	14
3.5.1 Some LPSPI's features are not supported:	14
3.5.2 Limitation of LPSPI Slave mode:	14
3.5.3 Some limitations of SPI over FLEXIO:	14
3.5.4 Some LPSPI's features are not supported:	15
3.5.5 Some limitations of SPI over FLEXIO:	15
3.6 Driver usage and configuration tips	15
3.6.1 How to configure handling Chip Select via general purpose IO (SpiCsSelection: CS_VIA_GPIO)	15
3.6.2 How to configure in multiple post build variants	17
3.6.3 How to configure MultiPartition	18
3.6.4 How to config interrupts for the transmission of Master and Slave on Spi examples in the same chip	19
3.6.5 How to use dual clock mode	20
3.6.6 How to configure handling Chip Select via general purpose IO (SpiCsSelection: CS_VIA_GPIO)	21
3.6.7 How to configure in multiple post build variants	23
3.6.8 How to configure Dma Fast transfer for a Sequence	24
3.6.9 How to configure SPI over FLEXIO	30
3.6.9.1 EB Tresos	31
3.6.9.2 Design Studio Tool:	33
3.6.9.3 Register ISR for INTERRUPT mode	35
3.6.10 How to configure to use half duplex mode	35
3.6.11 How to use Half Duplex mode	38
3.6.12 Pointer to buffer alignment	40
3.6.13 How to config interrupts for the transmission of Master and Slave on Spi examples in the same chip	40
3.6.14 How to configure to use Spi Host Request feature.	40
3.6.15 How to use byte swap feature for LPSPI hardware instance.	42
3.6.16 How to configure baudrate registers	42
3.6.17 How to use Half Duplex mode	43
3.6.18 How to configure to use Spi Host Request feature.	45
3.6.19 How to use byte swap feature for LPSPI hardware instance.	47
3.6.20 How to configure SPI over FLEXIO	47
3.6.20.1 EB Tresos	48
3.6.20.2 Configuration Tool: It is the same with EB tresos	50
3.6.20.3 Register ISR for INTERRUPT mode	52
3.7 Runtime errors	52
3.8 Symbolic Names Disclaimer	53
3.9 Usage in non AUTOSAR context	53
4 Configuration Parameters	55

4.1 Spi	56
4.1.1 Module Spi	59
4.1.2 Container SpiDemEventParameterRefs	59
4.1.3 Reference SPI_E_HARDWARE_ERROR	60
4.1.4 Container SpiDriver	60
4.1.5 Parameter SpiMaxChannel	60
4.1.6 Parameter SpiMaxJob	61
4.1.7 Parameter SpiMaxSequence	61
4.1.8 Container SpiChannel	62
4.1.9 Parameter SpiChannelId	62
4.1.10 Parameter SpiChannelType	63
4.1.11 Parameter SpiDataWidth	63
4.1.12 Parameter SpiDefaultData	64
4.1.13 Parameter SpiEbMaxLength	64
4.1.14 Parameter SpiIbNBuffers	65
4.1.15 Parameter SpiTransferStart	65
4.1.16 Parameter SpiByteSwapTransfer	66
4.1.17 Parameter SpiChannelHalfDuplexDirection	66
4.1.18 Reference SpiChannelEcucPartitionRef	67
4.1.19 Container SpiExternalDevice	67
4.1.20 Parameter SpiBaudrate	68
4.1.21 Parameter SpiUseBaudrateConfig	68
4.1.22 Parameter SpiCalculatedBaudrate	69
4.1.23 Parameter SpiCalculatedAlternateBaudrate	69
4.1.24 Parameter SpiCsIdentifier	70
4.1.25 Parameter SpiCsPolarity	70
4.1.26 Parameter SpiCsSelection	71
4.1.27 Parameter SpiDataShiftEdge	71
4.1.28 Parameter SpiEnableCs	72
4.1.29 Parameter SpiHwUnit	72
4.1.30 Parameter SpiShiftClockIdleLevel	73
4.1.31 Parameter SpiTimeClk2Cs	73
4.1.32 Parameter SpiTimeCs2Clk	74
4.1.33 Parameter SpiTimeCs2Cs	74
4.1.34 Parameter SpiCsBehavior	75
4.1.35 Parameter SpiHostRequest	75
4.1.36 Parameter SpiDeviceHalfDuplexSupport	76
4.1.37 Parameter SpiTransferWidth	76
4.1.38 Parameter SpiHalfDuplexPinSelect	77
4.1.39 Reference SpiBaudrateConfigRef	77
4.1.40 Reference SpiDeviceEcucPartitionRef	78
4.1.41 Container SpiBaudrateConfig	78
4.1.42 Parameter IpType	79
4.1.43 Container LpspiConfig	79
4.1.44 Parameter SpiPrescaler	79
4.1.45 Parameter SpiPrescalerValue	80
4.1.46 Parameter SpiScaler	80
4.1.47 Container SpiJob	81
4.1.48 Parameter SpiJobEndNotification	81
4.1.49 Parameter SpiJobStartNotification	81
4.1.50 Parameter SpiJobId	82
4.1.51 Parameter SpiJobPriority	82
4.1.52 Reference SpiDeviceAssignment	83
4.1.53 Container SpiChannelList	83

4.1.54	Parameter	SpiChannelIndex	84
4.1.55	Reference	SpiChannelAssignment	84
4.1.56	Container	SpiSequence	84
4.1.57	Parameter	SpiInterruptibleSequence	85
4.1.58	Parameter	SpiSeqEndNotification	85
4.1.59	Parameter	SpiSequenceId	86
4.1.60	Parameter	SpiEnableDmaFastTransfer	86
4.1.61	Reference	SpiJobAssignment	87
4.1.62	Container	SpiGeneral	87
4.1.63	Parameter	SpiMultipartitionSupport	87
4.1.64	Parameter	SpiCancelApi	88
4.1.65	Parameter	SpiChannelBuffersAllowed	88
4.1.66	Parameter	SpiDevErrorDetect	89
4.1.67	Parameter	SpiHwStatusApi	89
4.1.68	Parameter	SpiInterruptibleSeqAllowed	90
4.1.69	Parameter	SpiLevelDelivered	90
4.1.70	Parameter	SpiMainFunctionPeriod	90
4.1.71	Parameter	SpiSupportConcurrentSyncTransmit	91
4.1.72	Parameter	SpiVersionInfoApi	91
4.1.73	Parameter	SpiGlobalDmaEnable	92
4.1.74	Parameter	SpiTimeoutMethod	92
4.1.75	Parameter	SpiTransmitTimeout	93
4.1.76	Reference	SpiEcucPartitionRef	93
4.1.77	Reference	SpiKernelEcucPartitionRef	94
4.1.78	Container	SpiPhyUnit	94
4.1.79	Parameter	SpiPhyUnitMapping	95
4.1.80	Parameter	SpiPhyUnitMode	95
4.1.81	Parameter	SpiSamplePoint	96
4.1.82	Parameter	SpiPinConfiguration	96
4.1.83	Parameter	SpiPhyUnitAsyncUseDma	97
4.1.84	Parameter	SpiMaxDmaFastTransfer	97
4.1.85	Reference	SpiPhyUnitClockRef	98
4.1.86	Reference	SpiPhyUnitAlternateClockRef	98
4.1.87	Reference	SpiPhyTxDmaChannel	99
4.1.88	Reference	SpiPhyRxDmaChannel	99
4.1.89	Reference	SpiFlexioTxAndClkChannelsConfig	100
4.1.90	Reference	SpiFlexioRxAndCsChannelsConfig	100
4.1.91	Container	SpiPublishedInformation	101
4.1.92	Parameter	SpiMaxHwUnit	101
4.1.93	Container	CommonPublishedInformation	102
4.1.94	Parameter	ArReleaseMajorVersion	102
4.1.95	Parameter	ArReleaseMinorVersion	103
4.1.96	Parameter	ArReleaseRevisionVersion	103
4.1.97	Parameter	ModuleId	104
4.1.98	Parameter	SwMajorVersion	104
4.1.99	Parameter	SwMinorVersion	104
4.1.100	Parameter	SwPatchVersion	105
4.1.101	Parameter	VendorApiInfix	105
4.1.102	Parameter	VendorId	106
4.1.103	Container	SpiAutosarExt	106
4.1.104	Parameter	SpiEnableUserModeSupport	107
4.1.105	Parameter	SpiEnableDmaFastTransferSupport	107
4.1.106	Parameter	SpiHalfDuplexModeSupport	108
4.1.107	Parameter	SpiAllowBigSizeCollections	108

4.1.108 Parameter SpiEnableHWUnitAsyncMode	108
4.1.109 Parameter SpiJobStartNotificationEnable	109
4.1.110 Parameter SpiDisableDemReportErrorStatus	109
4.1.111 Parameter SpiUseLegacyEBLength	110
4.1.112 Parameter SpiFlexioEnable	110
4.2 LPSPI	111
4.2.1 Module Lpspi	112
4.2.2 Parameter LpspiIpDevErrorDetect	112
4.2.3 Parameter SpiEnableLargeFrameSize	113
4.2.4 Parameter SpiPhyUnitClockSourceValue	113
4.3 FLEXIO_SPI	114
4.3.1 Module Flexio_Spi	115
4.3.2 Parameter FlexioSpiIpDevErrorDetect	115
4.3.3 Parameter SpiPhyUnitClockSourceValue	115
5 Module Index	117
5.1 Software Specification	117
6 Module Documentation	118
6.1 Flexio_Spi Driver	118
6.1.1 Detailed Description	118
6.1.2 Data Structure Documentation	119
6.1.2.1 struct Flexio_Spi_Ip_DeviceParamsType	119
6.1.2.2 struct Flexio_Spi_Ip_ExternalDeviceType	120
6.1.2.3 struct Flexio_Spi_Ip_ConfigType	120
6.1.2.4 struct Flexio_Spi_Ip_StateStructureType	121
6.1.3 Macro Definition Documentation	121
6.1.3.1 SPI_STOP_SEC_CONFIG_DATA_UNSPECIFIED	121
6.1.4 Types Reference	121
6.1.4.1 Flexio_Spi_Ip_CallbackType	122
6.1.5 Enum Reference	122
6.1.5.1 Flexio_Spi_Ip_EventType	122
6.1.5.2 Flexio_Spi_Ip_ModeType	122
6.1.5.3 Flexio_Spi_Ip_HwStatusType	122
6.1.5.4 Flexio_Spi_Ip_StatusType	123
6.1.6 Function Reference	123
6.1.6.1 Flexio_Spi_Ip_Init()	123
6.1.6.2 Flexio_Spi_Ip_DeInit()	123
6.1.6.3 Flexio_Spi_Ip_SyncTransmit()	124
6.1.6.4 Flexio_Spi_Ip_AsyncTransmit()	124
6.1.6.5 Flexio_Spi_Ip_GetStatus()	125
6.1.6.6 Flexio_Spi_Ip_ManageBuffers()	125
6.1.6.7 Flexio_Spi_Ip_UpdateTransferParam()	125
6.1.6.8 Flexio_Spi_Ip_UpdateFrameSize()	126
6.1.6.9 Flexio_Spi_Ip_UpdateLsb()	126
6.1.6.10 Flexio_Spi_Ip_UpdateDefaultTransmitData()	127
6.1.6.11 Flexio_Spi_Ip_UpdateTransferMode()	127
6.1.6.12 Flexio_Spi_Ip_Cancel()	127
6.2 Lpspi Driver	129
6.2.1 Detailed Description	129
6.2.2 Data Structure Documentation	131
6.2.2.1 struct Lpspi_Ip_DeviceParamsType	131
6.2.2.2 struct Lpspi_Ip_ExternalDeviceType	131
6.2.2.3 struct Lpspi_Ip_CmdDmaFastType	131

6.2.2.4 struct Lpspi_Ip_FastTransferType	131
6.2.2.5 struct Lpspi_Ip_ConfigType	133
6.2.2.6 struct Lpspi_Ip_StateStructureType	133
6.2.3 Types Reference	134
6.2.3.1 Lpspi_Ip_CallbackType	135
6.2.4 Enum Reference	135
6.2.4.1 Lpspi_Ip_EventType	135
6.2.4.2 Lpspi_Ip_ModeType	135
6.2.4.3 Lpspi_Ip_HwStatusType	135
6.2.4.4 Lpspi_Ip_StatusType	136
6.2.4.5 Lpspi_Ip_HalfDuplexType	136
6.2.5 Function Reference	136
6.2.5.1 Lpspi_Ip_Init()	136
6.2.5.2 Lpspi_Ip_DeInit()	137
6.2.5.3 Lpspi_Ip_SyncTransmit()	137
6.2.5.4 Lpspi_Ip_AsyncTransmit()	138
6.2.5.5 Lpspi_Ip_SyncTransmitHalfDuplex()	138
6.2.5.6 Lpspi_Ip_AsyncTransmitHalfDuplex()	139
6.2.5.7 Lpspi_Ip_AsyncTransmitFast()	139
6.2.5.8 Lpspi_Ip_GetStatus()	141
6.2.5.9 Lpspi_Ip_ManageBuffers()	141
6.2.5.10 Lpspi_Ip_UpdateTransferParam()	141
6.2.5.11 Lpspi_Ip_UpdateFrameSize()	142
6.2.5.12 Lpspi_Ip_UpdateLsb()	142
6.2.5.13 Lpspi_Ip_UpdateByteSwap()	142
6.2.5.14 Lpspi_Ip_UpdateDefaultTransmitData()	143
6.2.5.15 Lpspi_Ip_UpdateTransferMode()	143
6.2.5.16 Lpspi_Ip_Cancel()	144
6.2.6 Variable Documentation	144
6.2.6.1 Lpspi_Ip_DeviceAttributes_SpiExternalDevice_LPSPi0_Instance_0_VS_0	144
6.3 Spi Driver	145
6.3.1 Detailed Description	145
6.3.2 Data Structure Documentation	149
6.3.2.1 struct Spi_SequenceConfigType	149
6.3.2.2 struct Spi_SeqsConfigType	150
6.3.2.3 struct Spi_JobConfigType	151
6.3.2.4 struct Spi_JobsCfgType	153
6.3.2.5 struct Spi_BufferDescriptorType	153
6.3.2.6 struct Spi_HWUnitQueue	153
6.3.2.7 struct Spi_ChannelConfigType	154
6.3.2.8 struct Spi_ChannelsCfgType	154
6.3.2.9 struct Spi_ConfigType	154
6.3.2.10 struct Spi_Ipw_ParamSyncTransmit	157
6.3.2.11 struct Spi_Ipw_ParamAsyncTransmit	157
6.3.2.12 struct Spi_Ipw_ParamDMAFast	157
6.3.2.13 struct Spi_Ipw_IpConfigType	157
6.3.2.14 struct Spi_Ipw_ExternalDeviceType	157
6.3.2.15 struct Spi_HWUnitConfigType	157
6.3.2.16 struct Spi_PhyUnitsConfigType	157
6.3.2.17 struct Spi_ExternalDeviceConfigType	158
6.3.2.18 struct Spi_ExDevicesConfigType	158
6.3.3 Macro Definition Documentation	158
6.3.3.1 SPI_E_PARAM_CHANNEL	158
6.3.3.2 SPI_E_PARAM_JOB	158

6.3.3.3	SPI_E_PARAM_SEQ	158
6.3.3.4	SPI_E_PARAM_LENGTH	159
6.3.3.5	SPI_E_PARAM_UNIT	159
6.3.3.6	SPI_E_PARAM_CONFIG	159
6.3.3.7	SPI_E_UNINIT	159
6.3.3.8	SPI_E_SEQ_PENDING	159
6.3.3.9	SPI_E_SEQ_IN_PROCESS	160
6.3.3.10	SPI_E_ALREADY_INITIALIZED	160
6.3.3.11	SPI_E_CONFIG_OUT_OF_RANGE	160
6.3.3.12	SPI_E_INIT_FAILED	160
6.3.3.13	SPI_E_PARAM_EB_UNIT	160
6.3.3.14	SPI_E_SEQ_EMPTY	161
6.3.3.15	SPI_E_JOB_EMPTY	161
6.3.3.16	SPI_E_PARAM_POINTER	161
6.3.3.17	SPI_INIT_ID	161
6.3.3.18	SPI_DEINIT_ID	161
6.3.3.19	SPI_WRITEIB_ID	162
6.3.3.20	SPI_ASYNCTRANSMIT_ID	162
6.3.3.21	SPI_READIB_ID	162
6.3.3.22	SPI_SETUPEB_ID	162
6.3.3.23	SPI_GETSTATUS_ID	163
6.3.3.24	SPI_GETJOBRESULT_ID	163
6.3.3.25	SPI_GETSEQUENCERESULT_ID	163
6.3.3.26	SPI_GETVERSIONINFO_ID	163
6.3.3.27	SPI_SYNCTRANSMIT_ID	164
6.3.3.28	SPI_GETHWUNITSTATUS_ID	164
6.3.3.29	SPI_CANCEL_ID	164
6.3.3.30	SPI_SETASYNCMODE_ID	164
6.3.3.31	SPI_MAINFUNCTION_HANDLING_ID	165
6.3.3.32	SPI_SETHWUNITASYNCMODE_ID	165
6.3.3.33	SPI_SETCLOCKMODE_ID	165
6.3.3.34	SPI_JOB_PRIORITY_LEVELS_COUNT	165
6.3.3.35	NUMBER_OF_DATA_TYPE_MAX	165
6.3.4	Types Reference	165
6.3.4.1	Spi_DataBufferType	166
6.3.4.2	Spi_NumberOfDataType	166
6.3.4.3	Spi_ChannelType	166
6.3.4.4	Spi_JobType	166
6.3.4.5	Spi_SequenceType	166
6.3.4.6	Spi_HWUnitType	166
6.3.4.7	Spi_ExternalDeviceType	167
6.3.5	Enum Reference	167
6.3.5.1	Spi_StatusType	167
6.3.5.2	Spi_JobResultType	167
6.3.5.3	Spi_SeqResultType	167
6.3.5.4	Spi_BufferType	168
6.3.5.5	Spi_AsyncModeType	168
6.3.5.6	Spi_HalfDuplexModeType	168
6.3.5.7	Spi_Ipw_SupportedIpsType	169
6.3.6	Function Reference	169
6.3.6.1	Spi_GetVersionInfo()	169
6.3.6.2	Spi_Init()	169
6.3.6.3	Spi_DeInit()	170
6.3.6.4	Spi_WriteIB()	170

6.3.6.5 Spi_ReadIB()	171
6.3.6.6 Spi_AsyncTransmit()	172
6.3.6.7 Spi_SetupEB()	172
6.3.6.8 Spi_GetStatus()	173
6.3.6.9 Spi_GetJobResult()	174
6.3.6.10 Spi_GetSequenceResult()	174
6.3.6.11 Spi_SyncTransmit()	175
6.3.6.12 Spi_GetHWUnitStatus()	175
6.3.6.13 Spi_Cancel()	176
6.3.6.14 Spi_SetAsyncMode()	177
6.3.6.15 Spi_SetHWUnitAsyncMode()	177
6.3.6.16 Spi_MainFunction_Handling()	178

Chapter 1

Revision History

Revision	Date	Author	Description
1.0	07.03.2025	NXP RTD Team	S32K1_S32M24X Real-Time Drivers AUTOSAR R21-11 Version 3.0.0

Chapter 2

Introduction

- [Supported Derivatives](#)
- [Overview](#)
- [About This Manual](#)
- [Acronyms and Definitions](#)
- [Reference List](#)

This User Manual describes NXP Semiconductor AUTOSAR Serial Peripheral Interface (SPI) for S32K1_S32M24X.

AUTOSAR SPI driver configuration parameters and deviations from the specification are described in SPI Driver chapter of this document. AUTOSAR SPI driver requirements and APIs are described in the AUTOSAR SPI driver software specification document.

2.1 Supported Derivatives

The software described in this document is intended to be used with the following microcontroller devices of NXP Semiconductors:

- s32k116_qfn32
- s32k116_lqfp48
- s32k118_lqfp48
- s32k118_lqfp64
- s32k142_lqfp48
- s32k142_lqfp64
- s32k142_lqfp100
- s32k142w_lqfp48
- s32k142w_lqfp64
- s32k144_lqfp48
- s32k144_lqfp64 / MWCT1014S_lqfp64
- s32k144_lqfp100 / MWCT1014S_lqfp100
- s32k144_mapbga100
- s32k144w_lqfp48

- s32k144w_lqfp64
- s32k146_lqfp64
- s32k146_lqfp100 / MWCT1015S_lqfp100
- s32k146_mapbga100 / MWCT1015S_mapbga100
- s32k146_lqfp144
- s32k148_lqfp100
- s32k148_mapbga100 / MWCT1016S_mapbga100
- s32k148_lqfp144
- s32k148_lqfp176
- s32m241_lqfp64
- s32m242_lqfp64
- s32m243_lqfp64
- s32m244_lqfp64

All of the above microcontroller devices are collectively named as S32K1_S32M24X. Note: MWCT part numbers contain NXP confidential IP for Qi Wireless Power

2.2 Overview

AUTOSAR (AUTomotive Open System ARchitecture) is an industry partnership working to establish standards for software interfaces and software modules for automobile electronic control systems.

AUTOSAR:

- paves the way for innovative electronic systems that further improve performance, safety and environmental friendliness.
- is a strong global partnership that creates one common standard: "Cooperate on standards, compete on implementation".
- is a key enabling technology to manage the growing electrics/electronics complexity. It aims to be prepared for the upcoming technologies and to improve cost-efficiency without making any compromise with respect to quality.
- facilitates the exchange and update of software and hardware over the service life of the vehicle.

2.3 About This Manual

This Technical Reference employs the following typographical conventions:

- **Boldface** style: Used for important terms, notes and warnings.
- *Italic* style: Used for code snippets in the text. Note that C language modifiers such "const" or "volatile" are sometimes omitted to improve readability of the presented code.

Notes and warnings are shown as below:

Note

This is a note.

Warning

This is a warning

2.4 Acronyms and Definitions

Term	Definition
API	Application Programming Interface
AUTOSAR	Automotive Open System Architecture
BSMI	Basic Software Make file Interface
CS	Chip Select
DEM	Diagnostic Event Manager
DET	Development Error Tracer
DMA	Direct Memory Access
ECU	Electronic Control Unit
FIFO	First In First Out
LSB	Least Significant Bit
MCU	Micro Controller Unit
MIDE	Multi Integrated Development Environment
MSB	Most Significant Bit
N/A	Not Applicable
RAM	Random Access Memory
SIU	Systems Integration Unit
SWS	Software Specification
SPI	Serial Peripheral Interface
XML	Extensible Markup Language
BSW	Basic Software
ISR	Interrupt Service Routine
OS	Operating System
GUI	Graphical User Interface
PB Variant	Post Build Variant
PC Variant	Pre Compile Variant
LT Variant	Link Time Variant
RTD	Real Time Drivers

Term	Definition
API	Application Programming Interface
AUTOSAR	Automotive Open System Architecture
BSMI	Basic Software Make file Interface
CS	Chip Select
DEM	Diagnostic Event Manager
DET	Development Error Tracer
DMA	Direct Memory Access
ECU	Electronic Control Unit
FIFO	First In First Out
LSB	Least Significant Bit
MCU	Micro Controller Unit
MIDE	Multi Integrated Development Environment
MSB	Most Significant Bit

Term	Definition
N/A	Not Applicable
RAM	Random Access Memory
SIU	Systems Integration Unit
SWS	Software Specification
SPI	Serial Peripheral Interface
XML	Extensible Markup Language
BSW	Basic Software
ISR	Interrupt Service Routine
OS	Operating System
GUI	Graphical User Interface
PB Variant	Post Build Variant
PC Variant	Pre Compile Variant
LT Variant	Link Time Variant

2.5 Reference List

#	Title	Version
1	Specification of Driver	AUTOSAR Release R21-11
2	S32K1xx Reference Manual	Rev.14.1, 01/2024
3	S32M24x Reference Manual	Rev. 5, 12/2024
4	S32K1xx Data Sheet	Rev. 14, 08/2021
5	S32M2xx Data Sheet	Rev. 7, 12/2024
6	S32K116 Errata	Mask Set Errata for Mask 0N96V Rev. 17/JAN/2024, 1/2024
7	S32K118 Errata	Mask Set Errata for Mask 0N97V Rev. 15/JAN/2024, 1/2024
8	S32K142 Errata	Mask Set Errata for Mask 0N33V Rev. 15/JAN/2024, 1/2024
9	S32K144 Errata	Mask Set Errata for Mask 0N57U Rev. 15/JAN/2024, 1/2024
10	S32K144W Errata	Mask Set Errata for Mask 0P64A Rev. 03/JUL/2024, 7/2024
11	S32K146 Errata	Mask Set Errata for Mask 0N73V Rev. 15/JAN/2024, 1/2024
12	S32K148 Errata	Mask Set Errata for Mask 0N20V Rev. 15/JAN/2024, 1/2024
13	S32M242 Errata	Mask Set Errata for Mask N33V+P69K Rev.1, 8/2024
14	S32M244 Errata	Mask Set Errata for Mask P64A+P69K Rev.1, 8/2024

Chapter 3

Driver

- [Requirements](#)
- [Driver Design Summary](#)
- [Hardware Resources](#)
- [Deviations from Requirements](#)
- [Driver Limitations](#)
- [Driver usage and configuration tips](#)
- [Runtime errors](#)
- [Symbolic Names Disclaimer](#)
- [Usage in non AUTOSAR context](#)

3.1 Requirements

Requirements for this driver are detailed in the AUTOSAR R21-11 Rev0000 SPI Driver Software Specification document (See Table [Reference List](#)).

AUTOSAR deviations from requirements are described in [Deviations from Requirements](#) chapter of this document..

3.2 Driver Design Summary

The SPI Handler and Driver provide services for reading from and writing to devices connected via SPI busses. It provides access to SPI communication to several users (e.g., EEPROM, Watchdog, I/O ASICs). It also provides the required mechanism to configure the on-chip SPI peripheral.

This specification describes the API, Mapping to SWS requirements for a monolithic SPI Handler and Driver. This software module includes handling and driving functionalities. Main objectives of this monolithic SPI Driver are to take the best of each microcontroller features and to allow implementation optimization depending on static configuration to fit as much as possible to ECU needs.

The general behavior of the SPI Handler and Driver could be asynchronous or synchronous according to the level of functionality selected.

The specification covers the Handler and Driver functionalities combined in one single module. The SPI handler controls multiple accesses to busses that could be located in the ECU Abstraction layer. The other part is the SPI driver that accesses the microcontroller hardware directly that could be located in the Microcontroller Abstraction layer.

SPI Dual Clock Mode

The SPI Driver allows to be used in Dual Clock Mode. This mode permits to change the clock reference(referred by the field SpiPhyUnitAlternateClockRef) and to keep the basic characteristics of the transmission(like baudrate). This is useful when it wants to be crossed to a low frequency(low power) or higher frequency.

Notification usage

To be able to use the SPI driver with DMA functionality, the following function need to be set as notification for the DMA channels used: Lpspi_Ip_LPSPi_X_IrqTxDmaHandler with Lpspi_Ip_LPSPi_X_IrqRxDmaHandler, Flexio_Spi_Ip_FLEXIO_SPI_X_IrqTxDmaHandler with Flexio_Spi_Ip_FLEXIO_SPI_X_IrqRxDmaHandler, and/or Dspi_Ip_DSPi_X_IRQHandler where X is the SPI unit used (eg: Lpspi_Ip_LPSPi_0_IrqTxDmaHandler if LPSPi0 is used).

Interrupt request usage

Every interrupt is guarded by #ifdef definitions that specify if the corresponded SPI is used. If not, the interrupt function is removed. A template of the #ifdef guard is: `#if (<SPI_X_ENABLED> == STD_ON)`

`<ISR_function_name()>`

`#endif`

Description of the symbolic names

When the plugin is generated, symbolic names of the sequences, jobs and channels are created by define macros. The templates of the defines are:

- for sequences: `#define SpiConf_SpiSequence_<SpiSequenceName> ((Spi_SequenceType)<SpiSequenceID>)`
- for jobs: `#define SpiConf_SpiJob_<SpiJobName> ((Spi_JobType)<SpiJobID>)`
- for channels: `#define SpiConf_SpiChannel_<SpiChannelName> ((Spi_ChannelType)<SpiChannelID>)`

Name is the name of the container and the ID is configurable by the user

The SPI Handler and Driver provide services for reading from and writing to devices connected via SPI busses. It provides access to SPI communication to several users (e.g., EEPROM, Watchdog, I/O ASICs). It also provides the required mechanism to configure the on-chip SPI peripheral.

This specification describes the API, Mapping to SWS requirements for a monolithic SPI Handler and Driver. This software module includes handling and driving functionalities. Main objectives of this monolithic SPI Driver are to take the best of each microcontroller features and to allow implementation optimization depending on static configuration to fit as much as possible to ECU needs.

The general behavior of the SPI Handler and Driver could be asynchronous or synchronous according to the level of functionality selected.

The specification covers the Handler and Driver functionalities combined in one single module. The SPI handler controls multiple accesses to busses that could be located in the ECU Abstraction layer. The other part is the SPI driver that accesses the microcontroller hardware directly that could be located in the Microcontroller Abstraction layer.

SPI Dual Clock Mode

The SPI Driver allows to be used in Dual Clock Mode. This mode permits to change the clock reference(referred by the field SpiPhyUnitAlternateClockRef) and to keep the basic characteristics of the transmission(like baudrate). This is useful when it wants to be crossed to a low frequency(low power) or higher frequency.

Notification usage

To be able to use the SPI driver with DMA functionality, the following function need to be set as notification for the DMA channels used: Lpspi_Ip_LPSPi_X_IrqTxDmaHandler, Flexio_Spi_Ip_FLEXIO_SPI_X_IrqTxDmaHandler and Lpspi_Ip_LPSPi_X_IrqRxDmaHandler, Flexio_Spi_Ip_FLEXIO_SPI_0_IrqRxDmaHandler where X is the SPI unit used (eg: Lpspi_Ip_LPSPi_0_IrqTxDmaHandler if LPSPi0 is used).

Interrupt request usage

Every interrupt is guarded by `#ifdef` definitions that specify if the corresponded SPI is used. If not, the interrupt function is removed. A template of the `#ifdef` guard is: `#if (<SPI_X_ENABLED> == STD_ON)`

```
<ISR_function_name()>
```

```
#endif
```

Description of the symbolic names

When the plugin is generated, symbolic names of the sequences, jobs and channels are created by define macros. The templates of the defines are:

- for sequences: `#define SpiConf_SpiSequence_<SpiSequenceName> ((Spi_SequenceType)<SpiSequenceID>)`
- for jobs: `#define SpiConf_SpiJob_<SpiJobName> ((Spi_JobType)<SpiJobID>)`
- for channels: `#define SpiConf_SpiChannel_<SpiChannelName> ((Spi_ChannelType)<SpiChannelID>)`

Name is the name of the container and the ID is configurable by the user

3.3 Hardware Resources

The hardware configured by the Spi driver is the same between derivatives.

SPI Physical Units: Has a different on number instance between each derivatives of the chip S32K1XX

- Only S32K116 has LPSPI_0, FLEXIO_SPI_0 and FLEXIO_SPI_1 hardware instances.
- Both S32K118 and S32K142 have LPSPI_0, LPSPI1, FLEXIO_SPI_0 and FLEXIO_SPI_1 hardware instances.
- And the rest S32k142w, S32K144, S32K144w, S32K146 and S32K148 have LPSPI_0, LPSPI_1, LPSPI_2, FLEXIO_SPI_0, FLEXIO_SPI_1 hardware instances.

Note

In EB tresos, SPI Physical Unit has selected by SpiPhyUnitMapping.

SpiPhyUnit



Figure 3.1 SPI Physical Unit has selected by SpiPhyUnitMapping in EB Tresos

For example with the chip S32K1XX:

The LPSPI_0, LPSPI_1 and LPSPI_2 use the pins correspondingly with naming is LPSPI0, LPSPI1, LPSPI2. The FLEXIO_SPI_0 and FLEXIO_SPI_1 use the pins correspondingly with naming is FXIO. The pins can find in the file IO Signal Description for each derivatives (e.g. "S32K148_IO_Signal_Description_Input_Multiplexing.xlsx" from attached file of S32K1XX Reference Manual Reference List.

LPSPI1 can be found in the xlsx file with naming is LPSPI1. And the Pin-Muxing is:

LPSPI1	LPSPI1_PCS0	1	0000_0011	IO_PAD	PTD3	D10	101	122	70
		2	0000_0101	IO_PAD	PTE1	A4	137	165	93
		3	0000_0100	IO_PAD	PTA21			6	
		4	0000_0011	IO_PAD	PTA26		19	26	
		5	-	-	disable low				
LPSPI1	LPSPI1_PCS1	1	0000_0011	IO_PAD	PTA6	G9	85	104	58
		2	0000_0100	IO_PAD	PTB18		36	43	
		3	0000_0100	IO_PAD	PTA22			9	
		4	-	-	disable low				
LPSPI1	LPSPI1_SCK	1	0000_0011	IO_PAD	PTD0	C2	4	8	4
		2	0000_0011	IO_PAD	PTB14	F10	96	116	66
		3	0000_0100	IO_PAD	PTA19			2	
		4	0000_0011	IO_PAD	PTA28		24	31	
		5	-	-	disable low				
LPSPI1	LPSPI1_SIN	1	0000_0011	IO_PAD	PTB15	F9	95	115	65
		2	0000_0011	IO_PAD	PTD1	B1	3	7	3
		3	0000_0100	IO_PAD	PTA20			3	
		4	0000_0101	IO_PAD	PTA29		26	33	
		5	-	-	disable low				
LPSPI1	LPSPI1_SOUT	1	0000_0011	IO_PAD	PTD2	D9	102	123	71
		2	0000_0011	IO_PAD	PTB16	F8	94	114	64
		3	0000_0101	IO_PAD	PTE0	B4	138	166	94
		4	0000_0100	IO_PAD	PTA18			1	
		5	0000_0011	IO_PAD	PTA27		22	29	
		6	-	-	disable low				

Figure 3.2 IO Signal Description for LPSPI_1

3.4 Deviations from Requirements

The driver deviates from the AUTOSAR Spi Driver software specification in some places. The table identifies the AUTOSAR requirements that are not fully implemented, implemented differently, not available, not testable or out of scope for the Spi Driver.

Term	Definition
N/S	Out of scope
N/I	Not implemented
N/F	Not fully implemented

Below table identifies the AUTOSAR requirements that are not fully implemented, implemented differently, not available, not testable or out of scope for the driver.

Table 3.2 Driver Deviations Table

Requirement	Status	Description	Notes
SWS_Spi_00040	N/S	The SPI Handler/Driver handles only the Master mode.	Both Master and Slave are supported by SPI driver.
SWS_Spi_00342	N/S	Depending on microcontrollers, the SPI peripheral could share registers with other peripherals. In this typical case, the SPI Handler/Driver has a relationship with MCU module [REF] for initialising and de-initialising those registers.	There is no register that is shared with other peripherals.

Requirement	Status	Description	Notes
SWS_Spi_00270	N/S	In case call end notification function and rescheduling are fully done by software, the order between these shall be first scheduling and then the call of end notification function executed.	Job and sequences notifications are performed before the scheduling of the next job (contrary to the recommendation given by SPI270) . In this way, calls like Spi_SetupIB() or Spi_WriteIB() can be targeted on the next schedulable jobs, before the starting of the job transfer.
SWS_Spi_00195	N/S	SPI Handler/driver shall be able to detect the error SPI_E↔_HARDWARE_ERROR when an hardware error occur during asynchronous or synchronous transmit. Please see also SW↔S_Spi_00267 and SWS_Spi_↔00384.	SPI_E_HARDWARE_↔ERROR is not supported for Spi_Async_Transmit() Function. It is supported for Spi_Sync_Transmit() function. To implement this requirement a timer should be set to that estimated value and if the timer expires, then it can be assumed that a hardware error occurred. This would add a dependency of Gpt.

Requirement	Status	Description	Notes
SWS_Spi_00383	N/S	Error Name: SPI_E_HARDWARE_ERROR Short Description: An hardware error occurred during asynchronous or synchronous SPI transmit. Long Description: This Extended Production Error shall be issued when any error bit inside the SPI hardware transmit status register is raised Detection Criteria: Fail The SPI transmit status register information shall be reported to DEM as Dem_ReportErrorStatus (SPI_E_HARDWARE_ERROR, DEM_EVENT_STATUS_FAILED) when any error bit inside the SPI transmit status register is set. (SWS_Spi_00385) Pass The SPI transmit status register information shall be reported to DEM as Dem_ReportErrorStatus (SPI_E_HARDWARE_ERROR, DEM_EVENT_STATUS_PASSED) when no error bit inside the SPI transmit status register is set. (SWS_Spi_00386)	SPI_E_HARDWARE_ERROR is not supported for Spi_Async_Transmit() Function. It is supported for Spi_Sync_Transmit() function. To implement this requirement a timer should be set to that estimated value and if the timer expires, then it can be assumed that a hardware error occurred. This would add a dependency of Gpt.
SWS_Spi_00385	N/S	When any error bit inside the SPI transmit status register is set, the SPI transmit status register information shall be reported to DEM as Dem_ReportErrorStatus (SPI_E_HARDWARE_ERROR, DEM_EVENT_STATUS_FAILED)	SPI_E_HARDWARE_ERROR is not supported for Spi_Async_Transmit() Function. It is supported for Spi_Sync_Transmit() function. To implement this requirement a timer should be set to that estimated value and if the timer expires, then it can be assumed that a hardware error occurred. This would add a dependency of Gpt.

Requirement	Status	Description	Notes
SWS_Spi_00386	N/S	When no error bit inside the S←PI transmit status register is set, the SPI transmit status register information shall be reported to DEM as Dem_ReportError←Status (SPI_E_HARDWAR←E_ERROR, DEM_EVENT←STATUS_PASSED)	SPI_E_HARDWARE←ERROR is not supported for Spi_Async_Transmit() Function. It is supported for Spi_Sync_Transmit() function. To implement this requirement a timer should be set to that estimated value and if the timer expires, then it can be assumed that a hardware error occurred. This would add a dependency of Gpt.
SWS_Spi_00293	N/S	When the function Spi_Async←Transmit is called, the SPI Handler/Driver shall handle the Job results. Result shall be SPI_JOB_FAILED when the transmission of Jobs is failed.	The Spi_AsyncTransmit can only schedule Jobs to be sent. So the function itself cannot detect if a job is failed.
SWS_Spi_00999	N/S	These requirements are not applicable to this specification.	This is not a requirement.
ECUC_Spi_00239	N/S	When the Chip select handling is enabled (see SpiEnableCs), then this parameter specifies if the chip select is handled automatically by Peripheral HW engine or via general purpose IO by Spi driver.	If user selects parameter CS←_VIA_GPIO, the user has to use SpiJobStartNotification' & 'SpiJobEndNotification' to toggle the CS (chip select pin) using DIO drivers .
SWS_Spi_CONSTR_00001	N/S	DRATF: The ECUC partitions referenced by SpiKernelEcuc←PartitionRef shall be a subset of the ECUC partitions referenced by SpiEcucPartitionRef.	Type IV Autosar MultiPartition not implemented for current module. MultiPartition Type II is implemented. AAI-445.

3.5 Driver Limitations

3.5.1 Some LPSPI's features are not supported:

- Receive data match logic supporting wakeup on data match
- Half duplex feature in receive mode doesn't support above 32 bits transfer and continuous CS.

3.5.2 Limitation of LPSPI Slave mode:

- In Slave mode and use interrupt mode without DMA, the application needs to make sure Slave's interrupt service is not delayed to avoid errors underflow and overflow occur.

3.5.3 Some limitations of SPI over FLEXIO:

- Do not support CS Non-continuous mode.
- Do not support the polarity of CS is HIGH (only LOW is supported).

- "SpiCsIdentifier" (in SpiExternalDevice) is not used to select the no. of CS. It will be configured by "Spi↔FlexioRxAndCsChannelsConfig" in "SpiPhyUnit". Please refer "How to configure SPI over FLEXIO" section for more details
- SPI FLEXIO does not support to configure the timing delay.
- In DMA mode, Spi over FLEXIO does not support MSB mode in both transmission and reception modes except data width differs to 8, 16 or 32 bits.
- Spi over FLEXIO does not support above 32 bits transfer.

3.5.4 Some LPSPI's features are not supported:

- Receive data match logic supporting wakeup on data match.

3.5.5 Some limitations of SPI over FLEXIO:

- For Flexio, don't use these configurations: (CS_KEEP_ASSERTED, SPI_MASTER, TRAILING), (CS_KEEP_ASSERTED, SPI_SLAVE, LEADING, SpiShiftClockIdleLevel = HIGH), (SPI_SLAVE, TRAILING).
- "SpiCsIdentifier" (in SpiExternalDevice) is not used to select the no. of CS. It will be configured by "Spi↔FlexioRxAndCsChannelsConfig" in "SpiPhyUnit". Please refer "How to configure SPI over FLEXIO" section for more details.
- SPI FLEXIO don't support to configure the timing delay.

3.6 Driver usage and configuration tips

3.6.1 How to configure handling Chip Select via general purpose IO (SpiCsSelection: CS_VIA_GPIO)

The chip select is handled automatically by Peripheral HW engine or via general purpose IO by Spi driver. In the case of the hardware does not support to keep chip select asserted between frame transfers. The user can use the CS_VIA_GPIO feature (selected by node SpiCsSelection) and the driver will call functions notification(defined by nodes SpiJobStartNotification and SpiJobEndNotification) to control CS pin via GPIO for each Job. By this way, SPI driver can communication with external device requires Continuous CS.

The configuration steps use the CS_VIA_GPIO feature as below:

- Select the CS_VIA_GPIO feature by node SpiCsSelection

SpiExternalDevice

Name  SpiExternalDevice_SJA1105Q

General SpiDeviceEcucPartitionRef

SpiBaudrate (4 -> 15151515)	 500000.0 
 SpiBaudrateFlexio1 (172000 -> 22000000)	 200000.0
 SpiBaudrateFlexio2 (107500 -> 1375000)	 107500.0
 SpiBaudrateFlexio3 (670 -> 172000)	 1000.0
SpiCsIdentifier	 PCS0 
SpiCsPolarity	 LOW 
 SpiCsSelection	 CS_VIA_GPIO 
SpiDataShiftEdge	 LEADING 
SpiEnableCs	 ☒ 
SpiHwUnit	 CSIB0 
SpiShiftClockIdleLevel	 LOW 

Figure 3.3 Select the CS_VIA_GPIO feature

- Enable Job Start Notification by node SpiJobStartNotificationEnable

▼ SpiAutosarExt

Name  SpiAutosarExt

SpiEnableUserModeSupport	 ☐	SpiEnableDmaFastTransferSupport*	 ☐ 
SpiHalfDuplexModeSupport*	 ☐ 	SpiAllowBigSizeCollections	 ☐ 
SpiEnableHWUnitAsyncMode*	 ☐ 	SpiJobStartNotificationEnable*	 ☒ 
Spi Disable Production Error Reporting	 ☐ 	SpiFlexioEnable*	 ☐ 

Figure 3.4 Enable Job Start Notification on SpiAutosarExt form

- Define two functions Job Start Notification and Job End Notification. Ex: Start_Job_Notification_Function, End_Job_Notification_Function. Start_Job_Notification_Function function will must assert CS via GPIO. End_Job_Notification_Function will must de-assert CS via GPIO. After that, enter name of two functions on nodes SpiJobStartNotification and SpiJobEndNotification.

SpiJob

Name  SpiJob_0

General SpiChannelList

 SpiJobEndNotification		Start_Job_Notification_Function		▼
 SpiJobStartNotification		End_Job_Notification_Function		▼
SpiJobId		0		▼
SpiJobPriority		0		▼
SpiDeviceAssignment		/Spi/Spi/EthSwt_SpiDriver/SpiExternalDevice_0		

Figure 3.5 Enter name of two functions SpiJobStartNotification and SpiJobEndNotification on SpiJob form

3.6.2 How to configure in multiple post build variants

To use multiple post build variants, the configurations need to have the same symbolic name. So, all Names and IDs of Sequences, Jobs, Channels must be the same between all post build variants. Names and Index of ExternalDevices must be the same between all post build variants.

Scenario:

Let's assume there are 2 post build variant configurations, VS_0 and VS_1.

- VS_0 defines 2 Sequences, 2 Jobs, 2 Channels and 2 ExternalDevices:
- Name of sequence is SEQ_DSPI0_1J_C0_1 and SpiSequenceId is 0.
- Name of job is JOB_DSPI0_C0_1 and SpiJobId is 0.
- Name of ExternalDevice is DEV_EXP_100K_LEAD_0 and Index is 0.
- Name of channel is CH_EB_10K and SpiChannelId is 0.
- Name of sequence is SEQ_DSPI1_1J_C0_2 and SpiSequenceId is 1.
- Name of job is JOB_DSPI1_C0_2 and SpiJobId is 1.
- Name of ExternalDevice is DEV_EXP_100K_LEAD_1 and Index is 1.
- Name of channel is CH_EB_1K and SpiChannelId is 1.

So, VS_1 must have configuration the same Names, IDs and Indexes with VS_0 for Sequences, Jobs, Channels and ExternalDevices.

- VS_1 defines 2 Sequences, 2 Jobs, 2 Channels and 2 ExternalDevices:

- Name of sequence is SEQ_DSPIO_1J_C0_1 and SpiSequenceId is 0.
- Name of job is JOB_DSPIO_C0_1 and SpiJobId is 0.
- Name of ExternalDevice is DEV_EXP_100K_LEAD_0 and Index is 0.
- Name of channel is CH_EB_10K and SpiChannelId is 0.
- Name of sequence is SEQ_DSPI1_1J_C0_2 and SpiSequenceId is 1.
- Name of job is JOB_DSPI1_C0_2 and SpiJobId is 1.
- Name of ExternalDevice is DEV_EXP_100K_LEAD_1 and Index is 1.
- Name of channel is CH_EB_1K and SpiChannelId is 1.

The generated symbolic names for VS_0 and VS_1 will be: *#define SpiConf_SpiSequence_SEQ_DSPIO_1J_C0_1 0*

#define SpiConf_SpiSequence_SEQ_DSPI1_1J_C0_2 1

#define SpiConf_SpiJob_JOB_DSPIO_C0_1 0

#define SpiConf_SpiJob_JOB_DSPI1_C0_2 1

#define SpiConf_SpiChannel_CH_EB_10K 0

#define SpiConf_SpiChannel_CH_EB_1K 1

It allows the upper layer to use same channel symbolic name(s) across multiple configurations.

3.6.3 How to configure MultiPartition

To enable MultiPartition Support, SpiGeneral/SpiMultipartitionSupport must be set to "True".



Figure 3.6 Enable MultiPartition Support

The partition for each HWUnit will be select by ExternalDevice. Each ExternalDevice select only a partition (reference from ECUC). Note: Alls ExternalDevice which selected same HWUnit need to assigned to same partition.

SpiExternalDevice

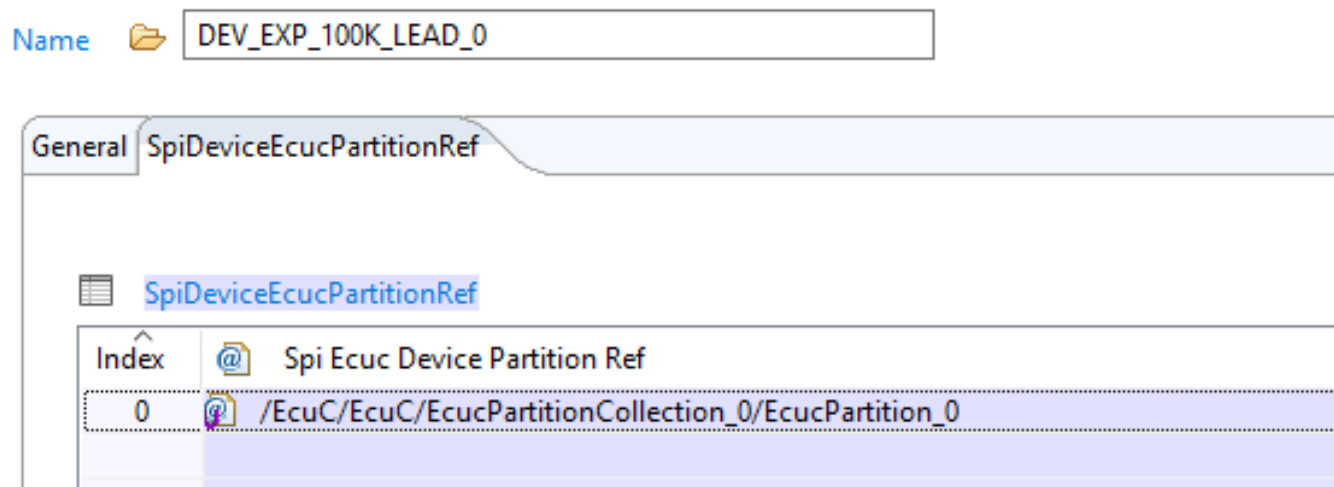


Figure 3.7 Partition for each ExternalDevice

Each partition will refer to a core by OS.

OsApplication

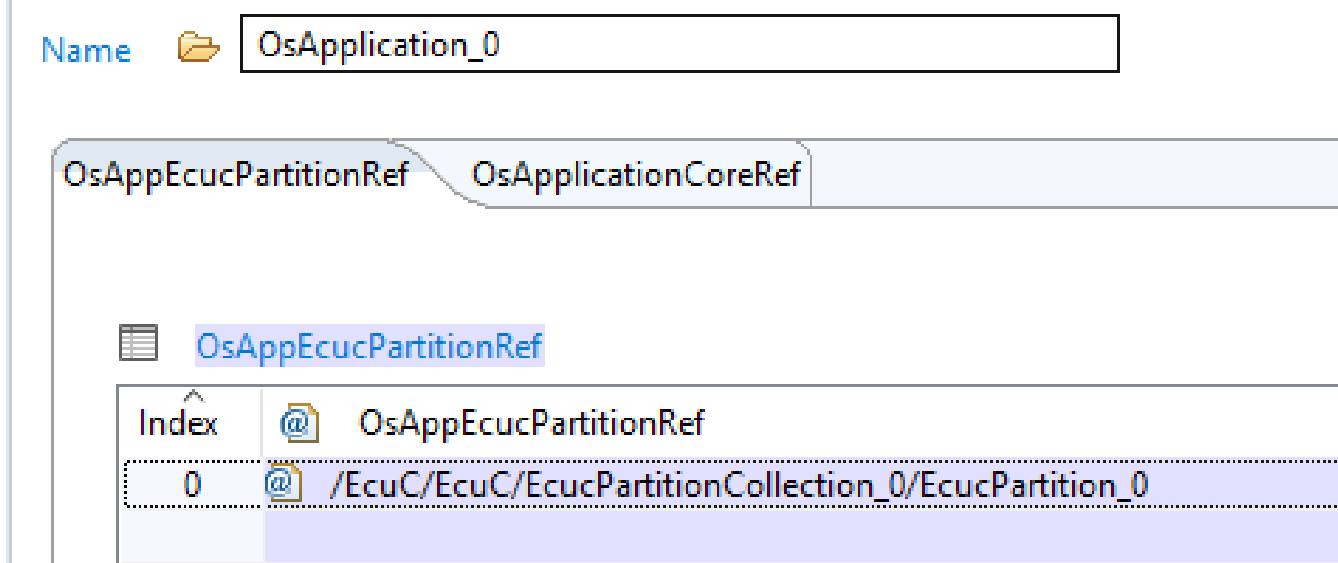


Figure 3.8 Partition is selected by OsApplication_0

3.6.4 How to config interrupts for the transmission of Master and Slave on Spi examples in the same chip

Prefer to use either Master or Slave in Dma mode to avoid interrupt overlapping.

- Configure DMA MUX in RM.

Index	Name	Dma Hardware Instance	Dma Hardware Channel	Enable Periodic Trigger ...	Dma Mux Source	Dma Mux Source
0	MUX_SPL_1_TX	DMA_INSTANCE_0	DMA_CHANNEL_3	☐	DMA_MUX_0_SPL_1_TX_FIFO_FULL	DMA_MUX_1_REQ_DISABLED
1	MUX_SPL_1_RX	DMA_INSTANCE_0	DMA_CHANNEL_5	☐	DMA_MUX_0_SPL_1_RX_FIFO_DRAIN	DMA_MUX_1_REQ_DISABLED
2	MUX_SPL_1_TXCMD	DMA_INSTANCE_0	DMA_CHANNEL_1	☐	DMA_MUX_0_SPL_2_CMD_FIFO_FULL	DMA_MUX_1_REQ_DISABLED

Figure 3.9 DMA MUX configuration in RM

3.6.5 How to use dual clock mode

This mode permits to change the clock reference(referred by the field SpiPhyUnitAlternateClockRef) and to keep the basic characteristics of the transmission(like baudrate, delay time). The user can configuration two McuClockSettingConfig are McuClockSettingConfig_Normal and McuClockSettingConfig_Alternate:

Index	Name	Mcu Cloc...	FXOSC u...	FXOSC E...	FXOSC B...	FXOSC C...	FXOSC E...	Crystal o...	FXOSC
0	McuClockSettingConfig_Normal	0	☐	☒	☐	☒	100	0	4
1	McuClockSettingConfig_Alternate	1	☐	☐	☐	☒	157	12	4

Figure 3.10 Configuration two McuClockSettingConfig

The field SpiPhyUnitClockRef will be referred to McuClockSettingConfig_Normal.

SpiPhyUnitClockRef /Mcu/McuModuleConfiguration_0/McuClockSettingConfig_0/McuClockSettingConfig_Normal

Figure 3.11 SpiPhyUnitClockRef will be referred to McuClockSettingConfig_Normal

The field SpiPhyUnitAlternateClockRef will be referred to McuClockSettingConfig_Alternate.

SpiPhyUnitAlternateClockRef /Mcu/Mcu/McuModuleConfiguration_0/McuClockSettingConfig_0/McuClockSettingConfig_Alternate

Figure 3.12 SpiPhyUnitAlternateClockRef will be referred to McuClockSettingConfig_Alternate

The default, Spi driver will use Normal Clock with the clock reference by the field SpiPhyUnitClockRef. If the user changes the clock setting to McuClockSettingConfig_Alternate by the function Mcu_InitClock(McuClockSettingConfig_Alternate), the user also changes clock mode for Spi driver by the function Spi_SetClockMode(SPI_ALTERNATE).

3.6.6 How to configure handling Chip Select via general purpose IO (SpiCsSelection: CS_VIA_GPIO)

The chip select is handled automatically by Pe-ripheral HW engine or via general purpose IO by Spi driver. In the case of the hardware does not support to keep chip select asserted between frame transfers. The user can use the CS_VIA_GPIO feature (selected by node SpiCsSelection) and the driver will call functions notification(defined by nodes SpiJobStartNotification and SpiJobEndNotification) to control CS pin via GPIO for each Job. By this way, SPI driver can communication with external device requires Continuous CS.

The configuration steps use the CS_VIA_GPIO feature as below:

- Select the CS_VIA_GPIO feature by node SpiCsSelection

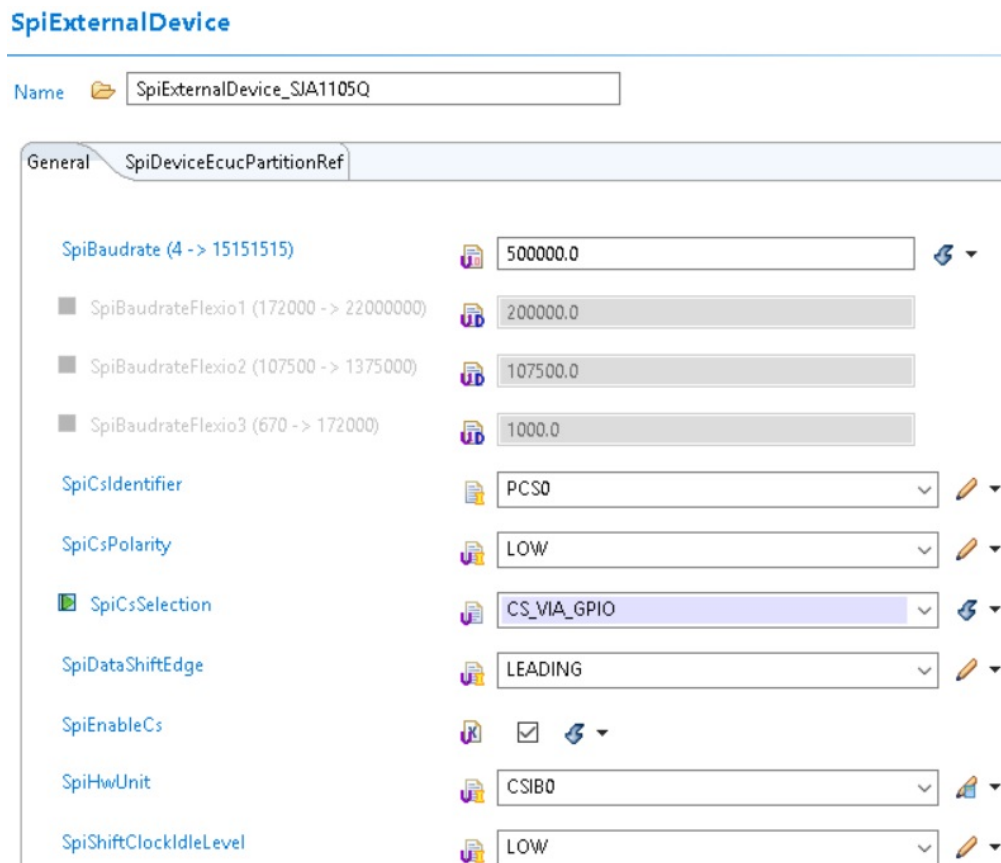


Figure 3.13 Select the CS_VIA_GPIO feature

- Enable Job Start Notification by node SpiJobStartNotificationEnable

SpiAutosarExt

Name SpiAutosarExt

SpiEnableUserModeSupport	☐	SpiEnableDmaFastTransferSupport*	☐
SpiHalfDuplexModeSupport*	☐	SpiAllowBigSizeCollections	☐
SpiEnableHWUnitAsyncMode*	☐	SpiJobStartNotificationEnable* 	☒
Spi Disable Production Error Reporting	☐	SpiFlexioEnable*	☐

Figure 3.14 Enable Job Start Notification on SpiAutosarExt form

- Define two functions Job Start Notification and Job End Notification. Ex: Start_Job_Notification_Function, End_Job_Notification_Function. Start_Job_Notification_Function function will must assert CS via GPIO. End_Job_Notification_Function will must de-assert CS via GPIO. After that, enter name of two functions on nodes SpiJobStartNotification and SpiJobEndNotification.

SpiJob

Name SpiJob_0

General SpiChannelList

SpiJobEndNotification	Start_Job_Notification_Function
SpiJobStartNotification	End_Job_Notification_Function
SpiJobId	0
SpiJobPriority	0
SpiDeviceAssignment	/Spi/Spi/EthSwt_SpiDriver/SpiExternalDevice_0

Figure 3.15 Enter name of two functions SpiJobStartNotification and SpiJobEndNotification on SpiJob form

3.6.7 How to configure in multiple post build variants

To use multiple post build variants, the configurations need to have the same symbolic name. So, all Names and IDs of Sequences, Jobs, Channels must be the same between all post build variants. Names and Index of ExternalDevices must be the same between all post build variants.

Scenario:

Let's assume there are 2 post build variant configurations, VS_0 and VS_1.

- VS_0 defines 2 Sequences, 2 Jobs, 2 Channels and 2 ExternalDevices:
- Name of sequence is SEQ_DSPIO_1J_C0_1 and SpiSequenceId is 0.
- Name of job is JOB_DSPIO_C0_1 and SpiJobId is 0.
- Name of ExternalDevice is DEV_EXP_100K_LEAD_0 and Index is 0.
- Name of channel is CH_EB_10K and SpiChannelId is 0.
- Name of sequence is SEQ_DSPI1_1J_C0_2 and SpiSequenceId is 1.
- Name of job is JOB_DSPI1_C0_2 and SpiJobId is 1.
- Name of ExternalDevice is DEV_EXP_100K_LEAD_1 and Index is 1.
- Name of channel is CH_EB_1K and SpiChannelId is 1.

So, VS_1 must have configuration the same Names, IDs and Indexes with VS_0 for Sequences, Jobs, Channels and ExternalDevices.

- VS_1 defines 2 Sequences, 2 Jobs, 2 Channels and 2 ExternalDevices:
- Name of sequence is SEQ_DSPIO_1J_C0_1 and SpiSequenceId is 0.
- Name of job is JOB_DSPIO_C0_1 and SpiJobId is 0.
- Name of ExternalDevice is DEV_EXP_100K_LEAD_0 and Index is 0.
- Name of channel is CH_EB_10K and SpiChannelId is 0.
- Name of sequence is SEQ_DSPI1_1J_C0_2 and SpiSequenceId is 1.
- Name of job is JOB_DSPI1_C0_2 and SpiJobId is 1.
- Name of ExternalDevice is DEV_EXP_100K_LEAD_1 and Index is 1.
- Name of channel is CH_EB_1K and SpiChannelId is 1.

The generated symbolic names for VS_0 and VS_1 will be: *#define SpiConf_SpiSequence_SEQ_DSPIO_1J_C0_1 0*

#define SpiConf_SpiSequence_SEQ_DSPI1_1J_C0_2 1

#define SpiConf_SpiJob_JOB_DSPIO_C0_1 0

#define SpiConf_SpiJob_JOB_DSPI1_C0_2 1

#define SpiConf_SpiChannel_CH_EB_10K 0

#define SpiConf_SpiChannel_CH_EB_1K 1

It allows the upper layer to use same channel symbolic name(s) across multiple configurations.

3.6.8 How to configure Dma Fast transfer for a Sequence

The SPI driver can be able to transfer a Sequence with multiple Channels, Jobs without any CPU intervention between the channels and jobs. The CPU is used only to start the transmission (Spi_AsyncTransmit) and to process end of sequence notification. The driver will use DMA Scatter/Gather feature for all 2 DMA channels SpiPhyTxDmaChannel and SpiPhyRxDmaChannel. SpiPhyTxDmaChannel will fill data to TCR and TDR registers. SpiPhyRxDmaChannel will read data from RDR register.

This feature requires:

- The parameters SpiBaudrate, SpiHwUnit, SpiTimeClk2Cs, SpiTimeCs2Clk, SpiTimeCs2Cs in External Device linked to each Job in this Sequence must be the same.
- The parameters SpiDataWidth and SpiTransferStart in Channel assigned to each Job in this Sequence must be the same.
- In each Channel, the number of data buffers is NOT higher than 32767 if SpiDataWidth < 9. So, SpiIbNBuffers and SpiEbMaxLength must be assigned to suitable values.
- Only Master mode is supported(SpiPhyUnitMode = SPI_MASTER).
- Make sure that SpiMaxDmaFastTransfer value in SpiPhyUnit allocated to this Sequence must NOT lower than total of Channels in this Sequence.
- Make sure that number of ScatterGathers configuration in each SpiPhyUnit/SpiPhyTxDmaChannel must NOT lower than (total of Channels * 2) plus total of Jobs in this Sequence.
- Make sure that number of ScatterGathers configuration in each SpiPhyUnit/SpiPhyRxDmaChannel must NOT lower than total of Channels in this Sequence.
- Only SpiJobStartNotification and SpiJobEndNotification can be supported for first Job in a Sequence. But SpiJobEndNotification will be called at the end of Sequence as SpiSeqEndNotification.

Configuration example for 4 Channels and 2 Jobs in Sequence:

- Enable Dma Fast transfer support.



Figure 3.16 Enable Dma Fast transfer support

- Enable Dma Fast transfer for Sequence.

SpiSequence

Name  SEQ_DSPI1_EXP_2J_C71_67_N

General SpiJobAssignment

SpiInterruptibleSequence  ☐ 

 SpiSeqEndNotification  EndSeq_DSPI1_EXP_2J_C71_67 

SpiSequenceld  0  

SpiEnableDmaFastTransfer  ☒ 

Figure 3.17 Enable Dma Fast transfer for Sequence

- Enable Dma mode and set maximum number of Dma Fast transfers session. Make sure that the value of SpiMaxDmaFastTransfer is higher or equal total number of Channels in a Sequence.

SpiPhyUnit

Name  SpiPhyUnit_0

General

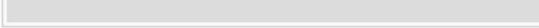
SpiPhyUnitMapping	 LPSPi_2  
SpiPhyUnitMode	 SPI_MASTER  
SpiPhyUnitSync	 ☐ 
SpiSamplePoint (0 -> 1)	 0 
SpiPinConfiguration (0 -> 3)	 0 
SpiPhyUnitClockRef	 /Mcu/Mcu/McuModuleConfiguration/McuClockSettingCor
 SpiPhyUnitAlternateClockRef	 
SpiPhyUnitAsyncUseDma	 ☒ 
 SpiPhyTxDmaChannel	 /Mcl/Mcl/MclConfig/DMA_LPSPi2_TX
 SpiPhyRxDmaChannel	 /Mcl/Mcl/MclConfig/DMA_LPSPi2_RX
 SpiMaxDmaFastTransfer	 4  

Figure 3.18 Enable Dma mode and set maximum number of Dma Fast transfers session

- Configure for SpiPhyTxDmaChannel in MCL.

Logic Channel

Name  SPI_1_TX

Logic Channel Configuration Global Transfer ScatterGather Crc

Logic Channel Name  DMA_LOGIC_CH_3 

Hardware Instance  DMA_IP_HW_INST_0  

Hardware Channel  DMA_IP_HW_CH_3  

Interrupt Callback  NULL_PTR 

Error Interrupt Callback  NULL_PTR 

 Ecuc Partition Ref 

Enable Global Config  ☒  Enable Transfer Config  ☐

Enable Scatter/Gather  ☒  Enable CRC Config  ☐ 

Figure 3.19 Configure for SpiPhyTxDmaChannel in MCL

Logic Channel

Name  SPI_1_TX

Logic Channel Configuration | Global | Transfer | ScatterGather | Crc

▼ dmaLogicChannel_GlobalConfigType

Name  dmaLogicChannel_GlobalConfigType

▼ Control

Name  dmaLogicChannelConfig_GlobalControlType

Enable Master Id Replication  ☐  Enable Buffered Writes  ☐ 

▼ Request

Name  dmaLogicChannelConfig_GlobalRequestType

Enable DMA Request  ☐ 

▼ Interrupt

Name  dmaLogicChannelConfig_GlobalInterruptType

Enable Error Interrupt  ☐ 

▼ Priority

Name  dmaLogicChannelConfig_GlobalPriorityType

Group Priority  DMA_IP_GROUP_PRIO0 

Level Priority  DMA_IP_LEVEL_PRIO0 

Enable Preemption  ☐  Disable Preempt  ☐ 

Figure 3.20 Global configuration for SpiPhyTxDmaChannel in MCL

Number of ScatterGather elements will equal to SpiMaxDmaFastTransfer.

Logic Channel

Name  DMA_LPSPi2_TX

Logic Channel Configuration				
Global				
Transfer				
ScatterGather				
 ScatterGather				
Index	Name	Element Name	Last Element of the Link	
0	TXSG0	DMA_LOGIC_CH_0_SGA_ELEMENT_0		☐
1	TXSG1	DMA_LOGIC_CH_0_SGA_ELEMENT_1		☐
2	TXSG2	DMA_LOGIC_CH_0_SGA_ELEMENT_2		☐
3	TXSG3	DMA_LOGIC_CH_0_SGA_ELEMENT_3		☐
4	TXSG4	DMA_LOGIC_CH_0_SGA_ELEMENT_4		☐
5	TXSG5	DMA_LOGIC_CH_0_SGA_ELEMENT_5		☐
6	TXSG6	DMA_LOGIC_CH_0_SGA_ELEMENT_6		☐
7	TXSG7	DMA_LOGIC_CH_0_SGA_ELEMENT_7		☐
8	TXSG8	DMA_LOGIC_CH_0_SGA_ELEMENT_8		☐
9	TXSG9	DMA_LOGIC_CH_0_SGA_ELEMENT_9		☒

Figure 3.21 ScatterGather configuration for SpiPhyTxDmaChannel in MCL

"Element Link" node must be configured for each ScatterGather element as a continuation sequence: TXSG0->TXSG1->TXSG2->TXSG3->TXSG4->TXSG5->TXSG6->TXSG7->TXSG8->TXSG9.

ScatterGather Element

Name  TXSG0

General

Element

Name  dmaLogicChannelConfig_ScatterGatherElementConfigType

Element Name  DMA_LOGIC_CH_0_SGA_ELEMENT_0

Last Element of the Link  ☐ 

Element Link  @ /Mcl/Mcl/MclConfig/DMA_LPSPi2_TX/dmaLogicChannel_ConfigType/dmaLogicChannel_ScatterGatherConfigType/TXSG1

Enable Scatter/Gather Config  ☐ 

Figure 3.22 Link each ScatterGather element as a continuation sequence in MCL

- Configure for SpiPhyRxDmaChannel in MCL is similar to SpiPhyTxDmaChannel. But "Interrupt Callback"

must be configured.

Logic Channel

Name  SPI_1_RX

Logic Channel Configuration

GlobalTransferScatterGatherCrc

Logic Channel Name

 DMA_LOGIC_CH_5 

Hardware Instance

 DMA_IP_HW_INST_0  

Hardware Channel

 DMA_IP_HW_CH_5  

Interrupt Callback

 Spi_Ip_SPI_1_IrqDmaHandler 

Error Interrupt Callback

 NULL_PTR 

Ecuc Partition Ref

Enable Global Config

 ☒ 

Enable Transfer Config

 ☐

Enable Scatter/Gather

 ☒ 

Enable CRC Config

 ☐ 

Figure 3.23 Configure for SpiPhyRxDmaChannel in MCL

- Notice that "Level Priority" value of SpiPhyRxDmaChannel is higher than SpiPhyTxDmaChannel.
- Configure DMA MUX in RM.

Dma_Mux_Configuration

Name  Dma_Mux_Configuration

Dma Mux Development Error Detection  ☐ 

Module Configurations

Index	Name	Dma Hardware Instance	Dma Hardware Channel	Enable Periodic Trigger ...	Dma Mux Source	Dma Mux Source
0	 MUX_SPI_1_TX	 DMA_INSTANCE_0	 DMA_CHANNEL_3	 ☐	 DMA_MUX_0_SPI_1_TX_FIFO_FULL	 DMA_MUX_1_REQ_DISABLED
1	 MUX_SPI_1_RX	 DMA_INSTANCE_0	 DMA_CHANNEL_5	 ☐	 DMA_MUX_0_SPI_1_RX_FIFO_DRAIN	 DMA_MUX_1_REQ_DISABLED
2	 MUX_SPI_1_TXCMD	 DMA_INSTANCE_0	 DMA_CHANNEL_1	 ☐	 DMA_MUX_0_SPI_2_CMD_FIFO_FULL	 DMA_MUX_1_REQ_DISABLED

Figure 3.24 DMA MUX configuration in RM

3.6.9 How to configure SPI over FLEXIO

First of all, SPI over FLEXIO must use MCL module to enable/disable Flexio module and configure Flexio Channels (Select PIN, Shifter, Timer registers) and ISR handler. SPI can not work by itself. Bellowing is guiding to configure SPI over FLEXIO for both EB tresos and CT configuration tool

Note

Mcl_Init() functions must be called before using SPI over FLEXIO.

3.6.9.1 EB Tresos

- Enable flexio mode first by set SpiFlexioEnable: (bellow picture)

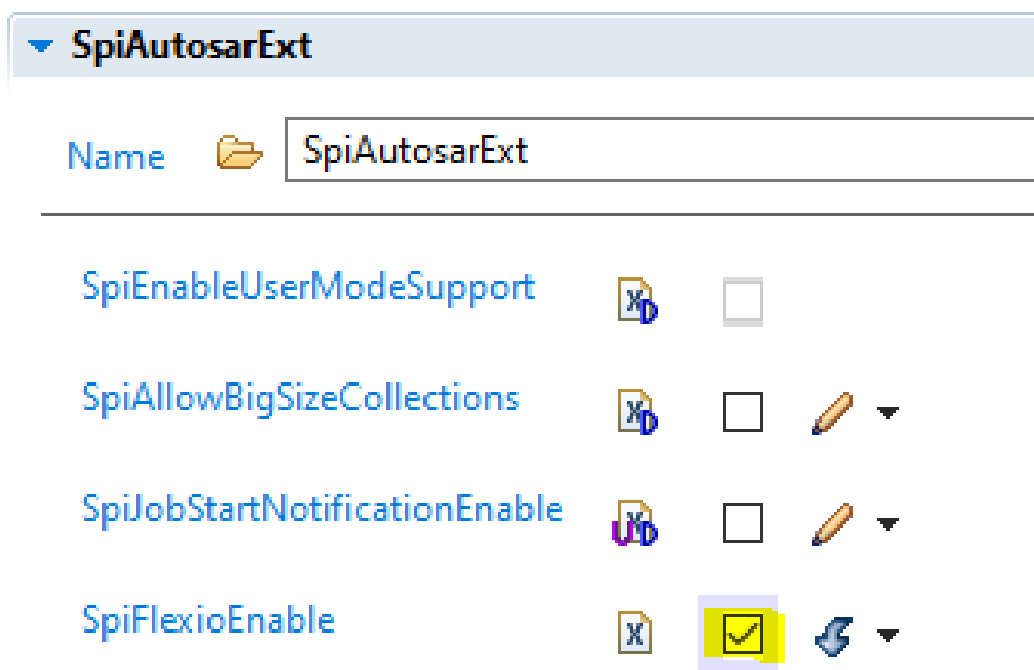


Figure 3.25 Enable SPI over FLEXIO

- Enable Flexio support on MCL site: (bellow picture)

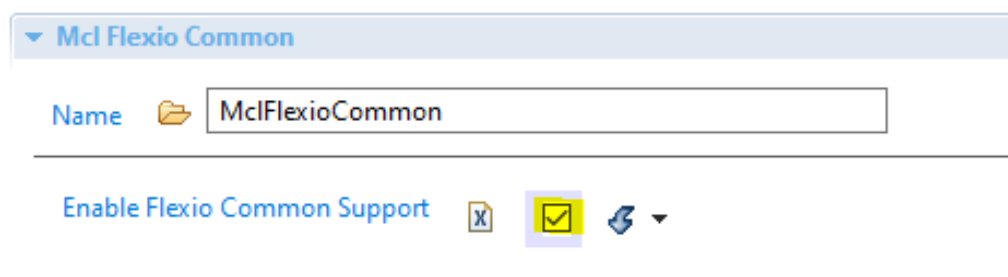


Figure 3.26 Enable Mcl FLEXIO support

- In MCL, setup a set for SPI FLEXIO Channels like: (bellow picture)
- *Flexio Channel*

is no. of Shifter and Timer registers will be selected for each FLEXIO channel. For example: CHANNEL_0 selected means Shifter_0 and Timer_0 will be selected and locked, other channels cannot select those registers.

- *Flexio Pin*

is no. of PIN will be selected for each FLEXIO channel. For example: PIN_0 selected means FLEXIO_D0 will be selected and locked, other channels cannot select this PIN.

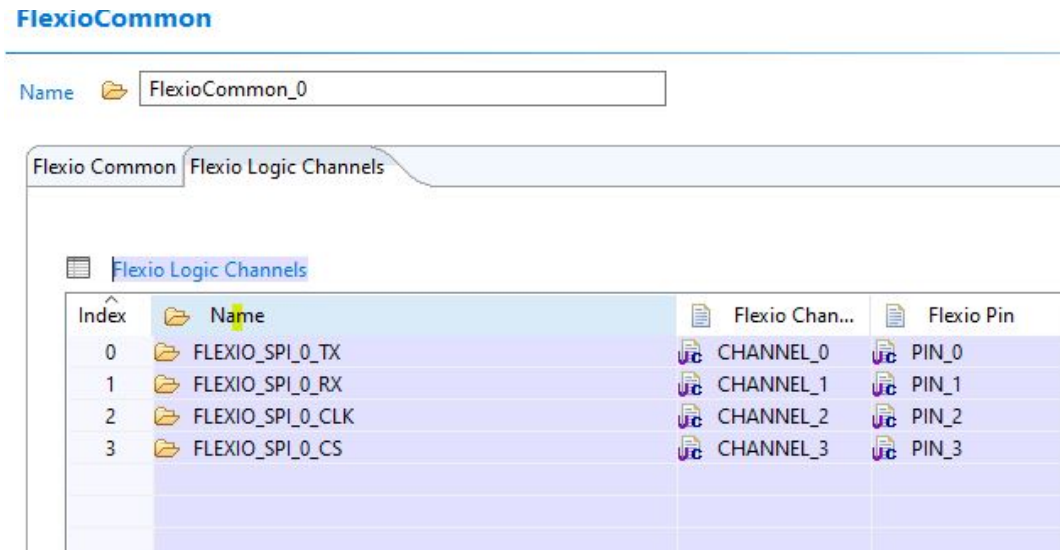


Figure 3.27 A set of FLEXIO SPI channels from MCL

- From SPI, each FLEXIO_SPI_x is selected in SpiPhyUnit then those set must be enabled and selected correctly value (refer from MCL which was configured above) (bellow picture)
 - SpiFlexioTxAndClkChannelsConfig: FLEXIO Logical Channel for TX(MOSI) and CLK(Clock). This selects 1 PIN to setup as TX(MOSI) channel and 1 PIN to set up as CLK channel. If you select CHANNEL_X, PIN_Y(FlexioMclPinId) and PIN_Z(FlexioMclAddPinId) that means:
 - * ShifterX is selected for TX(MOSI) channel
 - * TimerX is selected for CLK channel.
 - * PIN_Y is selected for TX(MOSI) channel
 - * PIN_Z is selected for CLK channel
 - SpiFlexioRxAndCsChannelsConfig: FLEXIO Logical Channel for RX(MISO) and CS(Chip select). This selects 1 PIN to setup as MISO channel and 1 PIN to set up as CS channel. If you select CHANNEL_X, PIN_Y(FlexioMclPinId) and PIN_Z(FlexioMclAddPinId) that mean:
 - * ShifterX is selected for RX(MISO) channel
 - * TimerX is selected for CS channel.
 - * PIN_Y is selected for RX(MISO) channel
 - * PIN_Z is selected for CS channel

SpiFlexioTxChannelConfig*	@ /Mcl/Mcl/MclConfig/FlexioCommon_0/FLEXIO_SPI_0_TX
SpiFlexioRxChannelConfig*	@ /Mcl/Mcl/MclConfig/FlexioCommon_0/FLEXIO_SPI_0_RX
SpiFlexioClkChannelConfig*	@ /Mcl/Mcl/MclConfig/FlexioCommon_0/FLEXIO_SPI_0_CLK
SpiFlexioCsChannelConfig*	@ /Mcl/Mcl/MclConfig/FlexioCommon_0/FLEXIO_SPI_0_CS

Figure 3.28 A set of FLEXIO SPI channels from SPI

3.6.9.2 Design Studio Tool:

It is the same with EB tresos

- Enable flexio mode first by set SpiFlexioEnable: (bellow picture)

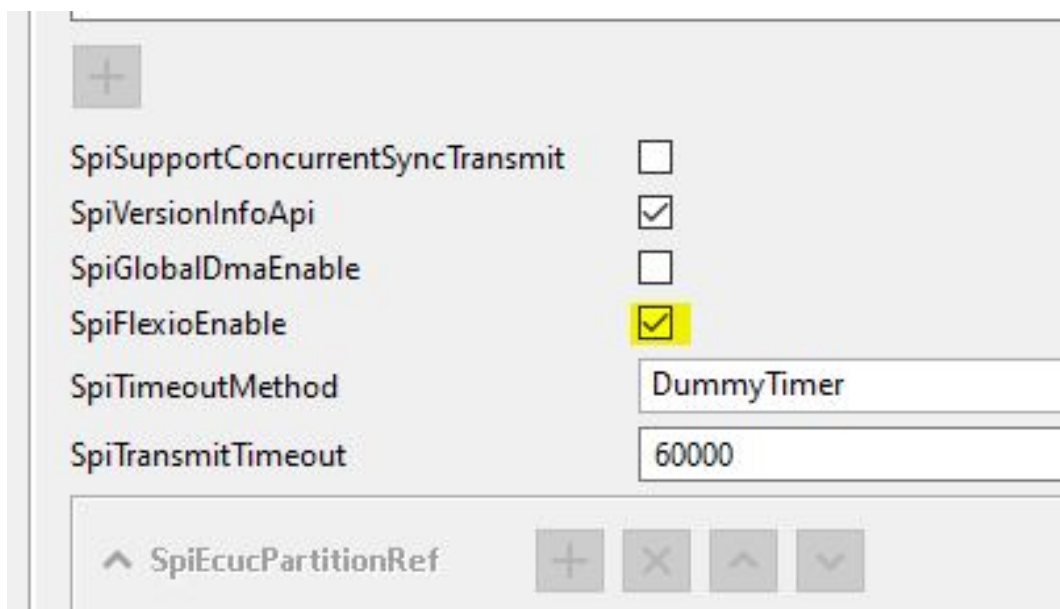


Figure 3.29 Enable SPI over FLEXIO

- Enable Flexio support on MCL site: (bellow picture)

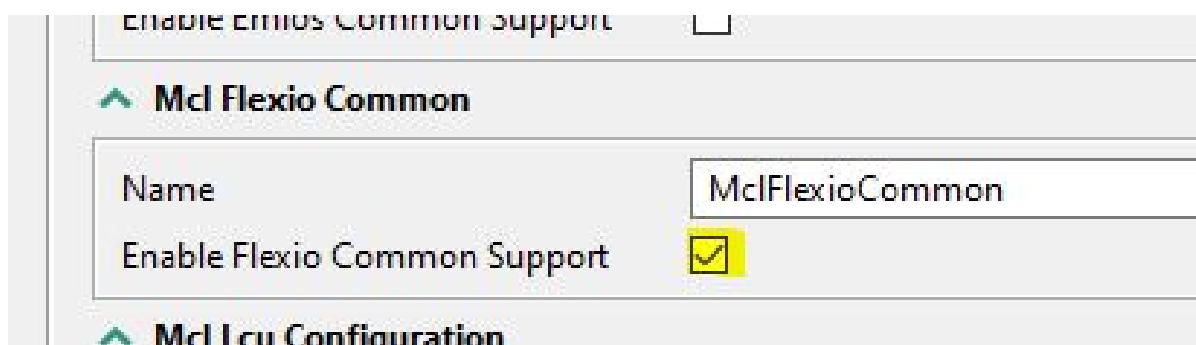


Figure 3.30 Enable Mcl FLEXIO support

- In MCL, setup a set for SPI FLEXIO Channels like: (bellow picture)
- *Flexio Channel*

is no. of Shifter and Timer registers will be selected for each FLEXIO channel. For example: CHANNEL_0 selected means Shifter_0 and Timer_0 will be selected and locked, other channels cannot select those registers.

- *Flexio Pin*

is no. of PIN will be selected for each FLEXIO channel. For example: PIN_0 selected means FLEXIO_D0 will be selected and locked, other channels cannot select this PIN.

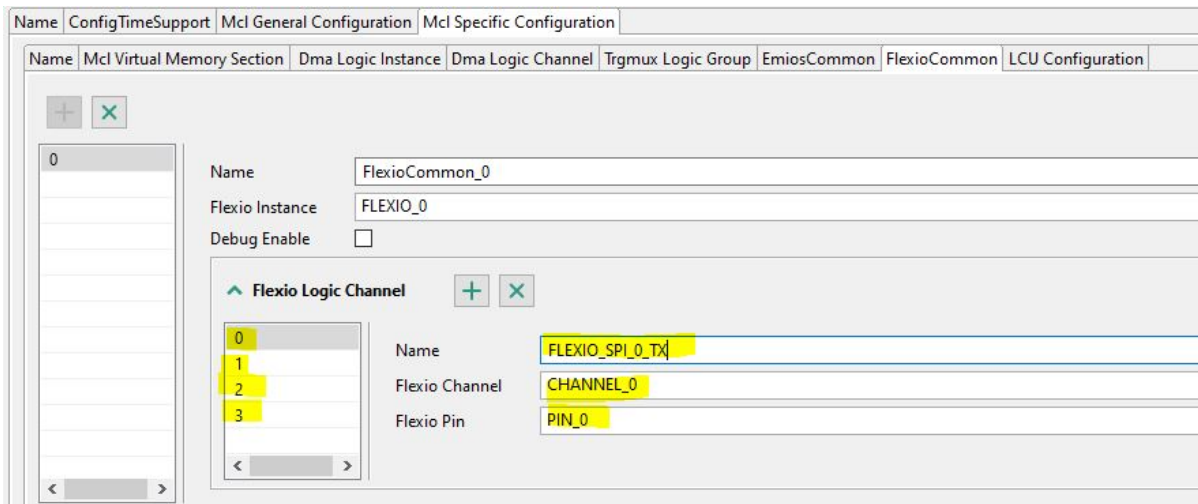


Figure 3.31 A set of FLEXIO SPI channels from MCL

- From SPI, each FLEXIO_SPI_x is selected in SpiPhyUnit then those set must be enabled and selected correctly value (refer from MCL which was configured above) (bellow picture)
 - SpiFlexioTxAndClkChannelsConfig: FLEXIO Logical Channel for TX(MOSI) and CLK(Clock). This selects a PIN to setup as TX(MOSI) channel and another for CLK channel. If you select CHANNEL_X, PIN_Y(FlexioMclPinId) and PIN_Z(FlexioMclAddPinId) that means:
 - * ShifterX is selected for TX(MOSI) channel
 - * TimerX is selected for CLK channel.
 - * PIN_Y is selected for TX(MOSI) channel
 - * PIN_Z is selected for CLK channel
 - SpiFlexioRxAndCsChannelsConfig: FLEXIO Logical Channel for RX(MISO) and CS(Chip select). This selects 1 PIN to setup as MISO channel and 1 PIN to set up as CS channel. If you select CHANNEL_X, PIN_Y(FlexioMclPinId) and PIN_Z(FlexioMclAddPinId) that mean:
 - * ShifterX is selected for RX(MISO) channel
 - * TimerX is selected for CS channel.
 - * PIN_Y is selected for RX(MISO) channel
 - * PIN_Z is selected for CS channel

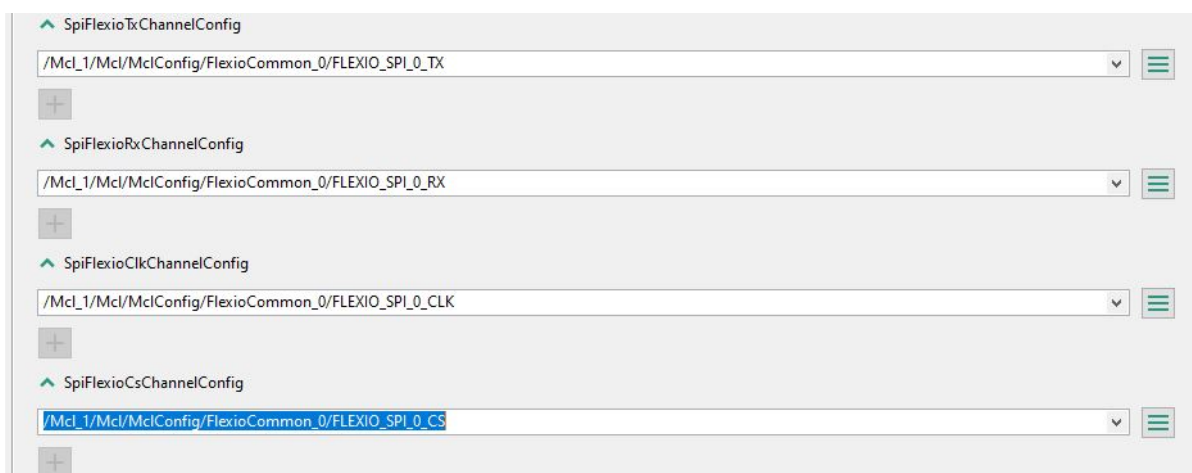


Figure 3.32 A set of FLEXIO SPI channels from SPI

3.6.9.3 Register ISR for INTERRUPT mode

- ISR name: **MCL_FLEXIO_ISR**
- Register FLEXIO ISR like document. For example: `sys_disableIsrSource(69);`
`sys_enableIsrSource(69, 7);`
`sys_registerIsrHandler(69, MCL_FLEXIO_ISR);`

where:

139: NVIC Interrupt ID of FLEXIO module

7: The priority

3.6.10 How to configure to use half duplex mode

To configure Spi in half duplex mode that has some different to normal configuration of full duplex mode. The different will proposed below:

- First of all, the SpiHalfDuplexModeSupport checkbox in SpiAutosarExt must be enabled to half-duplex mode can be configured in configurational tools:

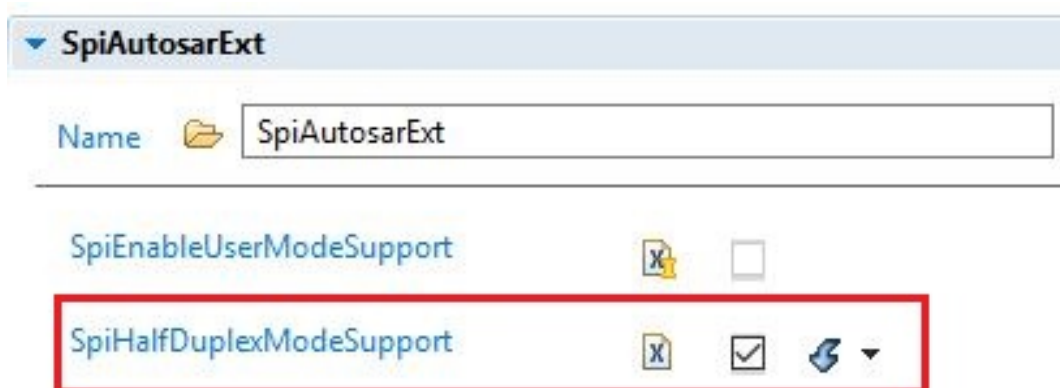


Figure 3.33 Enable Half-duplex mode support on EBTresos

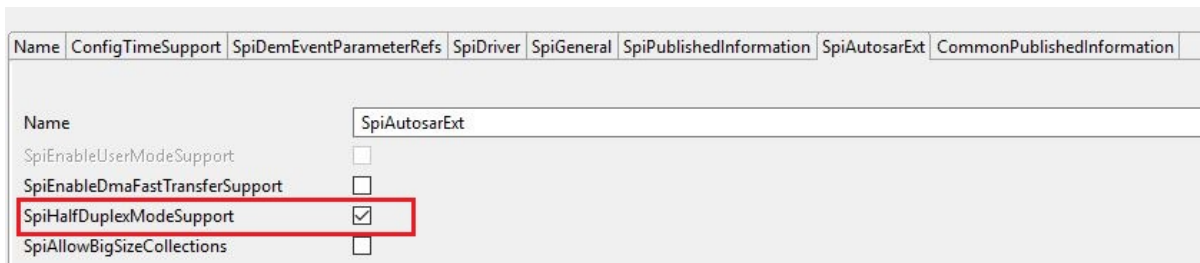


Figure 3.34 Enable Half-duplex mode support on S32 Design Studio

- Second, an external device has to be created with half-duplex transfer support by enabling SpiDeviceHalf↔DuplexSupport checkbox. After that, selecting number of bit transmitted and received on each clock edge. There are 3 types of transfer are supported: 1-bit, 2-bits or 4 bits.

Note

- Half-duplex mode only supports in external devices which are using Lpspi.
- With 1-bit transfer, the CIN or COUT pin could be chosen by SpiHalfDuplexPinSelect on External↔Device configuration.
- With 2-bits transfer, the CIN and COUT pins will be used.
- And with 4-bits transfer, the CIN, COUT, PCS2 and PCS3 will be used.

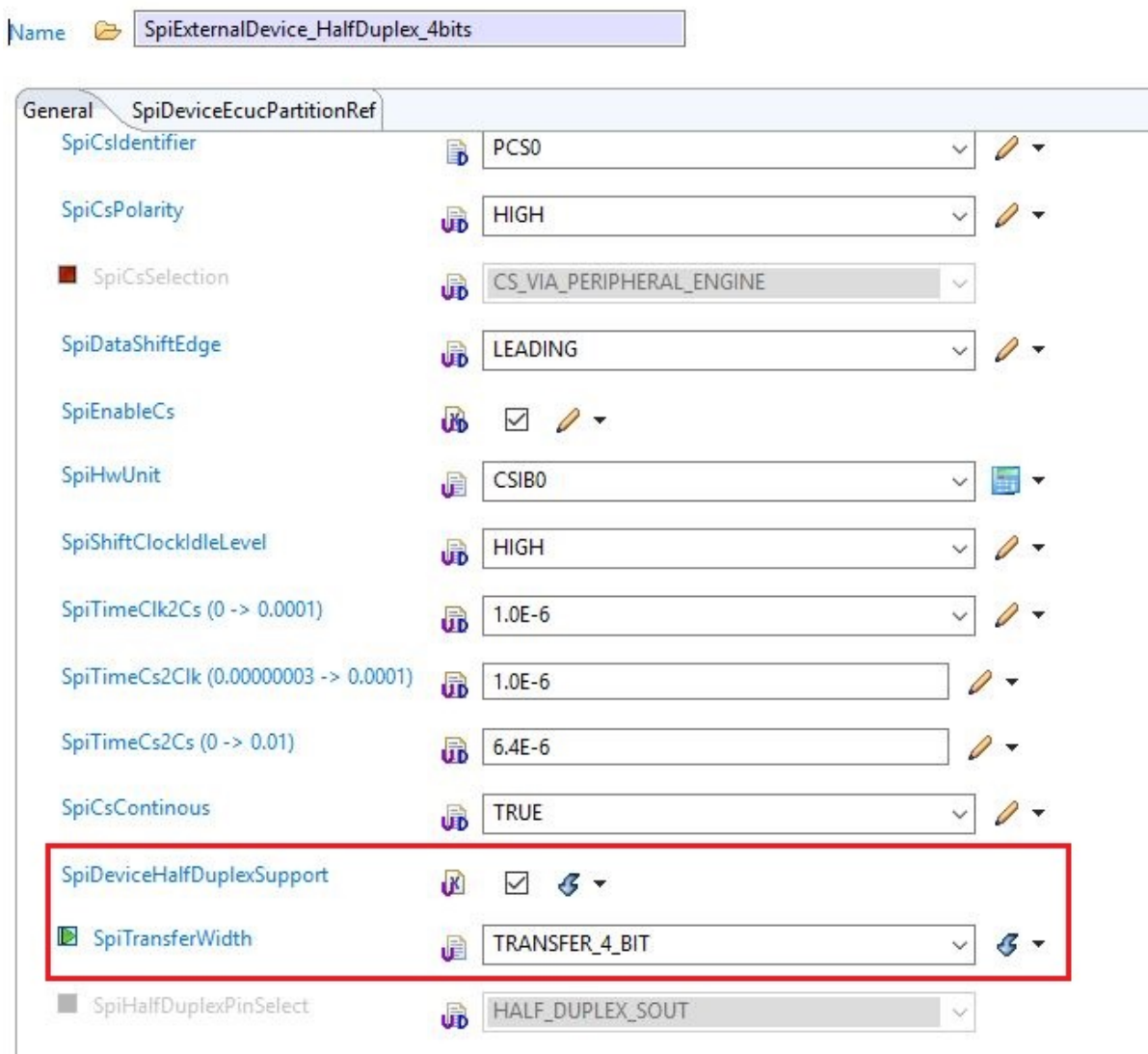


Figure 3.35 Configure Half-duplex mode for external device on EBTresos



Figure 3.36 Configure Half-duplex mode for external device on S32 Design Studio

- Finish, a channel should be selected as transmission or reception in half-duplex mode by configuring the Spi↔ChannelHalfDuplexDirection.

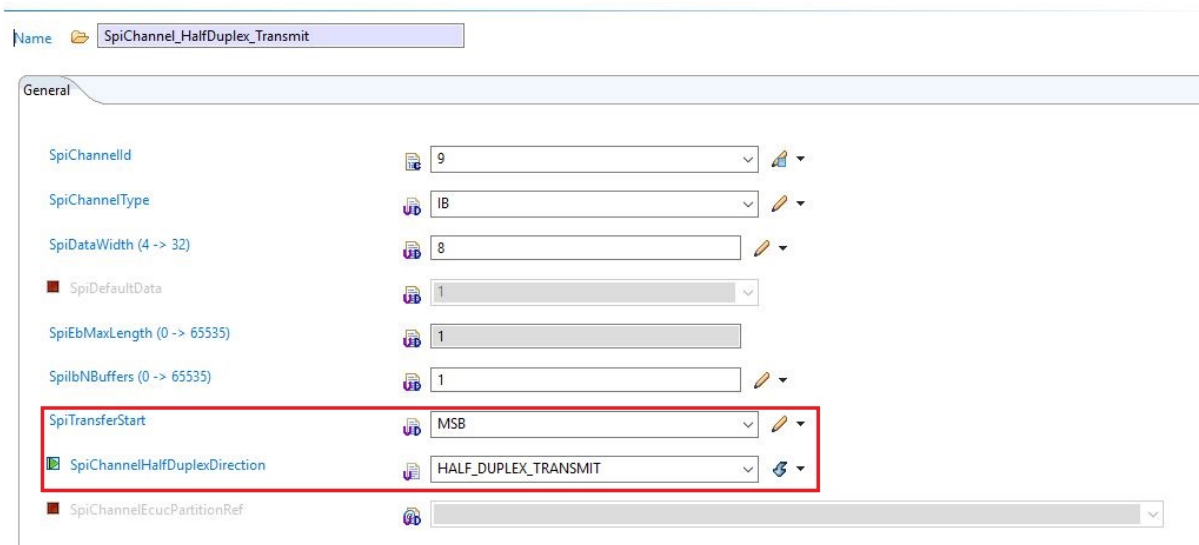


Figure 3.37 Configure Half-duplex mode for external device on EBTresos

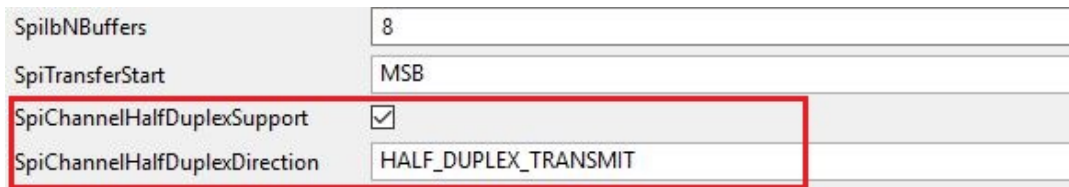


Figure 3.38 Configure Half-duplex mode for external device on S32 Design Studio

3.6.11 How to use Half Duplex mode

Below is presenting on EB tresos, this is same on CT.

- Enable Spi half duplex mode support:

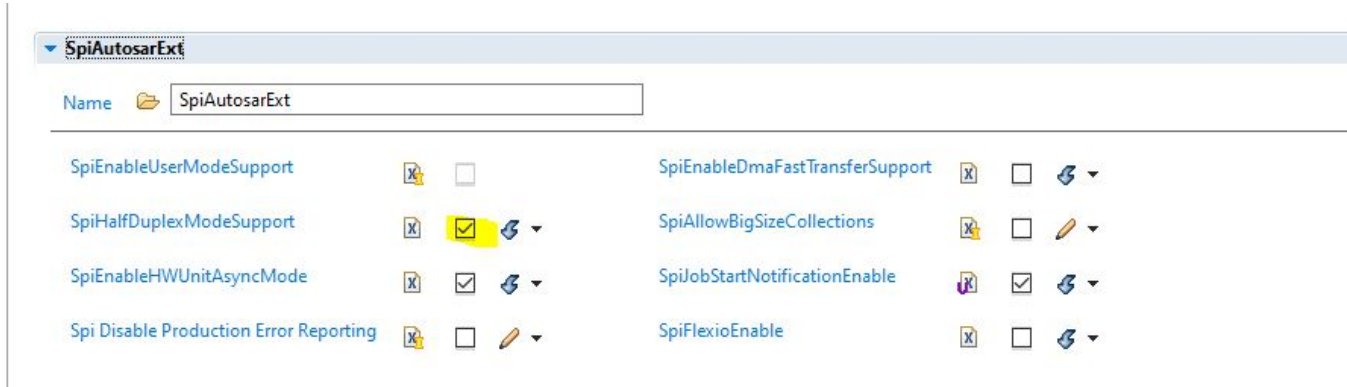


Figure 3.39 Enable Spi half duplex mode support

- Select direction for each channel:

- HALF_DUPLEX_TRANSMIT: Transmit only mode
- HALF_DUPLEX_RECEIVE: Receive only mode

SpiChannel

Name  SpiChannel_0

General

SpiChannelId	 0	
SpiChannelType	 EB	
SpiDataWidth (4 -> 32)	 8	
 SpiDefaultData	 1	
SpiEbMaxLength (0 -> 65535)	 10	
SpiNbBuffers (0 -> 65535)	 1	
SpiTransferStart	 LSB	
 SpiChannelHalfDuplexDirection	 HALF_DUPLEX_TRANSMIT	
 SpiChannelEcucPartitionRef		

Figure 3.40 Direction of channels

- Configure Externaldevice:
 - SpiTransferWidth: Number of bits will be transfered on per cycle clock. It supports 1/2/4 bits on Master mode, Slave mode only supports 1 bit mode.
 - SpiHalfDuplexPinSelect: Pin will be selected for the transfer (it is HALF_DUPLEX_SOUT means MOSI will be selected).

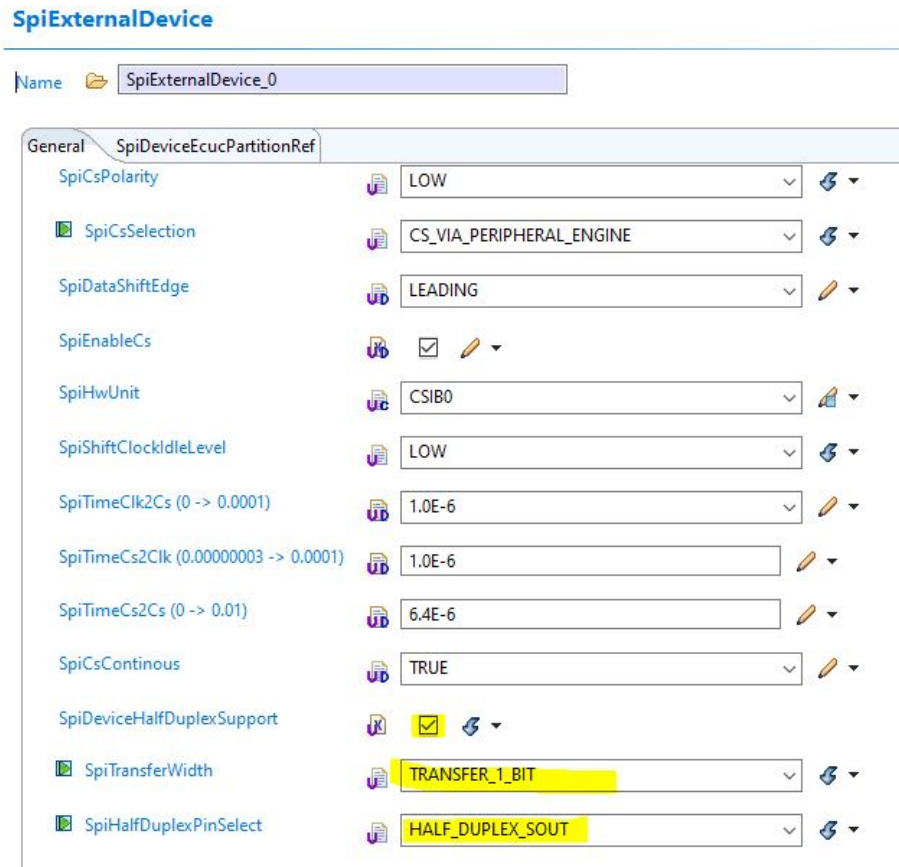


Figure 3.41 Direction of channels

3.6.12 Pointer to buffer alignment

- For $9 \leq \text{Framesize} < 17$: User or integrator have to make sure that pointer to 2-byte aligned buffer.
- For $17 \leq \text{Framesize}$: User or integrator have to make sure that pointer to 2-byte aligned buffer.

3.6.13 How to config interrupts for the transmission of Master and Slave on Spi examples in the same chip

Prefer to use either Master or Slave in Dma mode to avoid interrupt overlapping.

3.6.14 How to configure to use Spi Host Request feature.

This feature requires:

- Support LPSPI hardware instance only.

Configuration example for using Spi Host Request feature:

- Enable Spi Host Request support.

Spi Configuration [MCAL]

Name

Mode

Name	ConfigTimeSupport	SpiDemEventParameterRefs	SpiDriver	SpiGeneral	SpiPublishedInformation	SpiAutosarExt	CommonPublishedInf
Name SpiAutosarExt							
SpiEnableUserModeSupport							
SpiEnableDmaFastTransferSupport							
SpiEnableDmaContMemTransferSupport							
SpiHREQSupport						☒	
SpiHalfDuplexModeSupport							
SpiAllowBigSizeCollections							
SpiEnableHWUnitAsyncMode							
SpiJobStartNotificationEnable							
Spi Disable Production Error Reporting							
SpiFlexioEnable							

Figure 3.42 Enable Spi Host Request feature

- Change value of SpiHostRequest in external device configuration to choose what type of host request type, polarity and direction.

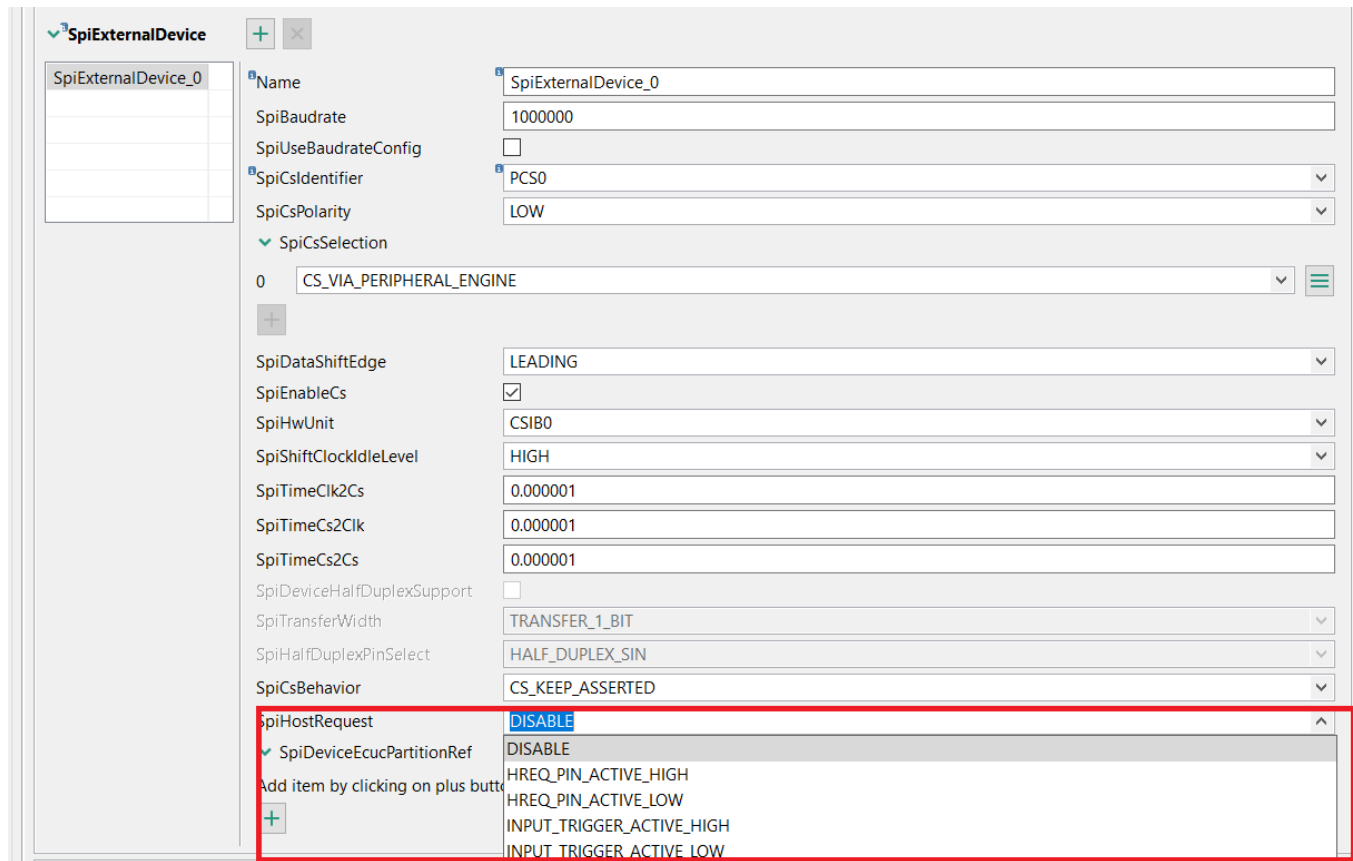


Figure 3.43 Spi Host Request

- For input trigger option, refer to Trigger MUX (TRGMUX) chapter in reference manual.
- For host request pin option, refer to Signals description in LPSPI chapter in reference manual.

3.6.15 How to use byte swap feature for LPSPI hardware instance.

This feature requires:

- Support 32 bits framesize only.
- Support LPSPI hardware instance only.

Guide for using byte swap feature: In order to use this feature:

- Enable SpiByteSwapTransfer in SpiChannel (HLD) or SpiExternalDevice (LPSPI IP)
- SpiByteSwapTransfer parameter can be changed in runtime by using Lpspi_Ip_UpdateByteSwap API.

3.6.16 How to configure baudrate registers

This feature requires:

- Support LPSPI hardware instance only.

This feature is useful to see and configure the final generated baudrate. Unlike SpiBaudrate which would generate the closest baudrate to the specified value.

1. Add as much baudrate configurations as needed
2. To edit a configuration: SpiPrescaler and SpiScaler should be edited
3. Click on generate for SpiPrescalerValue.
4. For each SpiExternalDevice, to use a baudrate configuration, select SpiUseBaudrateConfig and then in SpiBaudrateConfigRef select one of the previously created, generate the value for SpiCalculatedBaudrate.

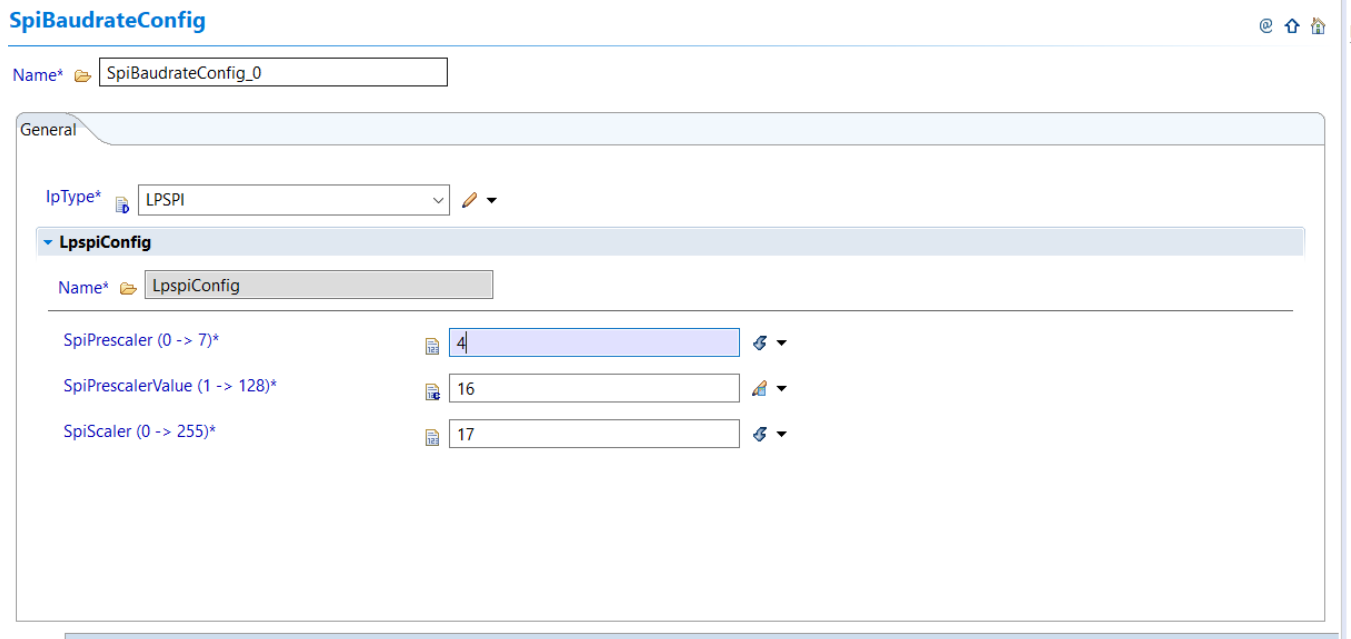


Figure 3.44 Configure baudrate registers

3.6.17 How to use Half Duplex mode

Below is presenting on EB tresos, this is same on CT.

- Enable Spi half duplex mode support:

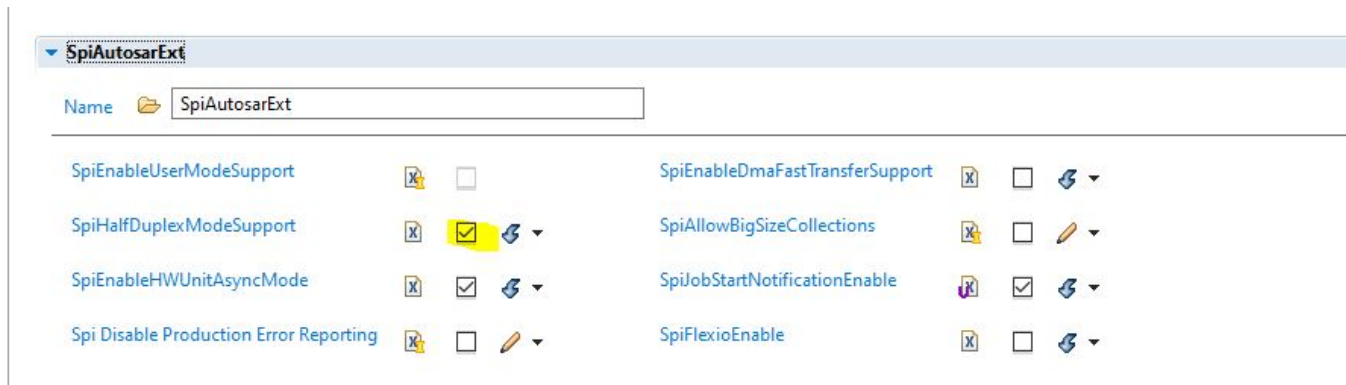


Figure 3.45 Enable Spi half duplex mode support

- Select direction for each channel:

- HALF_DUPLEX_TRANSMIT: Transmit only mode
- HALF_DUPLEX_RECEIVE: Receive only mode

SpiChannel

Name  SpiChannel_0

General

SpiChannelId	 0	
SpiChannelType	 EB	
SpiDataWidth (4 -> 32)	 8	
 SpiDefaultData	 1	
SpiEbMaxLength (0 -> 65535)	 10	
SpiNbBuffers (0 -> 65535)	 1	
SpiTransferStart	 LSB	
 SpiChannelHalfDuplexDirection	 HALF_DUPLEX_TRANSMIT	
 SpiChannelEcucPartitionRef		

Figure 3.46 Direction of channels

- Configure Externaldevice:
 - SpiTransferWidth: Number of bits will be transfered on per cycle clock. It supports 1/2/4/8 bits on Master mode, Slave mode only supports 1 bit mode. Minimum value of SpiDataWidth should be followed by:
 - * TRANSFER_1_BIT mode: 2
 - * TRANSFER_2_BIT mode: 4
 - * TRANSFER_4_BIT mode: 8
 - * TRANSFER_8_BIT mode: 16
 - SpiHalfDuplexPinSelect: Pin will be selected for the transfer (it is HALF_DUPLEX_SOUT means MOSI will be selected).

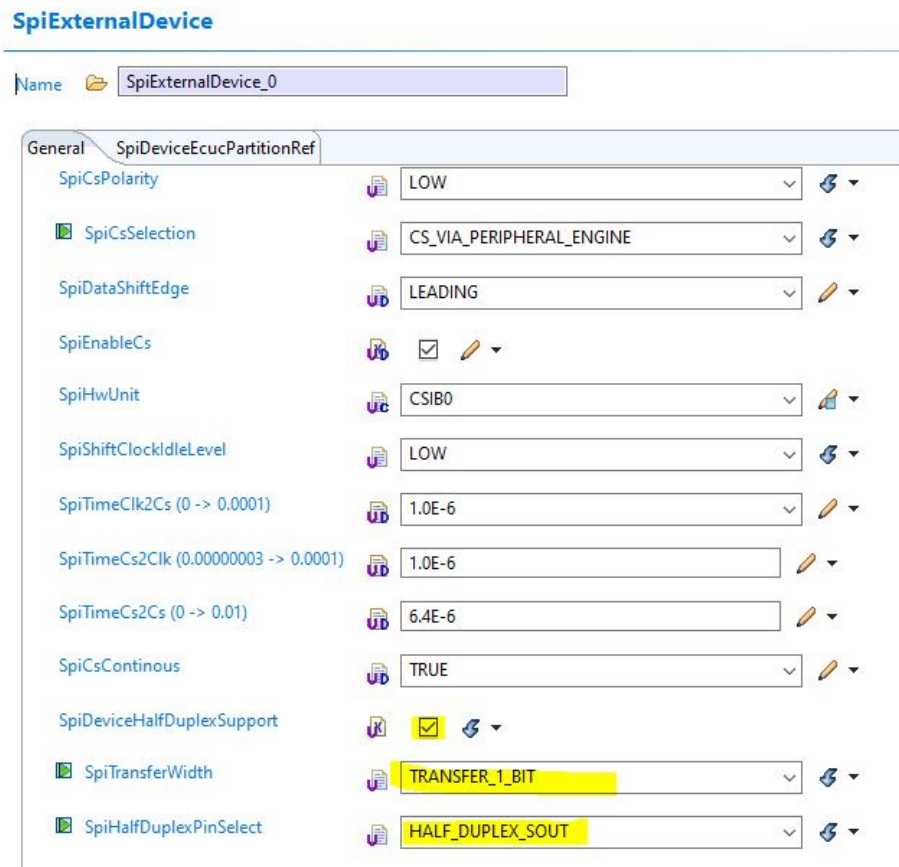


Figure 3.47 Direction of channels

3.6.18 How to configure to use Spi Host Request feature.

This feature requires:

- Support LPSPI hardware instance only.

Configuration example for using Spi Host Request feature:

- Enable Spi Host Request support.

Spi Configuration [MCAL]

Name

Mode

Name	ConfigTimeSupport	SpiDemEventParameterRefs	SpiDriver	SpiGeneral	SpiPublishedInformation	SpiAutosarExt	CommonPublishedInf
Name SpiAutosarExt							
SpiEnableUserModeSupport							
SpiEnableDmaFastTransferSupport							
SpiEnableDmaContMemTransferSupport							
SpiHREQSupport						☒	
SpiHalfDuplexModeSupport							
SpiAllowBigSizeCollections							
SpiEnableHWUnitAsyncMode							
SpiJobStartNotificationEnable							
Spi Disable Production Error Reporting							
SpiFlexioEnable							

Figure 3.48 Enable Spi Host Request feature

- Change value of SpiHostRequest in external device configuration to choose what type of host request type, polarity and direction.

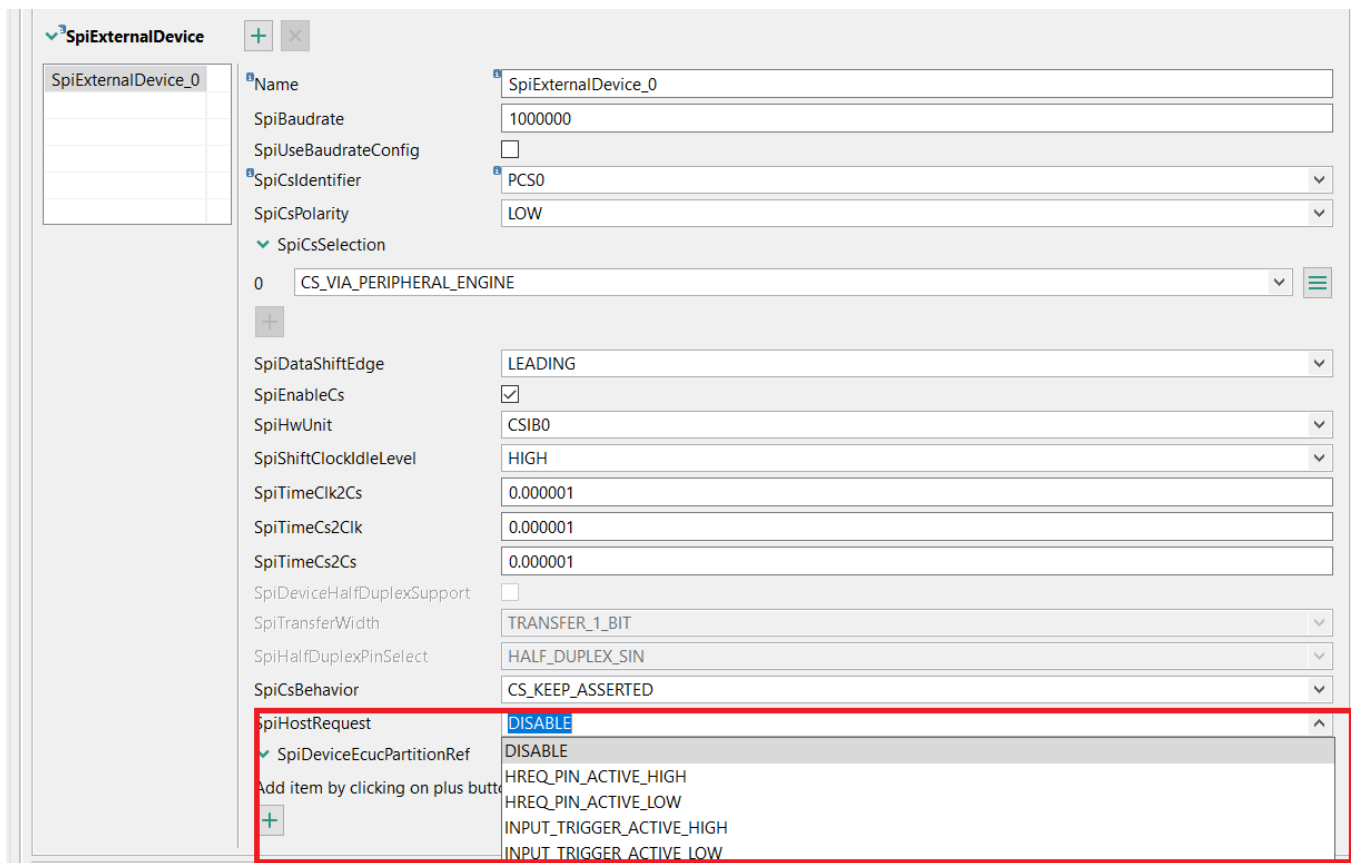


Figure 3.49 Spi Host Request

- For input trigger option, refer to Trigger MUX (TRGMUX) chapter in reference manual.
- For host request pin option, refer to Signals description in LPSPI chapter in reference manual.

3.6.19 How to use byte swap feature for LPSPI hardware instance.

This feature requires:

- Support 32 bits framesize only.
- Support LPSPI hardware instance only.

Guide for using byte swap feature: In order to use this feature:

- Enable SpiByteSwapTransfer in SpiChannel (HLD) or SpiExternalDevice (LPSPI IP)
- SpiByteSwapTransfer parameter can be changed in runtime by using Lpspi_Ip_UpdateByteSwap API.

3.6.20 How to configure SPI over FLEXIO

First of all, SPI over FLEXIO must use MCL module to enable/disable Flexio module and configure Flexio Channels (Select PIN, Shifter, Timer registers) and ISR handler. SPI can not work by itself. belowing is guiding to configure SPI over FLEXIO for both EB tresos and CT configuration tool

Note: Mcl_Init() functions must be called before using SPI over FLEXIO.

3.6.20.1 EB Tresos

- Enable flexio mode first by set SpiFlexioEnable: (below picture)

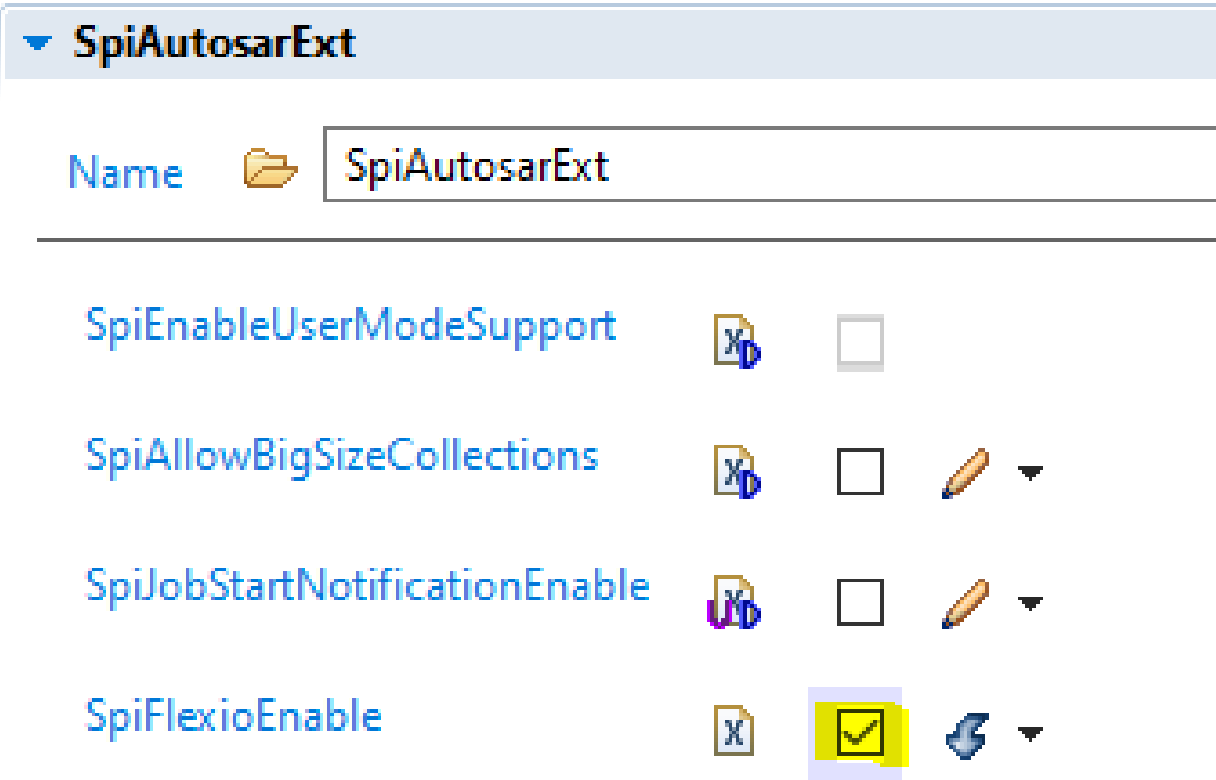


Figure 3.50 Enable SPI over FLEXIO

- Enable Flexio support on MCL site: (below picture)

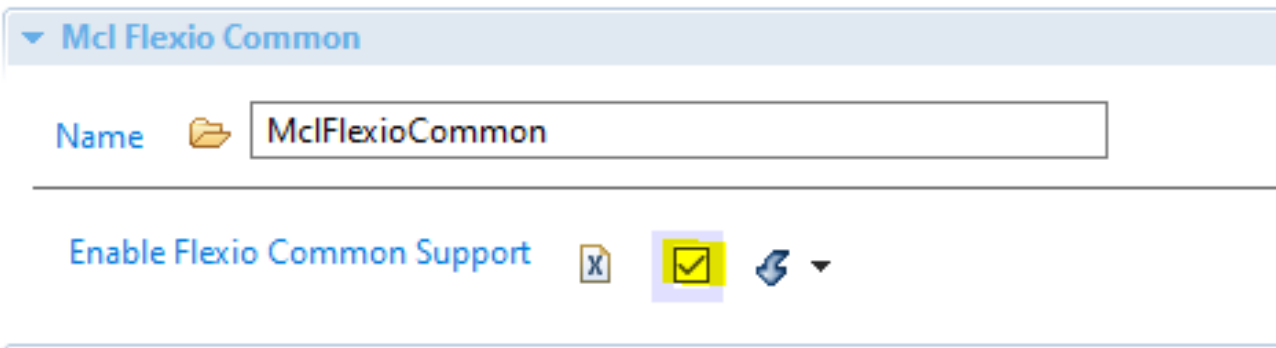


Figure 3.51 Enable Mcl FLEXIO support

- In MCL, setup a set for SPI FLEXO Channels like: (below picture)
- *Flexio Channel*

is no. of Shifter and Timer registers will be selected for each FLEXIO channel. For example: CHANNEL_0 selected means Shifter_0 and Timer_0 will be selected and locked, other channels cannot select those registers.

- *Flexio Pin*

is no. of PIN will be selected for each FLEXIO channel. For example: PIN_0 selected means FLEXIO_D0 will be selected and locked, other channels cannot select this PIN.

FlexioCommon

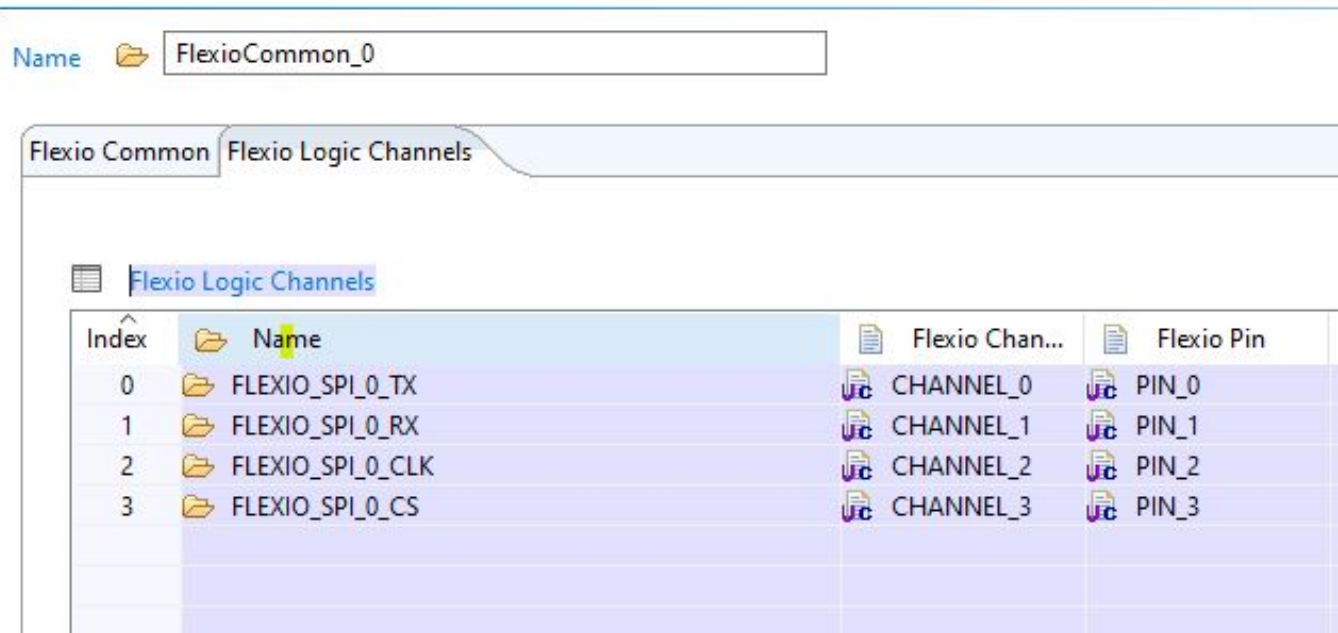


Figure 3.52 A set of FLEXIO SPI channels from MCL

- From SPI, each FLEXIO_SPI_x is selected in SpiPhyUnit then those set must be enabled and selected correctly value (refer from MCL which was configured above) (below picture)
 - SpiFlexioTxAndClkChannelsConfig: FLEXIO Logical Channel for TX(MOSI) and CLK(Clock). This selects 1 PIN to setup as TX(MOSI) channel and 1 PIN to set up as CLK channel. If you select CHANNEL_X, PIN_Y(FlexioMclPinId) and PIN_Z(FlexioMclAddPinId) that means:
 - * ShifterX is selected for TX(MOSI) channel
 - * TimerX is selected for CLK channel.
 - * PIN_Y is selected for TX(MOSI) channel
 - * PIN_Z is selected for CLK channel
 - SpiFlexioRxAndCsChannelsConfig: FLEXIO Logical Channel for RX(MISO) and CS(Chip select). This selects 1 PIN to setup as MISO channel and 1 PIN to set up as CS channel. If you select CHANNEL_X, PIN_Y(FlexioMclPinId) and PIN_Z(FlexioMclAddPinId) that mean:
 - * ShifterX is selected for RX(MISO) channel
 - * TimerX is selected for CS channel.
 - * PIN_Y is selected for RX(MISO) channel
 - * PIN_Z is selected for CS channel

 SpiFlexioTxChannelConfig*	 /Mcl/Mcl/MclConfig/FlexioCommon_0/FLEXIO_SPI_0_TX
 SpiFlexioRxChannelConfig*	 /Mcl/Mcl/MclConfig/FlexioCommon_0/FLEXIO_SPI_0_RX
 SpiFlexioClkChannelConfig*	 /Mcl/Mcl/MclConfig/FlexioCommon_0/FLEXIO_SPI_0_CLK
 SpiFlexioCsChannelConfig*	 /Mcl/Mcl/MclConfig/FlexioCommon_0/FLEXIO_SPI_0_CS

Figure 3.53 A set of FLEXIO SPI channels from SPI

3.6.20.2 Configuration Tool: It is the same with EB tresos

- Enable flexio mode first by set SpiFlexioEnable: (below picture)

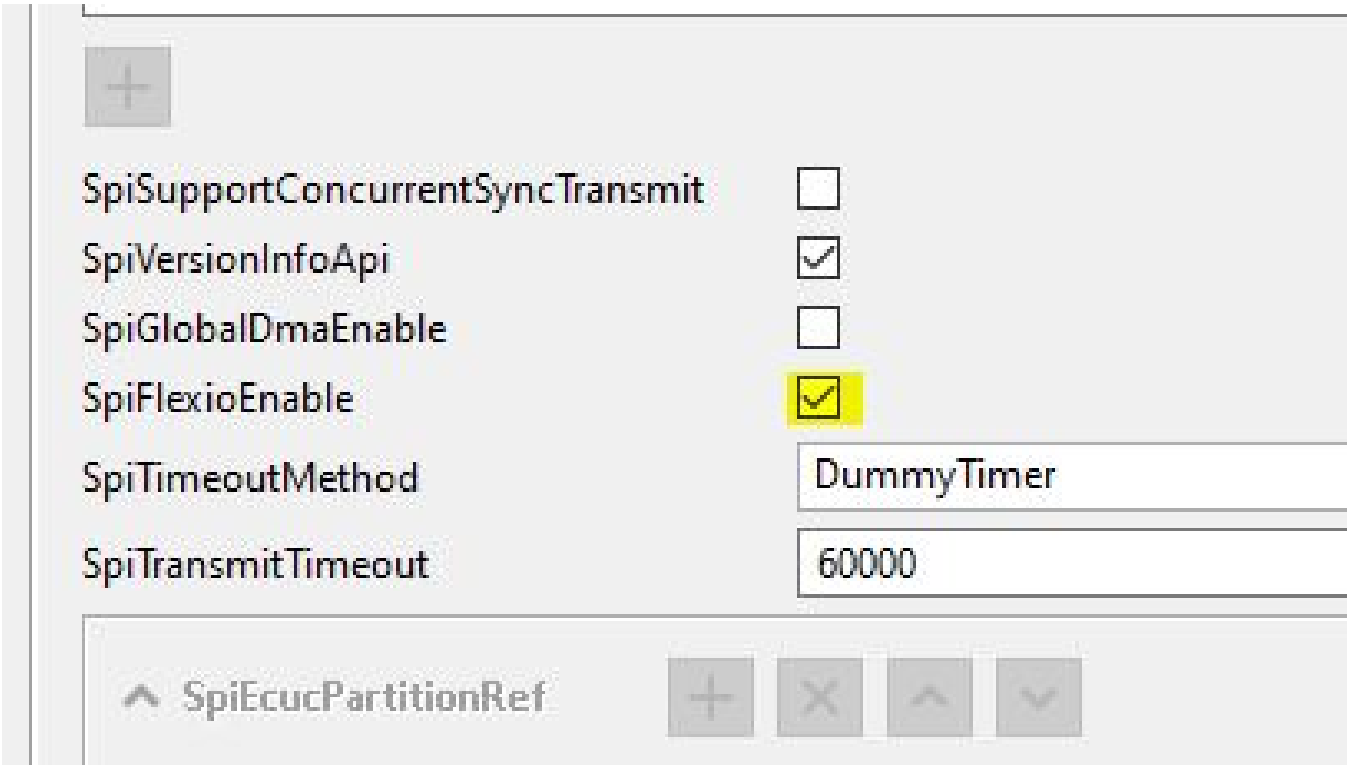


Figure 3.54 Enable SPI over FLEXIO

- Enable Flexio support on MCL site: (below picture)

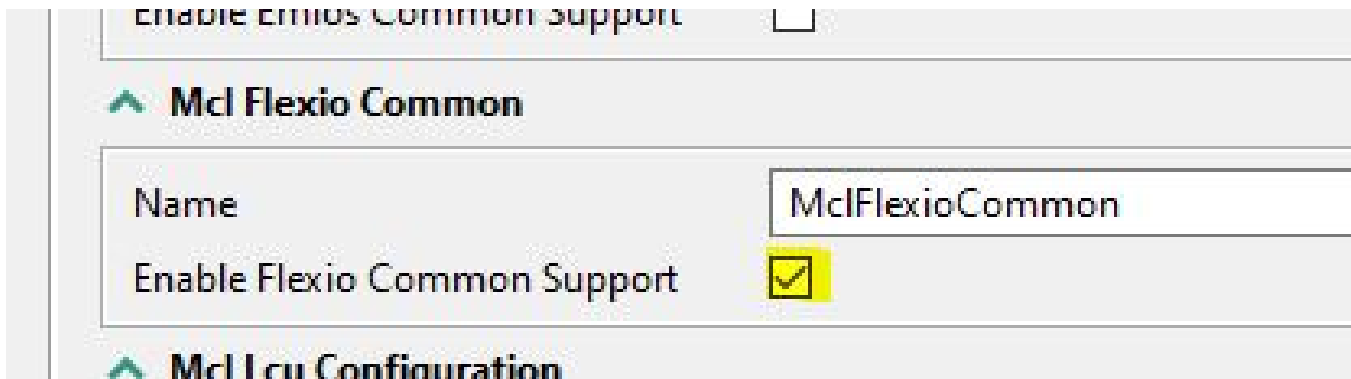


Figure 3.55 Enable Mcl FLEXIO support

- In MCL, setup a set for SPI FLEXIO Channels like: (below picture)

- *Flexio Channel*

is no. of Shifter and Timer registers will be selected for each FLEXIO channel. For example: CHANNEL_0 selected means Shifter_0 and Timer_0 will be selected and locked, other channels cannot select those registers.

- *Flexio Pin*

is no. of PIN will be selected for each FLEXIO channel. For example: PIN_0 selected means FLEXIO_D0 will be selected and locked, other channels cannot select this PIN.

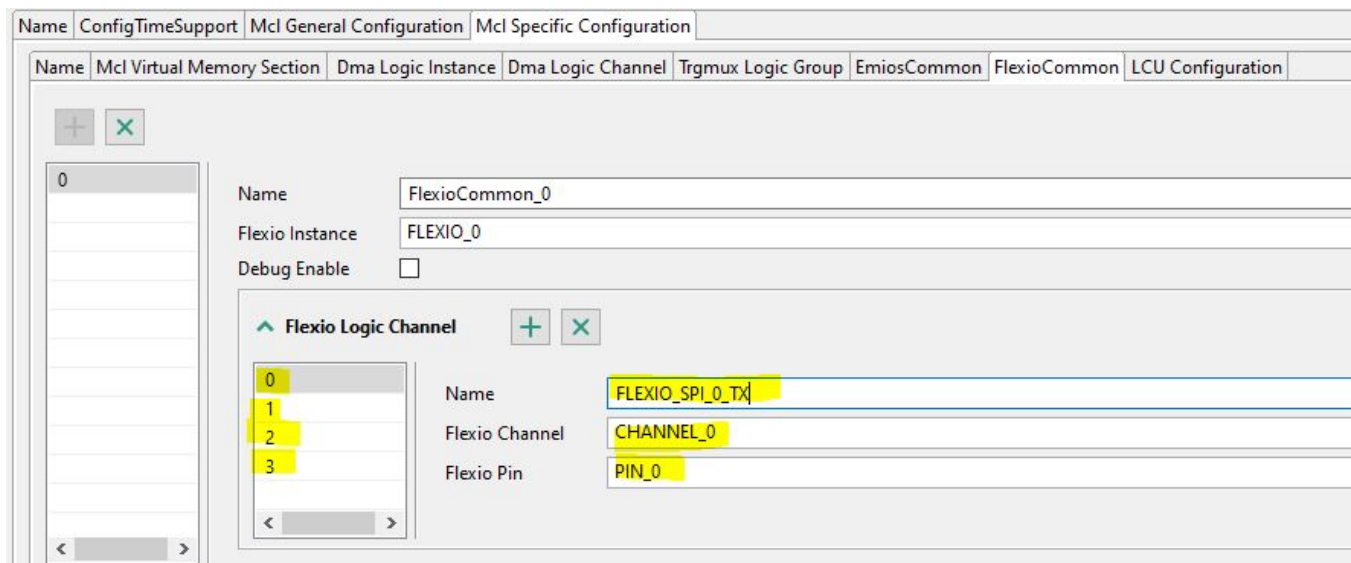


Figure 3.56 A set of FLEXIO SPI channels from MCL

- From SPI, each FLEXIO_SPI_x is selected in SpiPhyUnit then those set must be enabled and selected correctly value (refer from MCL which was configured above) (below picture)
 - SpiFlexioTxAndClkChannelsConfig: FLEXIO Logical Channel for TX(MOSI) and CLK(Clock). This selects 1 PIN to setup as TX(MOSI) channel and 1 PIN to set up as CLK channel. If you select CHANNEL_X, PIN_Y(FlexioMclPinId) and PIN_Z(FlexioMclAddPinId) that means:
 - * ShifterX is selected for TX(MOSI) channel
 - * TimerX is selected for CLK channel.

- * PIN_Y is selected for TX(MOSI) channel
- * PIN_Z is selected for CLK channel
- SpiFlexioRxAndCsChannelsConfig: FLEXIO Logical Channel for RX(MISO) and CS(Chip select). This selects 1 PIN to setup as MISO channel and 1 PIN to set up as CS channel. If you select CHANNEL_X, PIN_Y(FlexioMclPinId) and PIN_Z(FlexioMclAddPinId) that mean:
 - * ShifterX is selected for RX(MISO) channel
 - * TimerX is selected for CS channel.
 - * PIN_Y is selected for RX(MISO) channel
 - * PIN_Z is selected for CS channel



Figure 3.57 A set of FLEXIO SPI channels from SPI

3.6.20.3 Register ISR for INTERRUPT mode

- ISR name: **MCL_FLEXIO_ISR**
- Register FLEXIO ISR like document. For example: `sys_disableIsrSource(139);`
`sys_enableIsrSource(139, 7);`
`sys_registerIsrHandler(139, MCL_FLEXIO_ISR);`

where:

139: NVIC Interrupt ID of FLEXIO module

7: The priority

3.7 Runtime errors

The driver generates the following DEM errors at runtime.

Function	Error Code	Condition triggering the error
Spi_SyncTransmit	Spi_E_Hardware_ErrorCfg	The SPI driver cannot transmit complete or receive complete one frame in the allocated time defined by “SPI_TIMEOUT_COUNTER” parameter in configuration. Baud rate of HW might be low speed or timeout value too short. Timeout occurred.
Spi_SyncTransmit	SPI_E_SEQ_IN_PROCESS	Synchronous transmission service called at wrong time.
Spi_AsyncTransmit	SPI_E_SEQ_PENDING	Services called in a wrong sequence.

3.8 Symbolic Names Disclaimer

All containers having symbolicNameValue set to TRUE in the AUTOSAR schema will generate defines like: `#define <Mip>Conf_<Container_ShortName>_<Container_ID>`

For this reason it is forbidden to duplicate the names of such containers across the RTD configurations or to use names that may trigger other compile issues (e.g. match existing `#ifdefs` arguments).

3.9 Usage in non AUTOSAR context

For the non-ASR environments, both High-Level and IP layer interfaces are exposed and can be used by software units above. These may consist of either higher level NXP SW deliverables, like stacks and middleware, or the application code itself.

The goal of the product is to be versatile, easy to use and integrate with other SW environments, offering both standardized and optimized low-level interfaces.

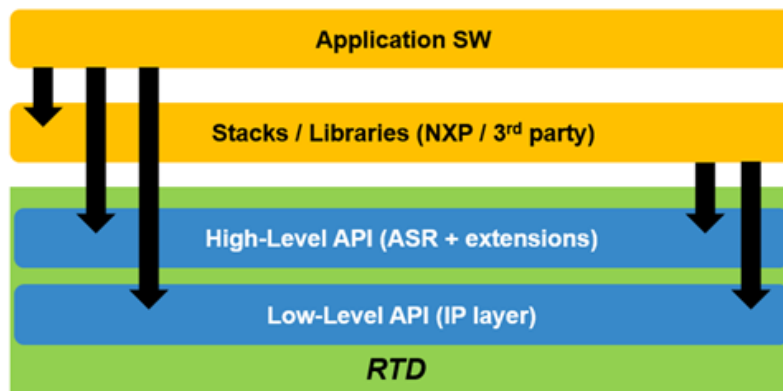


Figure 3.58 RTD Usage in non-ASR Context

In the non-AUTOSAR environment, there are no specific ECU states defined, but an external assumption for the application is that the correct sequence of initialization takes place prior to calling any runtime APIs.

The constraints considered for defining the APIs in the non-AUTOSAR environment are:

- Hardware resources used by the drivers must be properly initialized prior to runtime usage by calling the initialization function with configuration parameters generated by the configuration tools
- All dependencies must be initialized prior to the usage of a driver that depends on them; this translates to a general initialization sequence applicable for all applications, as follows:
 1. Initialization of the device clock tree so other periphery can be initialized

2. Configuration of the pinout so peripherals with external connections can communicate with the outside world
3. Configuration of the common resources (e.g. DMA channels, memory regions, access rights, etc) prior to the usage of any runtime functionality that has to benefit from this overall SoC configuration
4. Initialization of specific drivers
5. Runtime functionality
6. Deinitialization (inverse initialization order)

High-level and IP layer functionalities cannot be used simultaneously over the same hardware resource. The granularity of the resource must be defined specifically for each driver, but as a general rule the default granularity considered is the peripheral instance (e.g. Gpt high level APIs cannot be called interchangeably with Pit_Ip low level APIs over the same PIT hardware instance).

The diagram below shows a generic, high-level sequence of calls to RTD interfaces in the non-AUTOSAR scenario:

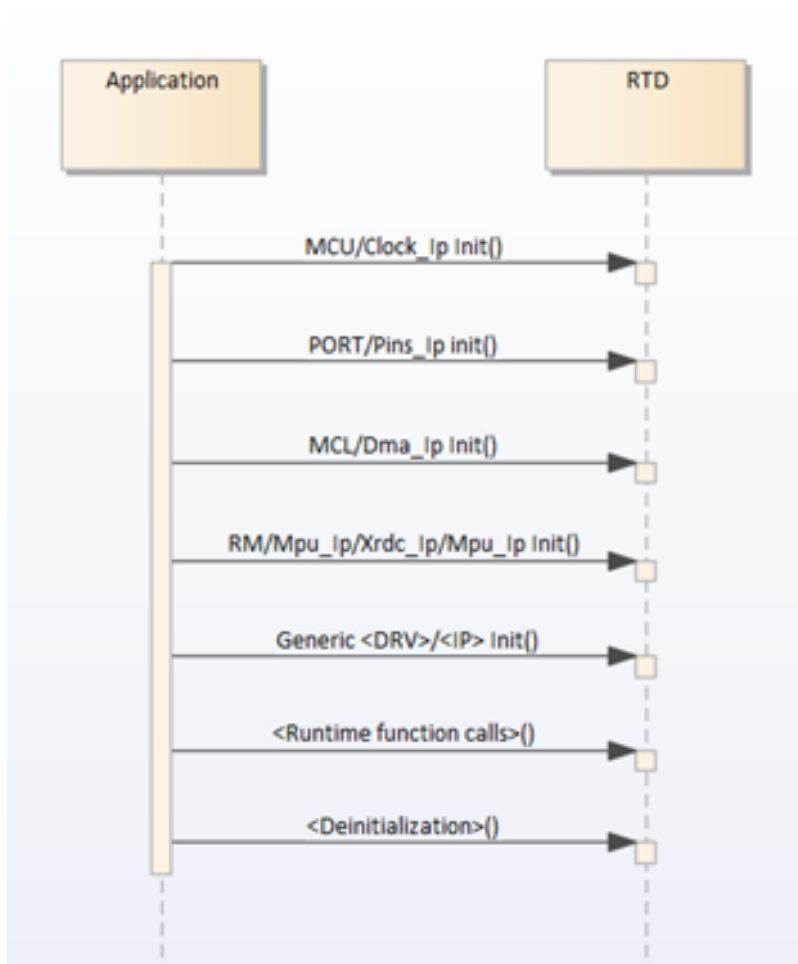


Figure 3.59 Non-AUTOSAR Sequence diagram

Chapter 4

Configuration Parameters

RTD provides configuration schema for the high level driver in EB Tresos proprietary format (**.xdm**), S32CT proprietary format (**.component**), but also in a standardized AUTOSAR format (**.epd**). For the low level driver configuration only the S32CT proprietary format (**.component**) is supported. The configuration data of the project can be saved in EB Tresos proprietary format (**.xdm**) and AUTOSAR standardized format (**.epc**) from EB Tresos and in S32CT proprietary format (**.mex**) and AUTOSAR standardized format (**.epc**) from S32 Design Studio. RTD provides templates for parsing the configuration data and generating configuration structures from it in XPath format, to be used with the EB Tresos generator and in JS format, to be used with the S32CT generator. Since the generation process is not standardized by AUTOSAR, only the two mentioned tools can be used with the provided templates. Using the same input configuration data, the same configuration structures will be obtained using any of the generators.

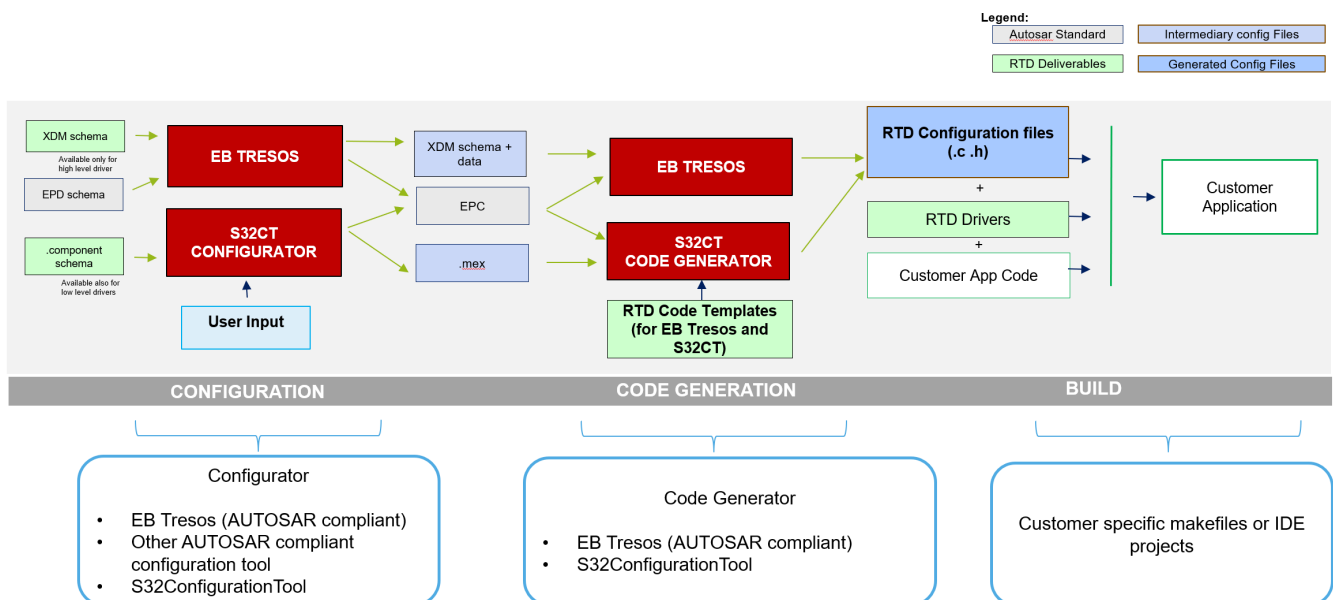


Figure 4.1 Configuration generation flow

- Spi
- LPSPI
- FLEXIO_SPI

4.1 Spi

- Module [Spi](#)
 - Container [SpiDemEventParameterRefs](#)
 - * Reference [SPI_E_HARDWARE_ERROR](#)
 - Container [SpiDriver](#)
 - * Parameter [SpiMaxChannel](#)
 - * Parameter [SpiMaxJob](#)
 - * Parameter [SpiMaxSequence](#)
 - * Container [SpiChannel](#)
 - Parameter [SpiChannelId](#)
 - Parameter [SpiChannelType](#)
 - Parameter [SpiDataWidth](#)
 - Parameter [SpiDefaultData](#)
 - Parameter [SpiEbMaxLength](#)
 - Parameter [SpiIbNBuffers](#)
 - Parameter [SpiTransferStart](#)
 - Parameter [SpiByteSwapTransfer](#)
 - Parameter [SpiChannelHalfDuplexDirection](#)
 - Reference [SpiChannelEcucPartitionRef](#)
 - * Container [SpiExternalDevice](#)
 - Parameter [SpiBaudrate](#)
 - Parameter [SpiUseBaudrateConfig](#)
 - Parameter [SpiCalculatedBaudrate](#)
 - Parameter [SpiCalculatedAlternateBaudrate](#)
 - Parameter [SpiCsIdentifier](#)
 - Parameter [SpiCsPolarity](#)
 - Parameter [SpiCsSelection](#)
 - Parameter [SpiDataShiftEdge](#)
 - Parameter [SpiEnableCs](#)
 - Parameter [SpiHwUnit](#)
 - Parameter [SpiShiftClockIdleLevel](#)
 - Parameter [SpiTimeClk2Cs](#)
 - Parameter [SpiTimeCs2Clk](#)
 - Parameter [SpiTimeCs2Cs](#)
 - Parameter [SpiCsBehavior](#)
 - Parameter [SpiHostRequest](#)

- Parameter [SpiDeviceHalfDuplexSupport](#)
- Parameter [SpiTransferWidth](#)
- Parameter [SpiHalfDuplexPinSelect](#)
- Reference [SpiBaudrateConfigRef](#)
- Reference [SpiDeviceEcucPartitionRef](#)
- * Container [SpiBaudrateConfig](#)
 - Parameter [IpType](#)
 - Container [LpspiConfig](#)
 - Parameter [SpiPrescaler](#)
 - Parameter [SpiPrescalerValue](#)
 - Parameter [SpiScaler](#)
- * Container [SpiJob](#)
 - Parameter [SpiJobEndNotification](#)
 - Parameter [SpiJobStartNotification](#)
 - Parameter [SpiJobId](#)
 - Parameter [SpiJobPriority](#)
 - Reference [SpiDeviceAssignment](#)
 - Container [SpiChannelList](#)
 - Parameter [SpiChannelIndex](#)
 - Reference [SpiChannelAssignment](#)
- * Container [SpiSequence](#)
 - Parameter [SpiInterruptibleSequence](#)
 - Parameter [SpiSeqEndNotification](#)
 - Parameter [SpiSequenceId](#)
 - Parameter [SpiEnableDmaFastTransfer](#)
 - Reference [SpiJobAssignment](#)
- Container [SpiGeneral](#)
 - * Parameter [SpiMultipartitionSupport](#)
 - * Parameter [SpiCancelApi](#)
 - * Parameter [SpiChannelBuffersAllowed](#)
 - * Parameter [SpiDevErrorDetect](#)
 - * Parameter [SpiHwStatusApi](#)
 - * Parameter [SpiInterruptibleSeqAllowed](#)
 - * Parameter [SpiLevelDelivered](#)
 - * Parameter [SpiMainFunctionPeriod](#)
 - * Parameter [SpiSupportConcurrentSyncTransmit](#)

- * Parameter [SpiVersionInfoApi](#)
- * Parameter [SpiGlobalDmaEnable](#)
- * Parameter [SpiTimeoutMethod](#)
- * Parameter [SpiTransmitTimeout](#)
- * Reference [SpiEcucPartitionRef](#)
- * Reference [SpiKernelEcucPartitionRef](#)
- * Container [SpiPhyUnit](#)
 - Parameter [SpiPhyUnitMapping](#)
 - Parameter [SpiPhyUnitMode](#)
 - Parameter [SpiSamplePoint](#)
 - Parameter [SpiPinConfiguration](#)
 - Parameter [SpiPhyUnitAsyncUseDma](#)
 - Parameter [SpiMaxDmaFastTransfer](#)
 - Reference [SpiPhyUnitClockRef](#)
 - Reference [SpiPhyUnitAlternateClockRef](#)
 - Reference [SpiPhyTxDmaChannel](#)
 - Reference [SpiPhyRxDmaChannel](#)
 - Reference [SpiFlexioTxAndClkChannelsConfig](#)
 - Reference [SpiFlexioRxAndCsChannelsConfig](#)
- Container [SpiPublishedInformation](#)
 - * Parameter [SpiMaxHwUnit](#)
- Container [CommonPublishedInformation](#)
 - * Parameter [ArReleaseMajorVersion](#)
 - * Parameter [ArReleaseMinorVersion](#)
 - * Parameter [ArReleaseRevisionVersion](#)
 - * Parameter [ModuleId](#)
 - * Parameter [SwMajorVersion](#)
 - * Parameter [SwMinorVersion](#)
 - * Parameter [SwPatchVersion](#)
 - * Parameter [VendorApiInfix](#)
 - * Parameter [VendorId](#)
- Container [SpiAutosarExt](#)
 - * Parameter [SpiEnableUserModeSupport](#)
 - * Parameter [SpiEnableDmaFastTransferSupport](#)
 - * Parameter [SpiHalfDuplexModeSupport](#)
 - * Parameter [SpiAllowBigSizeCollections](#)

- * Parameter [SpiEnableHWUnitAsyncMode](#)
- * Parameter [SpiJobStartNotificationEnable](#)
- * Parameter [SpiDisableDemReportErrorStatus](#)
- * Parameter [SpiUseLegacyEBLength](#)
- * Parameter [SpiFlexioEnable](#)

4.1.1 Module Spi

Configuration of the Spi (Serial Peripheral Interface) module.

Included containers:

- [SpiDemEventParameterRefs](#)
- [SpiDriver](#)
- [SpiGeneral](#)
- [SpiPublishedInformation](#)
- [CommonPublishedInformation](#)
- [SpiAutosarExt](#)

Property	Value
type	ECUC-MODULE-DEF
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantSupport	true
supportedConfigVariants	VARIANT-PRE-COMPILE, VARIANT-LINK-TIME, VARIANT-POST-BUILD

4.1.2 Container SpiDemEventParameterRefs

Container for the references to DemEventParameter elements which shall be invoked using the Dem_SetEventStatus API in case the corresponding error occurs.

The EventId is taken from the referenced DemEventParameter's DemEventId value. The standardized errors are provided in the container and can be extended by vendor specific error references.

Included subcontainers:

- None

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantMultiplicity	false
multiplicityConfigClasses	VARIANT-LINK-TIME: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE

4.1.3 Reference SPI_E_HARDWARE_ERROR

Reference to configured DEM event to report "Hardware failure". If the reference is not configured the error shall not be reported.

Property	Value
type	ECUC-REFERENCE-DEF
origin	AUTOSAR_ECUC
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantMultiplicity	false
multiplicityConfigClasses	VARIANT-LINK-TIME: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
postBuildVariantValue	false
valueConfigClasses	VARIANT-LINK-TIME: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
requiresSymbolicNameValue	False
destination	/AUTOSAR/EcucDefs/Dem/DemConfigSet/DemEventParameter

4.1.4 Container SpiDriver

This container contains the configuration parameters and sub containers of the AUTOSAR Spi module.

Included subcontainers:

- [SpiChannel](#)
- [SpiExternalDevice](#)
- [SpiBaudrateConfig](#)
- [SpiJob](#)
- [SpiSequence](#)

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A

4.1.5 Parameter SpiMaxChannel

This parameter contains the number of Channels configured. It will be gathered by tools during the configuration stage.

Note This parameter is not used, instead max channel value is derived from number of channels configured.

Property	Value
type	ECUC-INTEGER-PARAM-DEF

Property	Value
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantMultiplicity	true
multiplicityConfigClasses	VARIANT-LINK-TIME: LINK
	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
postBuildVariantValue	true
valueConfigClasses	VARIANT-LINK-TIME: LINK
	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
defaultValue	0
max	65535
min	0

4.1.6 Parameter SpiMaxJob

This parameter contains the number of Jobs configured. It will be gathered by tools during the configuration stage.

Note This parameter is not used, instead max jobs value is derived from number of jobs configured.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantMultiplicity	true
multiplicityConfigClasses	VARIANT-LINK-TIME: LINK
	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
postBuildVariantValue	true
valueConfigClasses	VARIANT-LINK-TIME: LINK
	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
defaultValue	0
max	65535
min	0

4.1.7 Parameter SpiMaxSequence

This parameter contains the number of Sequences configured. It will be gathered by tools during the configuration stage.

Note This parameter is not used, instead max Sequences value is derived from number of sequences configured.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantMultiplicity	true
multiplicityConfigClasses	VARIANT-LINK-TIME: LINK
	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
postBuildVariantValue	true
valueConfigClasses	VARIANT-LINK-TIME: LINK
	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
defaultValue	0
max	65535
min	0

4.1.8 Container SpiChannel

All data needed to configure one SPI-channel.

Included subcontainers:

- None

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	1
upperMultiplicity	Infinite
postBuildVariantMultiplicity	false
multiplicityConfigClasses	VARIANT-LINK-TIME: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE

4.1.9 Parameter SpiChannelId

SPI Channel ID, used as parameter in SPI API functions.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	true
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A

Property	Value
postBuildVariantValue	false
valueConfigClasses	VARIANT-LINK-TIME: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
defaultValue	0
max	255
min	0

4.1.10 Parameter SpiChannelType

Buffer usage with EB/IB channel.

Note This parameter is dependant on SpiChannelBuffersAllowed parameter.

When SpiChannelBuffersAllowed = 0; SpiChannelType should be IB

When SpiChannelBuffersAllowed = 1; SpiChannelType should be EB

When SpiChannelBuffersAllowed = 2; SpiChannelType can be IB or EB

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-LINK-TIME: LINK
	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
defaultValue	IB
literals	['EB', 'IB']

4.1.11 Parameter SpiDataWidth

This parameter is the width of a transmitted data unit.

Note The hardware supports data width from 1 to 64 bit. The unit is in bits.

When SpiChannelBuffersAllowed = 0; SpiChannelType should be IB

When SpiChannelBuffersAllowed = 1; SpiChannelType should be EB

When SpiChannelBuffersAllowed = 2; SpiChannelType can be IB or EB

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false

Configuration Parameters

Property	Value
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-LINK-TIME: LINK
	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
defaultValue	32
max	64
min	1

4.1.12 Parameter SpiDefaultData

The default data to be transmitted when (for internal buffer or external buffer)

the pointer passed to Spi_WriteIB (for internal buffer) or to Spi_SetupEB (for external buffer) is NULL.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantMultiplicity	true
multiplicityConfigClasses	VARIANT-LINK-TIME: LINK
	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
postBuildVariantValue	true
valueConfigClasses	VARIANT-LINK-TIME: LINK
	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
defaultValue	1
max	4294967295
min	0

4.1.13 Parameter SpiEbMaxLength

This parameter contains the maximum number of data elements of EB Channels.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	0
upperMultiplicity	1

Property	Value
postBuildVariantMultiplicity	true
multiplicityConfigClasses	VARIANT-LINK-TIME: LINK
	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
postBuildVariantValue	true
valueConfigClasses	VARIANT-LINK-TIME: LINK
	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
defaultValue	1024
max	1048576
min	1

4.1.14 Parameter SpiIbNBuffers

This parameter contains the maximum number of data buffers in case of IB Channels and only.

In case of channel's Spi_DataWidth ranges from 9 to 16, this parameter refers to

the number of bytes allocated to the buffers and MUST be even. Or divisible by 4 if the range from 17 to 32.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantMultiplicity	true
multiplicityConfigClasses	VARIANT-LINK-TIME: LINK
	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
postBuildVariantValue	true
valueConfigClasses	VARIANT-LINK-TIME: LINK
	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
defaultValue	1
max	65535
min	1

4.1.15 Parameter SpiTransferStart

This parameter defines the first starting bit for transmission.

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1

Property	Value
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-LINK-TIME: LINK
	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
defaultValue	LSB
literals	['LSB', 'MSB']

4.1.16 Parameter SpiByteSwapTransfer

Enables byte swap on each 32-bit word when transmitting and receiving data. Byte swapping can be useful when interfacing to devices that organize data as big-endian. This feature is only supported by LPSPI instance.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-LINK-TIME: LINK
	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
defaultValue	false

4.1.17 Parameter SpiChannelHalfDuplexDirection

This parameter select direction of transfer in half duplex mode

HALF_DUPLEX_TRANSMIT: Transmit only.

HALF_DUPLEX_RECEIVE: Receive only, supports DataWidth 1->32bits.

NoteThis parameter only is used in Half Duplex mode.

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantMultiplicity	false
multiplicityConfigClasses	VARIANT-LINK-TIME: LINK

Property	Value
	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
postBuildVariantValue	true
	VARIANT-LINK-TIME: LINK
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
defaultValue	HALF_DUPLEX_TRANSMIT
literals	['HALF_DUPLEX_TRANSMIT', 'HALF_DUPLEX_RECEIVE']

4.1.18 Reference SpiChannelEcucPartitionRef

Maps an SPI Channel to zero or ECUC partition to limit the access to this Channel. The ECUC partition referenced is a subset of the ECUC partitions where the SPI driver is mapped to.

Property	Value
type	ECUC-REFERENCE-DEF
origin	NXP
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantMultiplicity	true
	VARIANT-LINK-TIME: PRE-COMPILE
multiplicityConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
postBuildVariantValue	true
	VARIANT-LINK-TIME: PRE-COMPILE
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
requiresSymbolicNameValue	False
destination	/AUTOSAR/EcucDefs/EcuC/EcucPartitionCollection/EcucPartition

4.1.19 Container SpiExternalDevice

The communication settings of an external device. Closely linked to SpiJob.

Included subcontainers:

- None

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	1
upperMultiplicity	Infinite
postBuildVariantMultiplicity	false
	VARIANT-LINK-TIME: PRE-COMPILE
multiplicityConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE

4.1.20 Parameter SpiBaudrate

This parameter is the communication baudrate. This parameter allows using a range of values, from the point of view of configuration tools, from Hz up to MHz.

This field is used only in MASTER mode.

This field is used only for LPSPI

The baudrate must be in allowed range

LPSPI in normal Master mode:

- RUN MODE: [1 to 10Mhz]

- HSRUN MODE: [1 to 14Mhz] only support for K14x

- VLPR MODE: [1 to 2Mhz] for K1xx and [1 to 500Khz] for K14xW

FLEXIO in Master:

Note: Due to synchronization delays, the setup time for the serial input data is 1.5 FlexIO clock cycles, so the maximum baud rate is divide by 4 of the FlexIO clock frequency.

Theoretical baud rate =

$FLEXIO_CLK (BUS_CLK) \div (16 \div 2 \div (TIMCMPn[COMP] + 1))$

$FLEXIO_CLK (BUS_CLK) \div (2 \div (TIMCMPn[COMP] + 1))$

- RUN MODE: [1 to $(BUS_CLK/4) \div (2 \div (TIMCMPn[COMP] + 1)) = 6Mhz$]

- HSRUN MODE: [1 to $(BUS_CLK/4) \div (2 \div (TIMCMPn[COMP] + 1)) = 7Mhz$] only support for K14x

- VLPR MODE: [1 to $(BUS_CLK/4) \div (2 \div (TIMCMPn[COMP] + 1)) = 0.5Mhz$] for K11x, K14x and [1 to $(BUS_CLK/4) \div (2 \div (TIMCMPn[COMP] + 1)) = 62.5KhzMhz$] for K14xW

NoteThe precision of this value depends SPI clock source configuration. If the driver cannot generate correct of the value, approximate value will be used.

Property	Value
type	ECUC-FLOAT-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-LINK-TIME: LINK
	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
defaultValue	1000000.0
max	1.4E7
min	1.0

4.1.21 Parameter SpiUseBaudrateConfig

Calculate spi baudrate based on parameter values

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-LINK-TIME: LINK
	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
defaultValue	false

4.1.22 Parameter SpiCalculatedBaudrate

This parameter is the communication baudrate - This parameter allows using a range of values, from the point of view of configuration tools, from Hz up to MHz.

This field is used only in MASTER mode.

NoteThe precision of this value depends SPI clock source configuration. If the driver cannot generate correct of the value, approximate value will be used.

Property	Value
type	ECUC-FLOAT-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantMultiplicity	true
multiplicityConfigClasses	VARIANT-LINK-TIME: LINK
	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
postBuildVariantValue	true
valueConfigClasses	VARIANT-LINK-TIME: LINK
	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
defaultValue	1000000.0
max	2.5E7
min	1.0

4.1.23 Parameter SpiCalculatedAlternateBaudrate

This parameter is the communication baudrate - This parameter allows using a range of values, from the point of view of configuration tools, from Hz up to MHz.

This field is used only in MASTER mode.

Configuration Parameters

*Note*The precision of this value depends SPI clock source configuration. If the driver cannot generate correct of the value, approximate value will be used.

Property	Value
type	ECUC-FLOAT-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantMultiplicity	true
multiplicityConfigClasses	VARIANT-LINK-TIME: LINK
	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
postBuildVariantValue	true
valueConfigClasses	VARIANT-LINK-TIME: LINK
	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
defaultValue	1000000.0
max	2.5E7
min	1.0

4.1.24 Parameter SpiCsIdentifier

This parameter is the symbolic name to identify the Chip Select (CS) allocated to this Job.

The chip selects are specific per HwUnit. Please check in Reference Manual for information on available chip selects.

If FLEXIO channel used, Chip Select will be configured by SpiPhyUnit\SpiFlexioCsPinSelect and this parameter will be not used.

If SpiEnableCs is not set, value of this node will not be used by driver code. It should set to default value (PCS0)

Property	Value
type	ECUC-STRING-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	true
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-LINK-TIME: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
defaultValue	PCS0

4.1.25 Parameter SpiCsPolarity

This parameter defines the active polarity of Chip Select.

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-LINK-TIME: LINK
	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
defaultValue	HIGH
literals	['HIGH', 'LOW']

4.1.26 Parameter SpiCsSelection

When the Chip select handling is enabled (see *SpiEnableCs*), then this parameter specifies if the chip select is handled automatically by Peripheral HW engine or via general purpose IO by Spi driver.

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantMultiplicity	true
multiplicityConfigClasses	VARIANT-LINK-TIME: LINK
	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
postBuildVariantValue	true
valueConfigClasses	VARIANT-LINK-TIME: LINK
	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
defaultValue	CS_VIA_PERIPHERAL_ENGINE
literals	['CS_VIA_PERIPHERAL_ENGINE', 'CS_VIA_GPIO']

4.1.27 Parameter SpiDataShiftEdge

This parameter defines the SPI data shift edge.

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	AUTOSAR_ECUC

Property	Value
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-LINK-TIME: LINK
	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
defaultValue	LEADING
literals	['LEADING', 'TRAILING']

4.1.28 Parameter SpiEnableCs

This parameter enables or not the Chip Select handling functions.

This parameter is closely linked to Job.If This parameter is True,then chip select is asserted and if False No chip select is asserted.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-LINK-TIME: LINK
	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
defaultValue	true

4.1.29 Parameter SpiHwUnit

This parameter is the symbolic name to identify the HW SPI Hardware microcontroller peripheral allocated to this Job.

CSIBn references the n-th logical unit configured in SpiPhyUnit container. For example: CSIB0 references the first logical unit (not the first SPI_0 HW unit).

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1

Property	Value
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-LINK-TIME: LINK
	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
defaultValue	CSIB0
literals	['CSIB0', 'CSIB1', 'CSIB2', 'CSIB3', 'CSIB4']

4.1.30 Parameter SpiShiftClockIdleLevel

This parameter defines the SPI shift clock idle level.

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-LINK-TIME: LINK
	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
defaultValue	HIGH
literals	['HIGH', 'LOW']

4.1.31 Parameter SpiTimeClk2Cs

Timing between clock and chip select (in seconds) - This parameter allows to use a range of values from 30ns up to 0.01s.

Property	Value
type	ECUC-FLOAT-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-LINK-TIME: LINK
	VARIANT-PRE-COMPILE: PRE-COMPILE

Property	Value
	VARIANT-POST-BUILD: POST-BUILD
default Value	1.0E-6
max	0.01
min	1.0E-8

4.1.32 Parameter SpiTimeCs2Clk

Timing between chip select assertion and clock (in seconds) - This parameter allows to use a range of values from 30ns up to 0.01s.

Property	Value
type	ECUC-FLOAT-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-LINK-TIME: LINK
	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
default Value	1.0E-6
max	0.01
min	1.0E-8

4.1.33 Parameter SpiTimeCs2Cs

Timing between the negation of the chip select at the end of frame and the assertion of the chip select at the beginning of the next frame (in seconds) - This parameter allows to use a range of values from 10ns up to 0.01s.

For DSPI_IP in Cs continuous mode, SpiTimeCs2Cs is not inserted between the transfers.

For LPSPI_IP in Cs continuous mode, SpiTimeCs2Cs will be indirectly written to CCR1[SCKSCK] - the delay from the last SCK pulse of a frame and the first SCK pulse of the following frame.

Property	Value
type	ECUC-FLOAT-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-LINK-TIME: LINK
	VARIANT-PRE-COMPILE: PRE-COMPILE

Property	Value
	VARIANT-POST-BUILD: POST-BUILD
defaultValue	1.0E-6
max	0.01
min	1.0E-8

4.1.34 Parameter SpiCsBehavior

This parameter is used to define the chip select behavior.

Either the CS is toggled for each data frame (bit frame on the SPI bus in relation with SpiDataWidth) inside the channel(s) composing the job or the CS is kept asserted for the whole job.

Note: For Flexio, don't use these configurations: (SPI_SLAVE, TRAILING), (SPI_SLAVE, LEADING, SpiShiftClockIdleLevel = HIGH).

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-LINK-TIME: LINK
	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
defaultValue	CS_KEEP_ASSERTED
literals	['CS_KEEP_ASSERTED', 'CS_TOGGLE']

4.1.35 Parameter SpiHostRequest

In Master mode, allows LPSPI to start a new SPI bus transfer only if the host request input is asserted.

When LPSPI is busy, the host request input is ignored.

In Slave mode, causes the HREQ output pin to assert when data is available to be transmitted.

Source of the host request can be the HREQ Pin (in this case the PCS[1] function is disabled) or it can be an input trigger.

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A

Configuration Parameters

Property	Value
postBuildVariantValue	true
valueConfigClasses	VARIANT-LINK-TIME: LINK
	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
defaultValue	DISABLE
literals	['DISABLE', 'HREQ_PIN_ACTIVE_HIGH', 'HREQ_PIN_ACTIVE_LOW', 'INPUT_TRIGGER_ACTIVE_HIGH', 'INPUT_TRIGGER_ACTIVE_LOW']

4.1.36 Parameter SpiDeviceHalfDuplexSupport

This parameter enables or not half duplex mode for this external device.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-LINK-TIME: LINK
	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
defaultValue	false

4.1.37 Parameter SpiTransferWidth

In half duplex mode, this will select the number of bits are transfered on per cycle clock

TRANSFER_1_BIT: 1 bit will be transfered on per cycle clock

TRANSFER_2_BIT: 2 bits will be transfered on per cycle clock

TRANSFER_4_BIT: 4 bits will be transfered on per cycle clock

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantMultiplicity	false
multiplicityConfigClasses	VARIANT-LINK-TIME: LINK
	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD

Property	Value
postBuildVariantValue	true
valueConfigClasses	VARIANT-LINK-TIME: LINK
	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
defaultValue	TRANSFER_1_BIT
literals	['TRANSFER_1_BIT', 'TRANSFER_2_BIT', 'TRANSFER_4_BIT']

4.1.38 Parameter SpiHalfDuplexPinSelect

This node will support to select which Pin (SIN or SOUT) will be used for half duplex 1 bit mode.

HALF_DUPLEX_SIN: SIN will be used to transfer

HALF_DUPLEX_SOUT: SOUT will be used to transfer

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantMultiplicity	false
multiplicityConfigClasses	VARIANT-LINK-TIME: LINK
	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
postBuildVariantValue	true
valueConfigClasses	VARIANT-LINK-TIME: LINK
	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
defaultValue	HALF_DUPLEX_SOUT
literals	['HALF_DUPLEX_SIN', 'HALF_DUPLEX_SOUT']

4.1.39 Reference SpiBaudrateConfigRef

Config from which to calculate SpiBaudrate.

Property	Value
type	ECUC-REFERENCE-DEF
origin	NXP
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantMultiplicity	true
multiplicityConfigClasses	VARIANT-LINK-TIME: LINK
	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
postBuildVariantValue	true

Property	Value
valueConfigClasses	VARIANT-LINK-TIME: LINK
	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
requiresSymbolicNameValue	False
destination	/TS_T40D2M30I0R0/Spi/SpiDriver/SpiBaudrateConfig

4.1.40 Reference SpiDeviceEcucPartitionRef

ECUC_Spi_00246. Maps an SPI external device to zero or multiple ECUC partitions to limit the access to this external device. The ECUC partitions referenced are a subset of the ECUC partitions where the SPI driver is mapped to.

Property	Value
type	ECUC-REFERENCE-DEF
origin	AUTOSAR_ECUC
lowerMultiplicity	0
upperMultiplicity	Infinite
postBuildVariantMultiplicity	true
multiplicityConfigClasses	VARIANT-LINK-TIME: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
postBuildVariantValue	true
valueConfigClasses	VARIANT-LINK-TIME: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
requiresSymbolicNameValue	False
destination	/AUTOSAR/EcucDefs/EcuC/EcucPartitionCollection/EcucPartition

4.1.41 Container SpiBaudrateConfig

Manual configuration of parameters for baudrate generation.

*Baudrate = Function clock / (SpiPrescalerValue * (SpiScaler + 2))*

Included subcontainers:

- [LpspiConfig](#)

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	0
upperMultiplicity	Infinite
postBuildVariantMultiplicity	false
multiplicityConfigClasses	VARIANT-LINK-TIME: LINK
	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD

4.1.42 Parameter IpType

Select ip to enable its configuration

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-LINK-TIME: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
defaultValue	LPSPi
literals	['LPSPi']

4.1.43 Container LpspiConfig

Configuration for LPSPi buadrate parameters

Included subcontainers:

- None

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A

4.1.44 Parameter SpiPrescaler

Selects the prescaler value for the baud rate. Scaler value is 2^x | x beeing the selected value.

E.g. If 1 is set -> Prescaler = 2

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A

Property	Value
postBuildVariantValue	false
valueConfigClasses	VARIANT-LINK-TIME: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
defaultValue	0
max	7
min	0

4.1.45 Parameter SpiPrescalerValue

Calculated value of the prescaler bit

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-LINK-TIME: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
defaultValue	1
max	128
min	1

4.1.46 Parameter SpiScaler

Selects the scaler value for the baud rate.

$SpiScalerValue = SpiScaler + 2$

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-LINK-TIME: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE

Property	Value
defaultValue	0
max	255
min	0

4.1.47 Container SpiJob

All data needed to configure one SPI-Job, amongst others the connection between the internal SPI unit and the special settings for an external device is done.

Included subcontainers:

- [SpiChannelList](#)

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	1
upperMultiplicity	Infinite
postBuildVariantMultiplicity	false
multiplicityConfigClasses	VARIANT-LINK-TIME: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE

4.1.48 Parameter SpiJobEndNotification

This parameter is a reference to a notification function.

Property	Value
type	ECUC-FUNCTION-NAME-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantMultiplicity	true
multiplicityConfigClasses	VARIANT-LINK-TIME: LINK
	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
postBuildVariantValue	true
valueConfigClasses	VARIANT-LINK-TIME: LINK
	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
defaultValue	NULL_PTR

4.1.49 Parameter SpiJobStartNotification

This parameter is a reference to a notification function.

Property	Value
type	ECUC-FUNCTION-NAME-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantMultiplicity	true
multiplicityConfigClasses	VARIANT-LINK-TIME: LINK
	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
postBuildVariantValue	true
valueConfigClasses	VARIANT-LINK-TIME: LINK
	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
default Value	NULL_PTR

4.1.50 Parameter SpiJobId

SPI Job ID, used as parameter in SPI API functions.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	true
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-LINK-TIME: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
default Value	0
max	65535
min	0

4.1.51 Parameter SpiJobPriority

Priority set accordingly to SPI093: 0, lowest, 3, highest priority

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1

Property	Value
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-LINK-TIME: LINK
	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
defaultValue	0
max	3
min	0

4.1.52 Reference SpiDeviceAssignment

Reference to the external device used by this job.

Property	Value
type	ECUC-REFERENCE-DEF
origin	AUTOSAR_ECUC
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-LINK-TIME: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
requiresSymbolicNameValue	False
destination	/AUTOSAR/EcucDefs/Spi/SpiDriver/SpiExternalDevice

4.1.53 Container SpiChannelList

References to SPI channels and their order within the Job.

Included subcontainers:

- None

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	1
upperMultiplicity	Infinite
postBuildVariantMultiplicity	false
multiplicityConfigClasses	VARIANT-LINK-TIME: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE

4.1.54 Parameter SpiChannelIndex

This parameter specifies the order of Channels within the Job.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-LINK-TIME: LINK
	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
default Value	0
max	65535
min	0

4.1.55 Reference SpiChannelAssignment

A job references several channels.

Property	Value
type	ECUC-REFERENCE-DEF
origin	AUTOSAR_ECUC
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-LINK-TIME: LINK
	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
requiresSymbolicNameValue	False
destination	/AUTOSAR/EcucDefs/Spi/SpiDriver/SpiChannel

4.1.56 Container SpiSequence

All data needed to configure one SPI-sequence.

Included subcontainers:

- None

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF

Property	Value
lowerMultiplicity	1
upperMultiplicity	Infinite
postBuildVariantMultiplicity	false
multiplicityConfigClasses	VARIANT-LINK-TIME: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE

4.1.57 Parameter SpiInterruptibleSequence

This parameter allows or not this Sequence to be suspended by another one.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-LINK-TIME: LINK
	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
defaultValue	false

4.1.58 Parameter SpiSeqEndNotification

This parameter is a reference to a notification function.

Property	Value
type	ECUC-FUNCTION-NAME-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantMultiplicity	true
multiplicityConfigClasses	VARIANT-LINK-TIME: LINK
	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
postBuildVariantValue	true
valueConfigClasses	VARIANT-LINK-TIME: LINK
	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
defaultValue	NULL_PTR

4.1.59 Parameter SpiSequenceId

Sequence ID of configured SPI Sequence.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	true
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-LINK-TIME: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
defaultValue	0
max	255
min	0

4.1.60 Parameter SpiEnableDmaFastTransfer

When this parameter is enabled, this Sequence will be transferred using DMA ScatterGather and CPU used only for processing end of Sequence.

SpiAutosarExt/SpiEnableDmaFastTransferSupport must be checked to support this feature.

Note: This feature requires:

- 1. The parameters SpiBaudrate, SpiHwUnit, SpiTimeClk2Cs, SpiTimeCs2Clk, SpiTimeCs2Cs in External Device linked to each Job in this Sequence must be the same.*
- 2. The parameters SpiDataWidth and SpiTransferStart in Channel assigned to each Job in this Sequence must be the same.*
- 3. In each Channel, the number of data buffers is NOT higher than 32767 if SpiDataWidth < 9. So, SpiNbBuffers and SpiEbMaxLength must be assigned to suitable values.*
- 4. Only Master mode is supported.*

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-LINK-TIME: LINK
	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
defaultValue	false

4.1.61 Reference SpiJobAssignment

A sequence references several jobs, which are executed during a communication sequence.

Property	Value
type	ECUC-REFERENCE-DEF
origin	AUTOSAR_ECUC
lowerMultiplicity	1
upperMultiplicity	Infinite
postBuildVariantMultiplicity	true
multiplicityConfigClasses	VARIANT-LINK-TIME: LINK
	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
postBuildVariantValue	true
valueConfigClasses	VARIANT-LINK-TIME: LINK
	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
requiresSymbolicNameValue	False
destination	/AUTOSAR/EcucDefs/Spi/SpiDriver/SpiJob

4.1.62 Container SpiGeneral

General configuration settings for SPI-Handler.

Included subcontainers:

- [SpiPhyUnit](#)

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A

4.1.63 Parameter SpiMultipartitionSupport

SpiMultiPartitionEnable

When this parameter is enabled, multi-partition feature will be used in SPI driver.

That means mapping the SPI driver to multiple ECUC partitions to make the module API available in this partition.

The SPI driver will operate as an independent instance in each of the partitions.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1

Property	Value
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-LINK-TIME: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
defaultValue	false

4.1.64 Parameter SpiCancelApi

Switches the *Spi_Cancel* function ON or OFF.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-LINK-TIME: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
defaultValue	true

4.1.65 Parameter SpiChannelBuffersAllowed

Selects the SPI Handler/Driver Channel Buffers usage allowed and delivered.

Note

0 - Only Internal Buffers (IB) are allowed

1 - Only External buffers (EB) are allowed

2 - Both Internal (IB) and External (EB) buffers are allowed

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A

Property	Value
postBuildVariantValue	false
valueConfigClasses	VARIANT-LINK-TIME: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
defaultValue	0
max	2
min	0

4.1.66 Parameter SpiDevErrorDetect

Switches the Development Error Detection and Notification ON or OFF.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-LINK-TIME: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
defaultValue	true

4.1.67 Parameter SpiHwStatusApi

Switches the Spi_GetHWUnitStatus function ON or OFF.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-LINK-TIME: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
defaultValue	true

4.1.68 Parameter SpiInterruptibleSeqAllowed

Switches the Interruptible Sequences handling functionality ON or OFF.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-LINK-TIME: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
default Value	false

4.1.69 Parameter SpiLevelDelivered

Selects the SPI Handler/Driver level of scalable functionality that is available and delivered.

Note

Level 0 Only Simple Synchronous Behavior

Level 1 Basic Asynchronous Behaviour

Level 2 Enhanced Behaviour

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-LINK-TIME: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
default Value	2
max	2
min	0

4.1.70 Parameter SpiMainFunctionPeriod

This parameter defines the cycle time of the function Spi_MainFunction_Handling in seconds.

The parameter is not used by the driver it self, but it is used by upper layer.

Property	Value
type	ECUC-FLOAT-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantMultiplicity	false
multiplicityConfigClasses	VARIANT-LINK-TIME: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
postBuildVariantValue	false
valueConfigClasses	VARIANT-LINK-TIME: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
defaultValue	0.01
max	1.0
min	1.0E-7

4.1.71 Parameter SpiSupportConcurrentSyncTransmit

Specifies whether concurrent Spi_SyncTransmit() calls for different se-quences shall be configurable.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-LINK-TIME: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
defaultValue	false

4.1.72 Parameter SpiVersionInfoApi

Switches the Spi_GetVersionInfo function ON or OFF.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false

Configuration Parameters

Property	Value
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-LINK-TIME: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
default Value	true

4.1.73 Parameter SpiGlobalDmaEnable

If checked, it allows using the DMA module during the transfer.

For each SPI unit a transferring method can be configured: FIFO or DMA.

If not checked, all SPI units will use FIFO transferring mode.

Note This is an implementation parameter.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-LINK-TIME: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
default Value	false

4.1.74 Parameter SpiTimeoutMethod

SpiTimeoutMethod

Configures the timeout method.

Based on this selection a certain timeout method from *OsIf* will be used in the driver.

Note: If *SystemTimer* or *CustomTimer* are selected make sure the corresponding timer is enabled in *OsIf* General configuration.

Note: Implementation Specific Parameter.

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	NXP
symbolicNameValue	false

Property	Value
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-LINK-TIME: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
defaultValue	OSIF_COUNTER_DUMMY
literals	['OSIF_COUNTER_DUMMY', 'OSIF_COUNTER_SYSTEM', 'OSIF_COUNTER_CUSTOM']

4.1.75 Parameter SpiTransmitTimeout

Timeout value (microseconds) used to wait for TX/RX transmission to complete one frame in both full duplex and half duplex mode. *Note* This is an implementation parameter. The transmission will be unsuccessful if the Chip cannot completely transfer one frame during this timeout.

The precision of this value is quite low, it must be greater than the time needed to completely transmit one frame.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-LINK-TIME: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
defaultValue	50000
max	900000
min	1

4.1.76 Reference SpiEcucPartitionRef

ECUC_Spi_00244. Maps the SPI driver to zero or multiple ECUC partitions to make the driver API available in the according partition.

Property	Value
type	ECUC-REFERENCE-DEF
origin	AUTOSAR_ECUC
lowerMultiplicity	0

Property	Value
upperMultiplicity	Infinite
postBuildVariantMultiplicity	true
multiplicityConfigClasses	VARIANT-LINK-TIME: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
postBuildVariantValue	true
valueConfigClasses	VARIANT-LINK-TIME: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
requiresSymbolicNameValue	False
destination	/AUTOSAR/EcuDefs/EcuC/EcuPartitionCollection/EcuPartition

4.1.77 Reference SpiKernelEcuPartitionRef

ECUC_Spi_00245.Maps the SPI kernel to zero or one ECUC partitions to assign the driver

kernel to a certain core. The ECUC partition referenced is a subset of the

ECUC partitions where the SPI driver is mapped to. SPI driver is implemented according to MultiPartition type II, so this node is not used.

Property	Value
type	ECUC-REFERENCE-DEF
origin	AUTOSAR_ECUC
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantMultiplicity	true
multiplicityConfigClasses	VARIANT-LINK-TIME: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
postBuildVariantValue	true
valueConfigClasses	VARIANT-LINK-TIME: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
requiresSymbolicNameValue	False
destination	/AUTOSAR/EcuDefs/EcuC/EcuPartitionCollection/EcuPartition

4.1.78 Container SpiPhyUnit

Logical to Physical SPI Bus mapping.

Note This is an implementation specific container.

Included subcontainers:

- None

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF

Property	Value
lowerMultiplicity	1
upperMultiplicity	Infinite
postBuildVariantMultiplicity	false
multiplicityConfigClasses	VARIANT-LINK-TIME: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE

4.1.79 Parameter SpiPhyUnitMapping

Logical SpiHWunit to physical LPSPi_[0/1/2/3/4] or LPSPi_[0/1/2/3/4] assignment. It depends on the number of units present in the chip version.

*Note*This is an implementation specific parameter.

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-LINK-TIME: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
defaultValue	LPSPi_1
literals	['LPSPi_0', 'LPSPi_1', 'LPSPi_2', 'FLEXIO_SPI_0', 'FLEXIO_SPI_1']

4.1.80 Parameter SpiPhyUnitMode

Select between SPI_MASTER and SPI_SLAVE modes.

SPI Slave mode support only if SpiGeneral/SpiLevelDelivered is 1 or 2

*Note*This is an implementation specific parameter.

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-LINK-TIME: PRE-COMPILE

Property	Value
	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
defaultValue	SPI_MASTER
literals	['SPI_MASTER', 'SPI_SLAVE']

4.1.81 Parameter SpiSamplePoint

When set, the LPSPI master will sample the input data on a delayed LPSPI_SCK edge, which improves the setup time when sampling data.

? The input data setup time in master mode with delayed LPSPI_SCK edge is equal to the input data setup time in slave mode

? In slave mode, the SAMPLE bit is ignored

0b - Input data is sampled on SCK edge

1b - Input data is sampled on delayed SCK edge

Note This is an implementation specific parameter.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-LINK-TIME: LINK
	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
defaultValue	0
max	1
min	0

4.1.82 Parameter SpiPinConfiguration

Configures which pins are used for input and output data during serial transfers. When performing parallel transfers, the Pin Configuration field is ignored.

00b - SIN is used for input data and SOUT is used for output data

01b - SIN is used for both input and output data, only half-duplex serial transfers are supported

10b - SOUT is used for both input and output data, only half-duplex serial transfers are supported

11b - SOUT is used for input data and SIN is used for output data

Note This is an implementation specific parameter.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-LINK-TIME: LINK
	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
defaultValue	0
max	3
min	0

4.1.83 Parameter SpiPhyUnitAsyncUseDma

Select Asynchronous mechanism with DMA or not.

Note This is an implementation parameter.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-LINK-TIME: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
defaultValue	false

4.1.84 Parameter SpiMaxDmaFastTransfer

Number of transfer section allowed in Dma Fast transfer.

Note: This feature will be supported if SpiEnableDmaFastTransferSupport and SpiPhyUnitAsyncUseDma checked.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	0

Property	Value
upperMultiplicity	1
postBuildVariantMultiplicity	false
multiplicityConfigClasses	VARIANT-LINK-TIME: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
postBuildVariantValue	false
valueConfigClasses	VARIANT-LINK-TIME: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
defaultValue	1
max	65535
min	0

4.1.85 Reference SpiPhyUnitClockRef

Reference to the SPI clock source configuration, which is set into the MCU driver configuration.

This clock source is used for configure SPI baudrate.

Property	Value
type	ECUC-REFERENCE-DEF
origin	NXP
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-LINK-TIME: LINK
	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
requiresSymbolicNameValue	False
destination	/AUTOSAR/EcucDefs/Mcu/McuModuleConfiguration/McuClockSetting↔ Config/McuClockReferencePoint

4.1.86 Reference SpiPhyUnitAlternateClockRef

Reference to the alternate clock configuration, retrieved from the MCU plugin.

Use to enable Spi_SetClockMode() function, which allows dual MCU clock configuration settings.

Property	Value
type	ECUC-REFERENCE-DEF
origin	NXP
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantMultiplicity	false

Property	Value
multiplicityConfigClasses	VARIANT-LINK-TIME: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
postBuildVariantValue	false
valueConfigClasses	VARIANT-LINK-TIME: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
requiresSymbolicNameValue	False
destination	/AUTOSAR/EcuDefs/Mcu/McuModuleConfiguration/McuClockSetting↔ Config/McuClockReferencePoint

4.1.87 Reference SpiPhyTxDmaChannel

SPI Master Transmit DMA Logical Channel as configured by MCL plug-in, used to prepare the SPI transmission dataframes starting from the TX buffer content.

This parameter is required only if SpiPhyUnitAsyncUseDma is checked.

Note This is an implementation specific parameter. The current SPI TX source needs be configured for enabling this DMA channel.

Property	Value
type	ECUC-CHOICE-REFERENCE-DEF
origin	NXP
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantMultiplicity	false
multiplicityConfigClasses	VARIANT-LINK-TIME: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
postBuildVariantValue	false
valueConfigClasses	VARIANT-LINK-TIME: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
requiresSymbolicNameValue	False
destinations	['/TS_T40D2M30I0R0/Mcl/MclConfig/dmaLogicChannel_Type']

4.1.88 Reference SpiPhyRxDmaChannel

SPI Receive DMA Logical Channel as configured by MCL plug-in, used to read the deserialized dataframes into the RX buffers.

This parameter is required only if SpiPhyUnitAsyncUseDma is checked.

Note This is an implementation specific parameter.

Property	Value
type	ECUC-CHOICE-REFERENCE-DEF
origin	NXP

Configuration Parameters

Property	Value
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantMultiplicity	false
multiplicityConfigClasses	VARIANT-LINK-TIME: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
postBuildVariantValue	false
valueConfigClasses	VARIANT-LINK-TIME: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
requiresSymbolicNameValue	False
destinations	['/TS_T40D2M30I0R0/Mcl/MclConfig/dmaLogicChannel_Type']

4.1.89 Reference SpiFlexioTxAndClkChannelsConfig

FLEXIO Logical Channel for TX(MOSI) and CLK(Clock). This selects 1 PIN to setup as TX(MOSI) channel and 1 PIN to set up as CLK channel

If you select CHANNEL_X, PIN_Y(FlexioMclPinId) and PIN_Z(FlexioMclAddPinId) that mean:

- ShifterX is selected for TX(MOSI) channel
- TimerX is selected for CLK channel.
- PIN_Y is selected for TX(MOSI) channel
- PIN_Z is selected for CLK channel

Note This is an implementation specific parameter.

Property	Value
type	ECUC-CHOICE-REFERENCE-DEF
origin	NXP
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantMultiplicity	false
multiplicityConfigClasses	VARIANT-LINK-TIME: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
postBuildVariantValue	false
valueConfigClasses	VARIANT-LINK-TIME: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
requiresSymbolicNameValue	False
destinations	['/TS_T40D2M30I0R0/Mcl/MclConfig/FlexioCommon/FlexioMclLogicChannels']

4.1.90 Reference SpiFlexioRxAndCsChannelsConfig

FLEXIO Logical Channel for RX(MISO) and CS(Chip select). This selects 1 PIN to setup as MISO channel and 1 PIN to set up as CS channel

If you select *CHANNEL_X*, *PIN_Y*(FlexioMclPinId) and *PIN_Z*(FlexioMclAddPinId) that mean:

- *ShifterX* is selected for *RX(MISO)* channel
- *TimerX* is selected for *CS* channel.
- *PIN_Y* is selected for *RX(MISO)* channel
- *PIN_Z* is selected for *CS* channel

Note This is an implementation specific parameter.

Property	Value
type	ECUC-CHOICE-REFERENCE-DEF
origin	NXP
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantMultiplicity	false
multiplicityConfigClasses	VARIANT-LINK-TIME: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
postBuildVariantValue	false
valueConfigClasses	VARIANT-LINK-TIME: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
requiresSymbolicNameValue	False
destinations	['/TS_T40D2M30I0R0/Mcl/MclConfig/FlexioCommon/FlexioMclLogic↔ Channels', '/TS_T40D2M30I0R0/Mcl/MclConfig/FlexioCommon/Flexio↔ MclLogicChannels']

4.1.91 Container SpiPublishedInformation

Container holding all SPI specific published information parameters.

Included subcontainers:

- None

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A

4.1.92 Parameter SpiMaxHwUnit

Number of different SPI hardware microcontroller peripherals (units/busses) available and handled by this SPI Handler/Driver module.

Property	Value
type	ECUC-INTEGER-PARAM-DEF

Property	Value
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-LINK-TIME: PUBLISHED-INFORMATION
	VARIANT-POST-BUILD: PUBLISHED-INFORMATION
	VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION
defaultValue	0
max	65535
min	0

4.1.93 Container CommonPublishedInformation

Common container, aggregated by all modules. It contains published information about vendor and versions.

Included subcontainers:

- None

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A

4.1.94 Parameter ArReleaseMajorVersion

Major version number of AUTOSAR specification on which the appropriate implementation is based on.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-LINK-TIME: LINK
	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	4

Property	Value
max	4
min	4

4.1.95 Parameter ArReleaseMinorVersion

Minor version number of AUTOSAR specification on which the appropriate implementation is based on.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-LINK-TIME: LINK
	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	7
max	7
min	7

4.1.96 Parameter ArReleaseRevisionVersion

Patch level version number of AUTOSAR specification on which the appropriate implementation is based on.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-LINK-TIME: LINK
	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	0
max	0
min	0

4.1.97 Parameter ModuleId

Module ID of this module.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-LINK-TIME: LINK
	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	83
max	83
min	83

4.1.98 Parameter SwMajorVersion

Major version number of the vendor specific implementation of the module. The numbering is vendor specific.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-LINK-TIME: LINK
	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	3
max	3
min	3

4.1.99 Parameter SwMinorVersion

Minor version number of the vendor specific implementation of the module. The numbering is vendor specific.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP

Property	Value
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-LINK-TIME: LINK
	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	0
max	0
min	0

4.1.100 Parameter SwPatchVersion

Patch level version number of the vendor specific implementation of the module. The numbering is vendor specific.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-LINK-TIME: LINK
	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	0
max	0
min	0

4.1.101 Parameter VendorApiInfix

In driver modules which can be instantiated several times on a single ECU, BSW00347 requires that the name of APIs is extended by the VendorId and a vendor specific name.

This parameter is used to specify the vendor specific name. In total, the implementation specific name is generated as follows:

<ModuleName>_>VendorId>_<VendorApiInfix>.

E.g. assuming that the VendorId of the implementor is 123 and the implementer chose a VendorApiInfix of "v11r456" a api name Can_Write defined in the SWS will translate to Can_123_v11r456Write.

This parameter is mandatory for all modules with upper multiplicity > 1. It shall not be used for modules with upper multiplicity =1.

Property	Value
type	ECUC-STRING-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantMultiplicity	false
multiplicityConfigClasses	VARIANT-LINK-TIME: LINK
	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
postBuildVariantValue	false
valueConfigClasses	VARIANT-LINK-TIME: LINK
	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
default Value	

4.1.102 Parameter VendorId

Vendor ID of the dedicated implementation of this module according to the AUTOSAR vendor list.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-LINK-TIME: LINK
	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
default Value	43
max	43
min	43

4.1.103 Container SpiAutosarExt

Enabling the settings of this section will configure the driver in a mode not compliant with AUTOSAR requirements.

Included subcontainers:

- None

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	1

Property	Value
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A

4.1.104 Parameter SpiEnableUserModeSupport

When this parameter is enabled, the Spi module will adapt to run from User Mode.

Note Spi module does not include registers protection. So, It is accessible to all registers in any public mode.

SPI is not affected by this field.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-LINK-TIME: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
defaultValue	false

4.1.105 Parameter SpiEnableDmaFastTransferSupport

When this parameter is enabled, the SPI module can support to transfer a Sequence with multiple Channels, Jobs using DMA ScatterGather and CPU used only for processing end of sequence transfer.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-LINK-TIME: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
defaultValue	false

4.1.106 Parameter SpiHalfDuplexModeSupport

When this parameter is enabled, the SPI module can support to transfer in half duplex mode.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-LINK-TIME: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
default Value	false

4.1.107 Parameter SpiAllowBigSizeCollections

A feature to allow more than 256 sequences, jobs, and channels.

Note Enabling this option will violate the following requirements: SPI166, SPI167, SPI168.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-LINK-TIME: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
default Value	false

4.1.108 Parameter SpiEnableHWUnitAsyncMode

Enable `Spi_SetHWUnitAsyncMode()` function, which allows defining distinct operation mode (POLLING or INTERRUPT) for each HWUnit.

Note This feature is not required by Autosar, which defines asynchronous mode configuration at driver level only.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP

Property	Value
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-LINK-TIME: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
default Value	false

4.1.109 Parameter SpiJobStartNotificationEnable

settings.

Note This feature is a SpiAutosarExt feature to enable the start job notification.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-LINK-TIME: LINK
	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
default Value	false

4.1.110 Parameter SpiDisableDemReportErrorStatus

SpiDisableDemReportErrorStatus

Switches the Diagnostic Error Reporting and Notification OFF.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A

Property	Value
postBuildVariantValue	false
valueConfigClasses	VARIANT-LINK-TIME: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
default Value	false

4.1.111 Parameter SpiUseLegacyEBLength

Set meaning of Length in Spi_SetupEB

Starting with Autosar 4.1.1 Standard the Length parameter of Spi_SetupEB is considered to be the number of data elements

In previous Spi implementations it was considered Number of bytes.

If SpiUseLegacyEBLength is set, Spi_SetupEB Length will be considered as

'Number of bytes' similar to version lower than Autosar 4.1.1

SpiUseLegacyEBLength is added to be used when transition to the new version is difficult, however it is not recommended to be used.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-LINK-TIME: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
default Value	false

4.1.112 Parameter SpiFlexioEnable

SpiFlexioEnable

If it is true, FLEXIO feature is enabled

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A

Property	Value
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-LINK-TIME: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
defaultValue	false

4.2 LPSPI

The low level driver component can only be used with S32CT Configurator in S32 Design Studio.

- Module [Lpspi](#)
 - Container [SpiDriver](#)
 - * Container [SpiExternalDevice](#)
 - Parameter [SpiBaudrate](#)
 - Parameter [SpiUseBaudrateConfig](#)
 - Parameter [SpiCsIdentifier](#)
 - Parameter [SpiCsPolarity](#)
 - Parameter [SpiDataShiftEdge](#)
 - Parameter [SpiHwUnit](#)
 - Parameter [SpiShiftClockIdleLevel](#)
 - Parameter [SpiDataWidth](#)
 - Parameter [SpiDefaultData](#)
 - Parameter [SpiTransferStart](#)
 - Parameter [SpiByteSwapTransfer](#)
 - Parameter [SpiTimeClk2Cs](#)
 - Parameter [SpiTimeCs2Clk](#)
 - Parameter [SpiTimeCs2Cs](#)
 - Parameter [SpiDeviceHalfDuplexSupport](#)
 - Parameter [SpiTransferWidth](#)
 - Parameter [SpiHalfDuplexPinSelect](#)
 - Parameter [SpiCsBehavior](#)
 - Parameter [SpiHostRequest](#)
 - Reference [SpiBaudrateConfigRef](#)
 - * Container [SpiBaudrateConfig](#)
 - Parameter [IpType](#)
 - Container [SpiGeneral](#)
 - * Parameter [SpiEnableUserModeSupport](#)

Configuration Parameters

- * Parameter [SpiEnableDmaFastTransferSupport](#)
- * Parameter [SpiHalfDuplexModeSupport](#)
- * Parameter [LpspiIpDevErrorDetect](#)
- * Parameter [SpiGlobalDmaEnable](#)
- * Parameter [SpiEnableLargeFrameSize](#)
- * Parameter [SpiTimeoutMethod](#)
- * Parameter [SpiTransmitTimeout](#)
- * Container [SpiPhyUnit](#)
 - Parameter [SpiPhyUnitMapping](#)
 - Parameter [SpiPinConfiguration](#)
 - Parameter [SpiSamplePoint](#)
 - Parameter [SpiPhyUnitClockSourceValue](#)
 - Parameter [SpiPhyUnitMode](#)
 - Parameter [SpiPhyUnitAsyncUseDma](#)
 - Parameter [SpiMaxDmaFastTransfer](#)
 - Reference [SpiPhyTxDmaChannel](#)
 - Reference [SpiPhyRxDmaChannel](#)

4.2.1 Module Lpspi

Spi Driver

Included containers:

- [SpiDriver](#)
- [SpiGeneral](#)

Property	Value
type	MODULE-DEF
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantSupport	true
supportedConfigVariants	VARIANT-PRE-COMPILE;VARIANT-LINK-TIME;VARIANT-POST-BUILD

4.2.2 Parameter LpspiIpDevErrorDetect

Switches the Development Error Detection and Notification ON or OFF.

Property	Value
type	BOOLEAN

Property	Value
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-LINK-TIME: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
defaultValue	true

4.2.3 Parameter SpiEnableLargeFrameSize

This parameter allows user to configure *SpiDataWidth* > 64.

Note This feature is only available for LPSPI on IPL.

Property	Value
type	BOOLEAN
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-LINK-TIME: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
defaultValue	false

4.2.4 Parameter SpiPhyUnitClockSourceValue

N/A

Property	Value
type	INTEGER
origin	NXP
symbolicNameValue	False
lowerMultiplicity	1

Property	Value
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-LINK-TIME: LINK
	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
defaultValue	0
max	80000000
min	0

4.3 FLEXIO_SPI

The low level driver component can only be used with S32CT Configurator in S32 Design Studio.

- Module [Flexio_Spi](#)
 - Container [SpiDriver](#)
 - * Container [SpiExternalDevice](#)
 - Parameter [SpiBaudrate](#)
 - Parameter [SpiCsIdentifier](#)
 - Parameter [SpiCsPolarity](#)
 - Parameter [SpiDataShiftEdge](#)
 - Parameter [SpiHwUnit](#)
 - Parameter [SpiShiftClockIdleLevel](#)
 - Parameter [SpiDataWidth](#)
 - Parameter [SpiDefaultData](#)
 - Parameter [SpiTransferStart](#)
 - Parameter [SpiCsBehavior](#)
 - Container [SpiGeneral](#)
 - * Parameter [SpiEnableUserModeSupport](#)
 - * Parameter [FlexioSpiIpDevErrorDetect](#)
 - * Parameter [SpiGlobalDmaEnable](#)
 - * Parameter [SpiTimeoutMethod](#)
 - * Container [SpiPhyUnit](#)
 - Parameter [SpiPhyUnitMapping](#)
 - Parameter [SpiPinConfiguration](#)
 - Parameter [SpiSamplePoint](#)
 - Parameter [SpiPhyUnitClockSourceValue](#)
 - Parameter [SpiPhyUnitMode](#)

- Parameter [SpiPhyUnitAsyncUseDma](#)
- Reference [SpiPhyTxDmaChannel](#)
- Reference [SpiPhyRxDmaChannel](#)
- Reference [SpiFlexioTxAndClkChannelsConfig](#)
- Reference [SpiFlexioRxAndCsChannelsConfig](#)

4.3.1 Module Flexio_Spi

Spi Driver

Included containers:

- [SpiDriver](#)
- [SpiGeneral](#)

Property	Value
type	MODULE-DEF
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantSupport	true
supportedConfigVariants	VARIANT-PRE-COMPILE;VARIANT-LINK-TIME;VARIANT-POST-BUILD

4.3.2 Parameter FlexioSpiIpDevErrorDetect

Switches the Development Error Detection and Notification ON or OFF.

Property	Value
type	BOOLEAN
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-LINK-TIME: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
default Value	true

4.3.3 Parameter SpiPhyUnitClockSourceValue

N/A

Property	Value
type	INTEGER
origin	NXP
symbolicNameValue	False
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-LINK-TIME: LINK
	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
defaultValue	0
max	80000000
min	0



Chapter 5

Module Index

5.1 Software Specification

Here is a list of all modules:

Flexio_Spi Driver	118
Lpspi Driver	129
Spi Driver	145

Chapter 6

Module Documentation

6.1 Flexio_Spi Driver

6.1.1 Detailed Description

Data Structures

- struct [Flexio_Spi_Ip_DeviceParamsType](#)
Structure defining some parameters often change of the spi bus. [More...](#)
- struct [Flexio_Spi_Ip_ExternalDeviceType](#)
Structure defining the parameters of the spi bus. [More...](#)
- struct [Flexio_Spi_Ip_ConfigType](#)
Structure defining information needed for SPI driver initialization. [More...](#)
- struct [Flexio_Spi_Ip_StateStructureType](#)
Structure defining information needed for internal state of the driver. [More...](#)

Macros

- `#define SPI\_STOP\_SEC\_CONFIG\_DATA\_UNSPECIFIED`
Export Post-Build configurations.

Types Reference

- typedef void(* [Flexio_Spi_Ip_CallbackType](#)) (uint8 Instance, [Flexio_Spi_Ip_EventType](#) Event)
Callback for all peripherals which supports SPI features.

Enum Reference

- enum [Flexio_Spi_Ip_EventType](#)
Enum defining the possible events which triggers end of transfer callback.
- enum [Flexio_Spi_Ip_ModeType](#)
Enum defining the possible transfer modes.
- enum [Flexio_Spi_Ip_HwStatusType](#)
Enum defining the possible states of SPI/DSPI hardware unit.
- enum [Flexio_Spi_Ip_StatusType](#)

Enum defining the possible return types.

Function Reference

- [Flexio_Spi_Ip_StatusType Flexio_Spi_Ip_Init](#) (const [Flexio_Spi_Ip_ConfigType](#) *PhyUnitConfigPtr)
FLEXIO_SPI peripheral initialization.
- [Flexio_Spi_Ip_StatusType Flexio_Spi_Ip_DeInit](#) (uint8 Instance)
FLEXIO_SPI peripheral deinitialization.
- [Flexio_Spi_Ip_StatusType Flexio_Spi_Ip_SyncTransmit](#) (const [Flexio_Spi_Ip_ExternalDeviceType](#) *ExternalDevice, const uint8 *TxBuffer, uint8 *RxBuffer, uint16 Length, uint32 TimeOut)
FLEXIO_SPI synchronous transmission.
- [Flexio_Spi_Ip_StatusType Flexio_Spi_Ip_AsyncTransmit](#) (const [Flexio_Spi_Ip_ExternalDeviceType](#) *ExternalDevice, const uint8 *TxBuffer, uint8 *RxBuffer, uint16 Length, [Flexio_Spi_Ip_CallbackType](#) EndCallback)
FLEXIO_SPI asynchronous transmission.
- [Flexio_Spi_Ip_HwStatusType Flexio_Spi_Ip_GetStatus](#) (uint8 Instance)
Get status of HW unit.
- void [Flexio_Spi_Ip_ManageBuffers](#) (uint8 Instance)
Process transfer in POLLING mode.
- [Flexio_Spi_Ip_StatusType Flexio_Spi_Ip_UpdateTransferParam](#) (const [Flexio_Spi_Ip_ExternalDeviceType](#) *ExternalDevice, const [Flexio_Spi_Ip_TransferAdjustmentType](#) *Param)
Adjust transfer based on external conditions.
- [Flexio_Spi_Ip_StatusType Flexio_Spi_Ip_UpdateFrameSize](#) (const [Flexio_Spi_Ip_ExternalDeviceType](#) *ExternalDevice, uint8 FrameSize)
FLEXIO_SPI change frame size.
- [Flexio_Spi_Ip_StatusType Flexio_Spi_Ip_UpdateLsb](#) (const [Flexio_Spi_Ip_ExternalDeviceType](#) *ExternalDevice, boolean Lsb)
FLEXIO_SPI change bit order.
- [Flexio_Spi_Ip_StatusType Flexio_Spi_Ip_UpdateDefaultTransmitData](#) (const [Flexio_Spi_Ip_ExternalDeviceType](#) *ExternalDevice, uint32 DefaultData)
FLEXIO_SPI change default transmit data.
- [Flexio_Spi_Ip_StatusType Flexio_Spi_Ip_UpdateTransferMode](#) (uint8 Instance, [Flexio_Spi_Ip_ModeType](#) Mode)
FLEXIO_SPI change transfer mode.
- void [Flexio_Spi_Ip_Cancel](#) (uint8 Instance)
FLEXIO_SPI cancel current transmission.

6.1.2 Data Structure Documentation

6.1.2.1 struct Flexio_Spi_Ip_DeviceParamsType

Structure defining some parameters often change of the spi bus.

Definition at line 151 of file Flexio_Spi_Ip_Types.h.

Data Fields

uint8	FrameSize	Frame size configured
-------	-----------	-----------------------

Data Fields

boolean	Lsb	Transfer LSB first or MSB first
uint32	DefaultData	Default data to send when TxBuffer is NULL_PTR

6.1.2.2 struct Flexio_Spi_Ip_ExternalDeviceType

Structure defining the parameters of the spi bus.

Definition at line 159 of file Flexio_Spi_Ip_Types.h.

Data Fields

uint8	Instance	Instance of the hardware unit.
uint8	Cpol	
uint8	Cpha	
uint32	ClkTimeCmpBaudRate	
uint32	ClkTimeCfgTimDec	
uint32	TxShiftCtl	SHIFTCTL register of TX
uint32	TxShiftCfg	SHIFTCFG register of TX
uint32	RxShiftCtl	SHIFTCTL register of RX
uint32	RxShiftCfg	SHIFTCFG register of RX
uint32	ClkTimeCmp	TIMCMP register of CLK
uint32	ClkTimeCfg	TIMCFG register of CLK
uint32	ClkTimeCtl	TIMCTL register of CLK
uint32	CsTimeCmp	TIMCMP register of CS
uint32	CsTimeCfg	TIMCFG register of CS
uint32	CsTimeCtl	TIMCTL register of CS
Flexio_Spi_Ip_DeviceParamsType *	DeviceParams	Contain configuration for bit order, frame size, default transmit data.

6.1.2.3 struct Flexio_Spi_Ip_ConfigType

Structure defining information needed for SPI driver initialization.

Definition at line 185 of file Flexio_Spi_Ip_Types.h.

Data Fields

uint8	FlexioInstance	Flexio Instance
uint8	Instance	Instance of the hardware unit.
boolean	DmaUsed	DMA is used or not
uint8	TxDmaChannel	Id of TX DMA channel for transmission
uint8	RxDmaChannel	Id of RX DMA channel for receive
Flexio_Spi_Ip_ModeType	TransferMode	Transfer mode for HWunit
uint32	FrameSize	Frame size configured
boolean	Lsb	Transfer LSB first or MSB first
uint32	DefaultData	Default data to send when TxBuffer is NULL_PTR

Data Fields

uint8	TxShifterIndex	No. of shifter for TX
uint8	RxShifterIndex	No. of shifter for RX
uint8	ClkTimerIndex	No. of timer for CLK
uint8	CsTimerIndex	No. of timer for CS
uint8	StateIndex	State of current transfer

6.1.2.4 struct Flexio_Spi_Ip_StateStructureType

Structure defining information needed for internal state of the driver.

Definition at line 209 of file Flexio_Spi_Ip_Types.h.

Data Fields

Flexio_Spi_Ip_ModeType	TransferMode	Store current transfer mode for HWunit
boolean	FirstChannel	This is the first channel in a job
Flexio_Spi_Ip_HwStatusType	Status	0 = available, 1 = busy, 2 = fail due to overflow or underflow
uint8 *	RxBuffer	Store pointer for Rx buffer
const uint8 *	TxBuffer	Store pointer for Tx buffer
Flexio_Spi_Ip_CallbackType	Callback	Store pointer for call back function
uint16	RxIndex	Store current Rx index to receive data in Rx buffer
uint16	TxIndex	Store current Tx index to transmit data in Tx buffer
uint16	ExpectedFifoReads	Store number of frames needs to be receive for current transfer
uint16	ExpectedFifoWrites	Store number of frames needs to be transmit for current transfer
const Flexio_Spi_Ip_ConfigType *	PhyUnitConfig	
const Flexio_Spi_Ip_ExternalDeviceType *	ExternalDevice	Store externalDevice
uint8	FlexioInstance	Flexio Instance

6.1.3 Macro Definition Documentation**6.1.3.1 SPI_STOP_SEC_CONFIG_DATA_UNSPECIFIED**

```
#define SPI_STOP_SEC_CONFIG_DATA_UNSPECIFIED
```

Export Post-Build configurations.

Definition at line 116 of file Flexio_Spi_Ip.h.

6.1.4 Types Reference

6.1.4.1 Flexio_Spi_Ip_CallbackType

```
typedef void(* Flexio_Spi_Ip_CallbackType) (uint8 Instance, Flexio_Spi_Ip_EventType Event)
```

Callback for all peripherals which supports SPI features.

Definition at line 106 of file Flexio_Spi_Ip_Types.h.

6.1.5 Enum Reference

6.1.5.1 Flexio_Spi_Ip_EventType

```
enum Flexio_Spi_Ip_EventType
```

Enum defining the possible events which triggers end of transfer callback.

Enumerator

FLEXIO_SPI_IP_EVENT_END_TRANSFER	The transfer is successfully done.
FLEXIO_SPI_IP_EVENT_FAULT	The transfer failed due to overflow/underflow.

Definition at line 99 of file Flexio_Spi_Ip_Types.h.

6.1.5.2 Flexio_Spi_Ip_ModeType

```
enum Flexio_Spi_Ip_ModeType
```

Enum defining the possible transfer modes.

Enumerator

FLEXIO_SPI_IP_POLLING	For polling mode the application must call periodically Spi_Ip_ManageBuffers after asynchronous transfers.
FLEXIO_SPI_IP_INTERRUPT	For interrupt mode the application doesn't need to perform any additional operations after asynchronous transfers.

Definition at line 111 of file Flexio_Spi_Ip_Types.h.

6.1.5.3 Flexio_Spi_Ip_HwStatusType

```
enum Flexio_Spi_Ip_HwStatusType
```

Enum defining the possible states of SPI/DSPI hardware unit.

Enumerator

FLEXIO_SPI_IP_UNINIT	Module is not initialized.
FLEXIO_SPI_IP_IDLE	Module is not used.
FLEXIO_SPI_IP_BUSY	A transfer is in progress.
FLEXIO_SPI_IP_FAULT	During last transfer a fault occurred.

Definition at line 120 of file Flexio_Spi_Ip_Types.h.

6.1.5.4 Flexio_Spi_Ip_StatusType

enum `Flexio_Spi_Ip_StatusType`

Enum defining the possible return types.

Enumerator

<code>FLEXIO_SPI_IP_STATUS_SUCCESS</code>	Successful operation.
<code>FLEXIO_SPI_IP_STATUS_FAIL</code>	Failed operation.
<code>FLEXIO_SPI_IP_FIFO_ERROR</code>	Overflow or underflow error.
<code>FLEXIO_SPI_IP_TIMEOUT</code>	Timeout error.

Definition at line 131 of file Flexio_Spi_Ip_Types.h.

6.1.6 Function Reference

6.1.6.1 Flexio_Spi_Ip_Init()

```
Flexio_Spi_Ip_StatusType Flexio_Spi_Ip_Init (
    const Flexio_Spi_Ip_ConfigType * PhyUnitConfigPtr )
```

FLEXIO_SPI peripheral initialization.

The function initialize the SPI Unit specified in the configuration.

Parameters

in	<i>PhyUnitConfigPtr</i>	- pointer to the specified SPI Unit configuration.
----	-------------------------	--

Returns

`FLEXIO_SPI_IP_STATUS_SUCCESS`: Initialization command has been accepted. `FLEXIO_SPI_IP_STATUS_FAIL`: Initialization command has not been accepted.

6.1.6.2 Flexio_Spi_Ip_DeInit()

```
Flexio_Spi_Ip_StatusType Flexio_Spi_Ip_DeInit (
    uint8 Instance )
```

FLEXIO_SPI peripheral deinitialization.

The function de-initialize the SPI peripheral instance specified. All registers of SPI peripheral will be reset.

Parameters

in	<i>Instance</i>	- SPI peripheral instance number.
----	-----------------	-----------------------------------

Returns

FLEXIO_SPI_IP_STATUS_SUCCESS: De-initialization command has been accepted. FLEXIO_SPI_IP_↵
_STATUS_FAIL: De-initialization command has not been accepted.

6.1.6.3 Flexio_Spi_Ip_SyncTransmit()

```
Flexio_Spi_Ip_StatusType Flexio_Spi_Ip_SyncTransmit (
    const Flexio_Spi_Ip_ExternalDeviceType * ExternalDevice,
    const uint8 * TxBuffer,
    uint8 * RxBuffer,
    uint16 Length,
    uint32 TimeOut )
```

FLEXIO_SPI synchronous transmission.

This function initializes a synchronous transfer using the bus parameters provided by external device.

Parameters

in	<i>ExternalDevice</i>	- pointer to the external device where data is transmitted.
in	<i>TxBuffer</i>	- pointer to transmit buffer.
in, out	<i>RxBuffer</i>	- pointer to receive buffer.
in	<i>Length</i>	- number of bytes to be sent.
in	<i>TimeOut</i>	- duration for sending one frame.

Returns

FLEXIO_SPI_IP_STATUS_SUCCESS: Transmission command has been accepted. FLEXIO_SPI_IP_↵
FIFO_ERROR: Overflow or underflow error occurred. FLEXIO_SPI_IP_STATUS_FAIL: Transmission
command has not been accepted. FLEXIO_SPI_IP_TIMEOUT: Timeout error occurred.

6.1.6.4 Flexio_Spi_Ip_AsyncTransmit()

```
Flexio_Spi_Ip_StatusType Flexio_Spi_Ip_AsyncTransmit (
    const Flexio_Spi_Ip_ExternalDeviceType * ExternalDevice,
    const uint8 * TxBuffer,
    uint8 * RxBuffer,
    uint16 Length,
    Flexio_Spi_Ip_CallbackType EndCallback )
```

FLEXIO_SPI asynchronous transmission.

This function initializes an asynchronous transfer using the bus parameters provided by external device. After Flexio_Spi_Ip_Init function is called, FLEXIO_SPI_IP_POLLING mode is set as default to change the default mode Flexio_Spi_Ip_UpdateTransferMode should be called.

Parameters

in	<i>ExternalDevice</i>	- pointer to the external device where data is transmitted
in	<i>TxBuffer</i>	- pointer to transmit buffer.
in, out	<i>RxBuffer</i>	- pointer to receive buffer.
in	<i>Length</i>	- number of bytes to be sent.

Parameters

in	<i>EndCallback</i>	- callback function is called at the end of transfer.
----	--------------------	---

Returns

FLEXIO_SPI_IP_STATUS_SUCCESS: Transmission command has been accepted. FLEXIO_SPI_IP_STATUS_FAIL: Transmission command has not been accepted.

6.1.6.5 Flexio_Spi_Ip_GetStatus()

```
Flexio_Spi_Ip_HwStatusType Flexio_Spi_Ip_GetStatus (
    uint8 Instance )
```

Get status of HW unit.
This function will return status of HW unit assigned.

Parameters

in	<i>Instance</i>	Instance of the hardware unit.
----	-----------------	--------------------------------

Returns

Flexio_Spi_Ip_HwStatusType

Return values

<i>FLEXIO_SPI_IP_IDLE</i>	Hardware unit is not used
<i>FLEXIO_SPI_IP_BUSY</i>	A transfer is in progress
<i>FLEXIO_SPI_IP_FAULT</i>	During last transfer a fault occurred

6.1.6.6 Flexio_Spi_Ip_ManageBuffers()

```
void Flexio_Spi_Ip_ManageBuffers (
    uint8 Instance )
```

Process transfer in POLLING mode.
This function shall polls the SPI interrupts linked to SPI peripheral instance allocated to the transmission of data to enable the evolution of transmission state machine.

Parameters

in	<i>Instance</i>	- SPI peripheral instance number.
----	-----------------	-----------------------------------

6.1.6.7 Flexio_Spi_Ip_UpdateTransferParam()

```
Flexio_Spi_Ip_StatusType Flexio_Spi_Ip_UpdateTransferParam (
    const Flexio_Spi_Ip_ExternalDeviceType * ExternalDevice,
```

```
const Flexio_Spi_Ip_TransferAdjustmentType * Param )
```

Adjust transfer based on external conditions.

Use parameters to improve transfer

Parameters

in	<i>ExternalDevice</i>	- External device where data is transmitted
in	<i>Param</i>	- Parameter with adjustments

Returns

FLEXIO_SPI_IP_STATUS_SUCCESS: Setting command has been accepted. FLEXIO_SPI_IP_STAT←
US_FAIL: Setting command has not been accepted.

6.1.6.8 Flexio_Spi_Ip_UpdateFrameSize()

```
Flexio_Spi_Ip_StatusType Flexio_Spi_Ip_UpdateFrameSize (
    const Flexio_Spi_Ip_ExternalDeviceType * ExternalDevice,
    uint8 FrameSize )
```

FLEXIO_SPI change frame size.

This function updates frame size of specific external device configuration for next transfers.

Parameters

in	<i>ExternalDevice</i>	- pointer to the external device configuration.
in	<i>FrameSize</i>	- Frame size.

Returns

FLEXIO_SPI_IP_STATUS_SUCCESS: Setting command has been accepted. FLEXIO_SPI_IP_STAT←
US_FAIL: Setting command has not been accepted.

6.1.6.9 Flexio_Spi_Ip_UpdateLsb()

```
Flexio_Spi_Ip_StatusType Flexio_Spi_Ip_UpdateLsb (
    const Flexio_Spi_Ip_ExternalDeviceType * ExternalDevice,
    boolean Lsb )
```

FLEXIO_SPI change bit order.

This function updates bits order LSB or MSB of specific external device configuration for next transfer.

Parameters

in	<i>ExternalDevice</i>	- pointer to the external device configuration.
in	<i>Lsb</i>	- Data is transferred LSB first or not.

Returns

FLEXIO_SPI_IP_STATUS_SUCCESS: Setting command has been accepted. FLEXIO_SPI_IP_STAT←
US_FAIL: Setting command has not been accepted.

6.1.6.10 Flexio_Spi_Ip_UpdateDefaultTransmitData()

```
Flexio_Spi_Ip_StatusType Flexio_Spi_Ip_UpdateDefaultTransmitData (
    const Flexio_Spi_Ip_ExternalDeviceType * ExternalDevice,
    uint32 DefaultData )
```

FLEXIO_SPI change default transmit data.

This function updates default transmit data of specific external device configuration for next transfer.

Parameters

in	<i>ExternalDevice</i>	- pointer to the external device configuration.
in	<i>DefaultData</i>	- New default transmit data.

Returns

FLEXIO_SPI_IP_STATUS_SUCCESS: Setting command has been accepted. FLEXIO_SPI_IP_STAT←
US_FAIL: Setting command has not been accepted.

6.1.6.11 Flexio_Spi_Ip_UpdateTransferMode()

```
Flexio_Spi_Ip_StatusType Flexio_Spi_Ip_UpdateTransferMode (
    uint8 Instance,
    Flexio_Spi_Ip_ModeType Mode )
```

FLEXIO_SPI change transfer mode.

This function updates the asynchronous mechanism mode for the specified SPI Hardware microcontroller peripheral.

Parameters

in	<i>Instance</i>	- SPI peripheral instance number.
in	<i>Mode</i>	- new mode (interrupt or polling).

Returns

FLEXIO_SPI_IP_STATUS_SUCCESS: Setting command has been accepted. FLEXIO_SPI_IP_STAT←
US_FAIL: Setting command has not been accepted.

6.1.6.12 Flexio_Spi_Ip_Cancel()

```
void Flexio_Spi_Ip_Cancel (
    uint8 Instance )
```

FLEXIO_SPI cancel current transmission.

This function will cancel current asynchronous transmission.

Module Documentation

Parameters

in	<i>Instance</i>	Instance of the hardware unit.
----	-----------------	--------------------------------

6.2 Lpspi Driver

6.2.1 Detailed Description

Data Structures

- struct [Lpspi_Ip_DeviceParamsType](#)
Structure defining some parameters often change of the spi bus. [More...](#)
- struct [Lpspi_Ip_ExternalDeviceType](#)
Structure defining the parameters of the spi bus. [More...](#)
- struct [Lpspi_Ip_CmdDmaFastType](#)
Structure defining transmittion command needed for Dma Fast transfer. [More...](#)
- struct [Lpspi_Ip_FastTransferType](#)
Structure defining information needed for Dma Fast transfer session. [More...](#)
- struct [Lpspi_Ip_ConfigType](#)
Structure defining information needed for SPI driver initialization. [More...](#)
- struct [Lpspi_Ip_StateStructureType](#)
Structure defining information needed for internal state of the driver. [More...](#)

Types Reference

- typedef void(* [Lpspi_Ip_CallbackType](#)) (uint8 Instance, [Lpspi_Ip_EventType](#) Event)
Callback for all peripherals which supports SPI features.

Enum Reference

- enum [Lpspi_Ip_EventType](#)
Enum defining the possible events which triggers end of transfer callback.
- enum [Lpspi_Ip_ModeType](#)
Enum defining the possible transfer modes.
- enum [Lpspi_Ip_HwStatusType](#)
Enum defining the possible states of SPI/DSPI hardware unit.
- enum [Lpspi_Ip_StatusType](#)
Enum defining the possible return types.
- enum [Lpspi_Ip_HalfDuplexType](#)
Enum defining the half duplex types.

Function Reference

- [Lpspi_Ip_StatusType](#) [Lpspi_Ip_Init](#) (const [Lpspi_Ip_ConfigType](#) *PhyUnitConfigPtr)
LP SPI peripheral initialization.
- [Lpspi_Ip_StatusType](#) [Lpspi_Ip_DeInit](#) (uint8 Instance)
LP SPI peripheral deinitialization.
- [Lpspi_Ip_StatusType](#) [Lpspi_Ip_SyncTransmit](#) (const [Lpspi_Ip_ExternalDeviceType](#) *ExternalDevice, const uint8 *TxBuffer, uint8 *RxBuffer, uint16 Length, uint32 TimeOut)
LP SPI synchronous transmission.

- [Lpspi_Ip_StatusType Lpspi_Ip_AsyncTransmit](#) (const [Lpspi_Ip_ExternalDeviceType](#) *ExternalDevice, const uint8 *TxBuffer, uint8 *RxBuffer, uint16 Length, [Lpspi_Ip_CallbackType](#) EndCallback)
LPSPi asynchronous transmission.
- [Lpspi_Ip_StatusType Lpspi_Ip_SyncTransmitHalfDuplex](#) (const [Lpspi_Ip_ExternalDeviceType](#) *ExternalDevice, uint8 *Buffer, uint16 Length, [Lpspi_Ip_HalfDuplexType](#) TransferType, uint32 TimeOut)
LPSPi synchronous transmission support half duplex mode.
- [Lpspi_Ip_StatusType Lpspi_Ip_AsyncTransmitHalfDuplex](#) (const [Lpspi_Ip_ExternalDeviceType](#) *ExternalDevice, uint8 *Buffer, uint16 Length, [Lpspi_Ip_HalfDuplexType](#) TransferType, [Lpspi_Ip_CallbackType](#) EndCallback)
LPSPi asynchronous transmission support half duplex mode.
- [Lpspi_Ip_StatusType Lpspi_Ip_AsyncTransmitFast](#) (const [Lpspi_Ip_FastTransferType](#) *FastTransferCfg, uint8 NumberOfTransfer, [Lpspi_Ip_CallbackType](#) EndCallback)
LPSPi asynchronous transmission fast.
- [Lpspi_Ip_HwStatusType Lpspi_Ip_GetStatus](#) (uint8 Instance)
Get status of HW unit.
- void [Lpspi_Ip_ManageBuffers](#) (uint8 Instance)
Process transfer in POLLING mode.
- [Lpspi_Ip_StatusType Lpspi_Ip_UpdateTransferParam](#) (const [Lpspi_Ip_ExternalDeviceType](#) *ExternalDevice, const [Lpspi_Ip_TransferAdjustmentType](#) *Param)
Adjust transfer based on external conditions.
- [Lpspi_Ip_StatusType Lpspi_Ip_UpdateFrameSize](#) (const [Lpspi_Ip_ExternalDeviceType](#) *ExternalDevice, uint16 FrameSize)
LPSPi change frame size.
- [Lpspi_Ip_StatusType Lpspi_Ip_UpdateLsb](#) (const [Lpspi_Ip_ExternalDeviceType](#) *ExternalDevice, boolean Lsb)
LPSPi change bit order.
- [Lpspi_Ip_StatusType Lpspi_Ip_UpdateByteSwap](#) (const [Lpspi_Ip_ExternalDeviceType](#) *ExternalDevice, boolean ByteSwap)
LPSPi change byte swap.
- [Lpspi_Ip_StatusType Lpspi_Ip_UpdateDefaultTransmitData](#) (const [Lpspi_Ip_ExternalDeviceType](#) *ExternalDevice, uint32 DefaultData)
LPSPi change default transmit data.
- [Lpspi_Ip_StatusType Lpspi_Ip_UpdateTransferMode](#) (uint8 Instance, [Lpspi_Ip_ModeType](#) Mode)
LPSPi change transfer mode.
- void [Lpspi_Ip_Cancel](#) (uint8 Instance)
LPSPi cancel current asynchronous transmission.

Variables

- const [Lpspi_Ip_ExternalDeviceType Lpspi_Ip_DeviceAttributes_SpiExternalDevice_LPSPi0_Instance_0_VS_0](#)
Export Post-Build configurations.

6.2.2 Data Structure Documentation

6.2.2.1 struct Lpspi_Ip_DeviceParamsType

Structure defining some parameters often change of the spi bus.

Definition at line 162 of file Lpspi_Ip_Types.h.

Data Fields

uint16	FrameSize	Frame size configured
boolean	Lsb	Transfer LSB first or MSB first
boolean	ByteSwap	Enables byte swap on each 32-bit word when transmitting and receiving data
uint32	DefaultData	Default data to send when TxBuffer is NULL_PTR
Lpspi_Ip_HalfDuplexType	TransferType	TransferType

6.2.2.2 struct Lpspi_Ip_ExternalDeviceType

Structure defining the parameters of the spi bus.

Definition at line 174 of file Lpspi_Ip_Types.h.

Data Fields

uint8	Instance	Instance of the hardware unit.
uint32	Cfgr0	CFGR0 register contains HREQ configuration.
uint32	Ccr	CCR register which contains clocking and frame size configuration.
uint32	Tcr	TCR register which contains clock polarities, frame size, which PCS and continuous mode.
uint32	HalfDuplexCfgr1	CFGR1 register which contains bit fields to support half duplex mode .
Lpspi_Ip_DeviceParamsType *	DeviceParams	Contain configuration for bit order, frame size, default transmit data.

6.2.2.3 struct Lpspi_Ip_CmdDmaFastType

Structure defining transmission command needed for Dma Fast transfer.

Definition at line 205 of file Lpspi_Ip_Types.h.

Data Fields

uint32	DmaFastTcrCmd	Contains transfer command for Dma Fast transfer.
uint32	DmaFastTcrCmdLast	Contains transfer command and disable continuous mode for Dma Fast transfer.
uint32	DefaultData	Default data to send when TxBuffer is NULL_PTR

6.2.2.4 struct Lpspi_Ip_FastTransferType

Structure defining information needed for Dma Fast transfer session.

Module Documentation

Definition at line 215 of file Lpspi_Ip_Types.h.

Data Fields

- const [Lpspi_Ip_ExternalDeviceType](#) * [ExternalDevice](#)
- const uint8 * [TxBuffer](#)
- uint8 * [RxBuffer](#)
- uint32 [DefaultData](#)
- uint16 [Length](#)
- boolean [KeepCs](#)

Field Documentation

ExternalDevice

```
const Lpspi\_Ip\_ExternalDeviceType* ExternalDevice
```

Point to external device configuration

Definition at line 217 of file Lpspi_Ip_Types.h.

TxBuffer

```
const uint8* TxBuffer
```

Store pointer for Tx buffer

Definition at line 218 of file Lpspi_Ip_Types.h.

RxBuffer

```
uint8* RxBuffer
```

Store pointer for Rx buffer

Definition at line 219 of file Lpspi_Ip_Types.h.

DefaultData

```
uint32 DefaultData
```

Default data to send when TxBuffer is NULL_PTR

Definition at line 220 of file Lpspi_Ip_Types.h.

Length

uint16 Length

Number of bytes to be sent

Definition at line 221 of file Lpspi_Ip_Types.h.

KeepCs

boolean KeepCs

Keep CS signal after transfer session completed

Definition at line 222 of file Lpspi_Ip_Types.h.

6.2.2.5 struct Lpspi_Ip_ConfigType

Structure defining information needed for SPI driver initialization.

Definition at line 228 of file Lpspi_Ip_Types.h.

Data Fields

uint8	Instance	Instance of the hardware unit.
uint32	Cr	It contains only debug enable.
uint32	Cfgr1	It contains PCS polarities.
boolean	SlaveMode	
boolean	DmaUsed	DMA is used or not
uint8	TxDmaChannel	Id of TX DMA channel for transmission
uint8	RxDmaChannel	Id of RX DMA channel for receive
uint8	MaxNumOfFastTransfer	Maximum number of transfers in Dma Fast
Lpspi_Ip_CmdDmaFastType *	CmdDmaFast	Point to list of TCR command used in Dma Fast transfer
uint8	NumberTxSG	Number of TCD Scatter Gather for Tx DMA channel used in Dma Fast transfer
uint8	NumberRxSG	Number of TCD Scatter Gather for Rx DMA channel used in Dma Fast transfer
const uint8 *	TxDmaFastSGId	Point to list of TCD Scatter Gather Id for Tx DMA channel used in Dma Fast transfer
const uint8 *	RxDmaFastSGId	Point to list of TCD Scatter Gather Id for Rx DMA channel used in Dma Fast transfer
Lpspi_Ip_ModeType	TransferMode	Transfer mode for HWunit
uint8	StateIndex	State of current transfer

6.2.2.6 struct Lpspi_Ip_StateStructureType

Structure defining information needed for internal state of the driver.

Definition at line 254 of file Lpspi_Ip_Types.h.

Data Fields

Lpspi_Ip_ModeType	TransferMode	Store current transfer mode for HWunit
Lpspi_Ip_HwStatusType	Status	0 = available, 1 = busy, 2 = fail due to overflow or underflow
uint8 *	RxBuffer	Store pointer for Rx buffer
const uint8 *	TxBuffer	Store pointer for Tx buffer
Lpspi_Ip_CallbackType	Callback	Store pointer for call back function
uint16	RxIndex	Store current Rx index to receive data in Rx buffer
uint16	TxIndex	Store current Tx index to transmit data in Tx buffer
uint16	ExpectedFifoReads	Store number of words needed to be receive for current transfer
uint16	ExpectedFifoWrites	Store number of words needed to be transmitted for current transfer
boolean	KeepCs	Keep CS signal after transfers completed.
boolean	FirstCmd	
const Lpspi_Ip_ConfigType *	PhyUnitConfig	
const Lpspi_Ip_ExternalDeviceType *	ExternalDevice	
uint16	TxFramSize	Store current frame size for HWunit
boolean	TxLsb	Transfer LSB first or MSB first
const uint8 *	TxBufferNext	Store pointer for Tx buffer
uint16	FrameSizeNext	Store current frame size for HWunit
boolean	LsbNext	Transfer LSB first or MSB first
uint32	DefaultDataNext	Default data to send when TxBuffer is NULL_PTR
uint16	LengthNext	Store number of frames needs to be transmit for current transfer
boolean	ByteSwapNext	Store byteswap for current transfer
boolean	NextTransferConfigAvailable	Flag to check next transfer configuration is available
boolean	NextTransferDone	Flag to check next transfer done
uint8	CurrentTxFifoSlot	Number of TX FIFO slots are current available.
uint32	HalfDuplexTcrCommand	Save the value which will be written to TCR register in DMA mode
boolean	NextChannellsRX	Save the value which will be written to TCR register in DMA mode
boolean	TxDoneFlag	Flag to check TX is done

6.2.3 Types Reference

6.2.3.1 Lpspi_Ip_CallbackType

```
typedef void(* Lpspi_Ip_CallbackType) (uint8 Instance, Lpspi_Ip_EventType Event)
```

Callback for all peripherals which supports SPI features.

Definition at line 116 of file Lpspi_Ip_Types.h.

6.2.4 Enum Reference

6.2.4.1 Lpspi_Ip_EventType

```
enum Lpspi_Ip_EventType
```

Enum defining the possible events which triggers end of transfer callback.

Enumerator

LPSPI_IP_EVENT_END_TRANSFER	The transfer is successfully done.
LPSPI_IP_EVENT_FAULT	The transfer failed due to overflow/underflow.

Definition at line 109 of file Lpspi_Ip_Types.h.

6.2.4.2 Lpspi_Ip_ModeType

```
enum Lpspi_Ip_ModeType
```

Enum defining the possible transfer modes.

Enumerator

LPSPI_IP_POLLING	For polling mode the application must call periodically Spi_Ip_ManageBuffers after asynchronous transfers.
LPSPI_IP_INTERRUPT	For interrupt mode the application doesn't need to perform any additional operations after asynchronous transfers. The application must enable the interrupt requests and install the right callbacks.

Definition at line 121 of file Lpspi_Ip_Types.h.

6.2.4.3 Lpspi_Ip_HwStatusType

```
enum Lpspi_Ip_HwStatusType
```

Enum defining the possible states of SPI/DSPI hardware unit.

Enumerator

LPSPI_IP_UNINIT	Module is not initialized.
LPSPI_IP_IDLE	Module is not used.
LPSPI_IP_BUSY	A transfer is in progress.
LPSPI_IP_FAULT	During last transfer a fault occurred.

Definition at line 131 of file Lpspi_Ip_Types.h.

6.2.4.4 Lpspi_Ip_StatusType

enum `Lpspi_Ip_StatusType`

Enum defining the possible return types.

Enumerator

LPSPi_IP_STATUS_SUCCESS	Successful operation.
LPSPi_IP_STATUS_FAIL	Failed operation.
LPSPi_IP_FIFO_ERROR	Overflow or underflow error.
LPSPi_IP_TIMEOUT	Timeout error.

Definition at line 142 of file Lpspi_Ip_Types.h.

6.2.4.5 Lpspi_Ip_HalfDuplexType

enum `Lpspi_Ip_HalfDuplexType`

Enum defining the half duplex types.

Enumerator

LPSPi_IP_HALF_DUPLEX_TRANSMIT	Transmit only.
LPSPi_IP_HALF_DUPLEX_RECEIVE	Receive only.
LPSPi_IP_FULL_DUPLEX	Full duplex mode.

Definition at line 153 of file Lpspi_Ip_Types.h.

6.2.5 Function Reference

6.2.5.1 Lpspi_Ip_Init()

```
Lpspi_Ip_StatusType Lpspi_Ip_Init (
    const Lpspi_Ip_ConfigType * PhyUnitConfigPtr )
```

LPSPi peripheral initialization.

The function initialize the SPI Unit specified in the configuration.

Parameters

in	<i>PhyUnitConfigPtr</i>	- pointer to the specified SPI Unit configuration.
----	-------------------------	--

Returns

LPSPi_IP_STATUS_SUCCESS: Initialization command has been accepted. LPSPi_IP_STATUS_FAIL: Initialization command has not been accepted.

6.2.5.2 Lpspi_Ip_DeInit()

```
Lpspi_Ip_StatusType Lpspi_Ip_DeInit (
    uint8 Instance )
```

LPSPi peripheral deinitialization.

The function de-initialize the SPI peripheral instance specified. All registers of SPI peripheral will be reset.

Parameters

in	<i>Instance</i>	- SPI peripheral instance number.
----	-----------------	-----------------------------------

Returns

LPSPi_IP_STATUS_SUCCESS: De-initialization command has been accepted. LPSPi_IP_STATUS_FAIL: De-initialization command has not been accepted.

6.2.5.3 Lpspi_Ip_SyncTransmit()

```
Lpspi_Ip_StatusType Lpspi_Ip_SyncTransmit (
    const Lpspi_Ip_ExternalDeviceType * ExternalDevice,
    const uint8 * TxBuffer,
    uint8 * RxBuffer,
    uint16 Length,
    uint32 TimeOut )
```

LPSPi synchronous transmission.

This function initializes a synchronous transfer using the bus parameters provided by external device. Depending on the FrameSize TxBuffer and RxBuffer are handled in the following way:

- 8: Array is interpreted as uint8
- 16: Array is interpreted as uint16
- 32: Array is interpreted as uint32
- >32: Array is interpreted as uint32 and if Length is not multiple of 4 the last bytes of the frame, until a multiple of 4, are ignored.

Parameters

in	<i>ExternalDevice</i>	- pointer to the external device where data is transmitted.
in	<i>TxBuffer</i>	- pointer to transmit buffer.
	<i>[in-out]</i>	RxBuffer - pointer to receive buffer.
in	<i>Length</i>	- number of bytes to be sent.
in	<i>TimeOut</i>	- duration for sending one frame.

Returns

LPSPi_IP_STATUS_SUCCESS: Transmission command has been accepted. LPSPi_IP_FIFO_ERROR← : Overflow or underflow error occurred. LPSPi_IP_STATUS_FAIL: Transmission command has not been accepted. LPSPi_IP_TIMEOUT: Timeout error occurred.

6.2.5.4 Lpspi_Ip_AsyncTransmit()

```
Lpspi_Ip_StatusType Lpspi_Ip_AsyncTransmit (
    const Lpspi_Ip_ExternalDeviceType * ExternalDevice,
    const uint8 * TxBuffer,
    uint8 * RxBuffer,
    uint16 Length,
    Lpspi_Ip_CallbackType EndCallback )
```

LPSPi asynchronous transmission.

This function initializes an asynchronous transfer using the bus parameters provided by external device. After Lpspi_Ip_Init function is called, LPSPi_IP_POLLING mode is set as default to change the default mode Lpspi_Ip_UpdateTransferMode should be called.

Parameters

in	<i>ExternalDevice</i>	- pointer to the external device where data is transmitted
in	<i>TxBuffer</i>	- pointer to transmit buffer.
	<i>[in-out]</i>	RxBuffer - pointer to receive buffer.
in	<i>Length</i>	- number of bytes to be sent.
in	<i>EndCallback</i>	- callback function is called at the end of transfer.

Returns

LPSPi_IP_STATUS_SUCCESS: Transmission command has been accepted. LPSPi_IP_STATUS_FAIL: Transmission command has not been accepted.

6.2.5.5 Lpspi_Ip_SyncTransmitHalfDuplex()

```
Lpspi_Ip_StatusType Lpspi_Ip_SyncTransmitHalfDuplex (
    const Lpspi_Ip_ExternalDeviceType * ExternalDevice,
    uint8 * Buffer,
    uint16 Length,
    Lpspi_Ip_HalfDuplexType TransferType,
    uint32 TimeOut )
```

LPSPi synchronous transmission support half duplex mode.

This function initializes a synchronous transfer using the bus parameters provided by external device.

Parameters

in	<i>ExternalDevice</i>	- pointer to the external device where data is transmitted.
in	<i>Buffer</i>	- pointer to transmit buffer.
in	<i>Length</i>	- number of bytes to be sent.
in	<i>TimeOut</i>	- duration for sending one frame.

Returns

LPSPi_IP_STATUS_SUCCESS: Transmission command has been accepted. LPSPi_IP_FIFO_ERROR↵: Overflow or underflow error occurred. LPSPi_IP_STATUS_FAIL: Transmission command has not been accepted. LPSPi_IP_TIMEOUT: Timeout error occurred.

6.2.5.6 Lpspi_Ip_AsyncTransmitHalfDuplex()

```
Lpspi_Ip_StatusType Lpspi_Ip_AsyncTransmitHalfDuplex (
    const Lpspi_Ip_ExternalDeviceType * ExternalDevice,
    uint8 * Buffer,
    uint16 Length,
    Lpspi_Ip_HalfDuplexType TransferType,
    Lpspi_Ip_CallbackType EndCallback )
```

LPSPi asynchronous transmission support half duplex mode.

This function initializes an asynchronous transfer using the bus parameters provided by external device.

Parameters

in	<i>ExternalDevice</i>	- pointer to the external device where data is transmitted
in	<i>Buffer</i>	- pointer to transmit buffer.
in	<i>Length</i>	- number of bytes to be sent.
in	<i>EndCallback</i>	- callback function is called at the end of transfer.

Returns

LPSPi_IP_STATUS_SUCCESS: Transmission command has been accepted. LPSPi_IP_STATUS_FAIL: Transmission command has not been accepted.

6.2.5.7 Lpspi_Ip_AsyncTransmitFast()

```
Lpspi_Ip_StatusType Lpspi_Ip_AsyncTransmitFast (
    const Lpspi_Ip_FastTransferType * FastTransferCfg,
    uint8 NumberOfTransfer,
    Lpspi_Ip_CallbackType EndCallback )
```

LPSPi asynchronous transmission fast.

This function initializes an asynchronous transmission for multiple transfers session and CPU used only for processing at the end of sequence transfer. The list of transfers session is composed of an array of fast transfers settings. The settings array is defined by the user needs: it contains entries parameters to be configured for each transfer session as defined in [Lpspi_Ip_FastTransferType](#).

How to use this interface:

1. Use the "Lpspi_Ip_FastTransferType" to create a list(array) of transfer session. Each field in [Lpspi_Ip_FastTransferType](#) for each transfer session must be configured. Note: This feature requires:
 - a. The parameters SpiBaudrate, SpiHwUnit, SpiTimeClk2Cs, SpiTimeCs2Clk, SpiTimeCs2Cs, SpiData↵Width, SpiTransferStart in all External Devices used(pointed by ExternalDevice) must be the same in each transfer session.
 - b. In each transfer section, the number of data buffer(Length) is NOT higher than 32767 if SpiDataWidth < 9.
 - c. Only Master mode is supported(SpiPhyUnit/SpiPhyUnitMode = SPI_MASTER).
 - d. Make sure that SpiPhyUnit/SpiMaxDmaFastTransfer value must NOT lower than total of transfer sessions.
 - e. Make sure that number of ScatterGathers configuration in SpiPhyUnit/SpiPhyTxDmaChannel must NOT

lower than total of transfer sessions plus number of time request CS de-assert(KeepCs = FALSE) at the end of transfer session in the list configured. f. Make sure that number of ScatterGathers configuration in each SpiPhyUnit/SpiPhyRxDmaChannel must NOT lower than total of transfer sessions.

2. Call the "Lpspi_Ip_AsyncTransmitFast()" interface.

Example: The user shall create the desired configuration list for his specific application. For example use case:

- Requiring 2 transfers session, keep CS assert at the end of first transfer session.
- Transfer session 1:
 - Use SpiExternalDevice_0 with SpiCsIdentifier = PCS0, SpiCsContinuous = TRUE.
 - Send 5 bytes. Tx buffer is "uint8 u8TxBuffer1[5u]={0,1,2,3,4};". Rx buffer is "uint8 u8RxBuffer1[5u];".
 - Keep CS assert at the end of this transfer session.
- Transfer session 2:
 - Use SpiExternalDevice_0 with SpiCsIdentifier = PCS0, SpiCsContinuous = TRUE.
 - Send 10 bytes with default transmit data value is 5. Tx buffer is NULL_PTR. Rx buffer is "uint8 u8RxBuffer2[10u];".
 - This is last transfer session, so CS will not kipped by default at the end of last transfer session.
- Configuration example on configuration tool:
 - SpiGeneral/SpiEnableDmaFastTransferSupport = true.
 - SpiPhyUnit/SpiMaxDmaFastTransfer = 2(2 transfers session).
 - Number of ScatterGathers configuration for SpiPhyTxDmaChannel is 3(2 transfers session + 1 time CS de-assert at the end of last transfer session).
 - Number of ScatterGathers configuration for SpiPhyRxDmaChannel is 2(2 transfers session).
- Call "UserCallbackFunc" when Fast transfer completed.
- Coding example: void UserCallbackFunc(uint8 Instance, Lpspi_Ip_EventType event); [Lpspi_Ip_FastTransferType](#)
 aUserFastTransferCfgList[2u] = { { Lpspi_Ip_DeviceAttributes_SpiExternalDevice_0_BOARD_Init←
 Peripherals, ->Point to External Device 0 configuration generated by configuration tool u8TxBuffer1, ->
 Store pointer for Tx buffer u8RxBuffer1, -> Store pointer for Rx buffer 0u, -> Default transmit data, don't
 care due to Tx buffer is not NULL_PTR 5u, -> Number of bytes to be sent (boolean)TRUE -> Keep CS signal
 at the end of this transfer session }, { Lpspi_Ip_DeviceAttributes_SpiExternalDevice_0_BOARD_Init←
 Peripherals, -> Point to external device configuration NULL_PTR, -> Store pointer for Tx buffer u8RxBuffer2,
 -> Store pointer for Rx buffer 5u, -> Default transmit data, don't care due to Tx buffer is not NULL_PTR
 10u, -> Number of bytes to be sent (boolean)FALSE -> Not keep CS signal at the end of this transfer session,
 don't care this parameter for last transfer } }; Lpspi_Ip_AsyncTransmitFast(aUserFastTransferCfgList, 2u,
 &UserCallbackFunc);

Parameters

	<i>[in-out]</i>	
		FastTransferCfg - pointer to the list of transfers section configuration.
in	<i>NumberOfTransfer</i>	- number of transfers session in the list is pointed by FastTransferCfg.
in	<i>EndCallback</i>	- callback function is called at the end of sequence transfer.

Returns

LPSPi_IP_STATUS_SUCCESS: Transmission command has been accepted. LPSPi_IP_STATUS_FAIL: Transmission command has not been accepted.

6.2.5.8 Lpspi_Ip_GetStatus()

```
Lpspi_Ip_HwStatusType Lpspi_Ip_GetStatus (
    uint8 Instance )
```

Get status of HW unit.

This function returns the status of the specified SPI Hardware microcontroller peripheral.

Parameters

in	<i>Instance</i>	- SPI peripheral instance number.
----	-----------------	-----------------------------------

Returns

Lpspi_Ip_HwStatusType

6.2.5.9 Lpspi_Ip_ManageBuffers()

```
void Lpspi_Ip_ManageBuffers (
    uint8 Instance )
```

Process transfer in POLLING mode.

This function shall polls the SPI interrupts linked to SPI peripheral instance allocated to the transmission of data to enable the evolution of transmission state machine.

Parameters

in	<i>Instance</i>	- SPI peripheral instance number.
----	-----------------	-----------------------------------

Returns

void

6.2.5.10 Lpspi_Ip_UpdateTransferParam()

```
Lpspi_Ip_StatusType Lpspi_Ip_UpdateTransferParam (
    const Lpspi_Ip_ExternalDeviceType * ExternalDevice,
    const Lpspi_Ip_TransferAdjustmentType * Param )
```

Adjust transfer based on external conditions.

Use parameters to improve transfer

Parameters

in	<i>ExternalDevice</i>	- External device where data is transmitted
in	<i>Param</i>	- Parameter with adjustments

Returns

LPSPi_IP_STATUS_SUCCESS: Setting command has been accepted. LPSPi_IP_STATUS_FAIL: Setting command has not been accepted.

6.2.5.11 Lpspi_Ip_UpdateFrameSize()

```
Lpspi_Ip_StatusType Lpspi_Ip_UpdateFrameSize (
    const Lpspi_Ip_ExternalDeviceType * ExternalDevice,
    uint16 FrameSize )
```

LPSPi change frame size.

This function updates frame size of specific external device configuration for next transfers.

Parameters

in	<i>ExternalDevice</i>	- pointer to the external device configuration.
in	<i>FrameSize</i>	- Frame size. Supported values are: 8, 16, 32 and if > 32 it should be a pair number.

Returns

LPSPi_IP_STATUS_SUCCESS: Setting command has been accepted. LPSPi_IP_STATUS_FAIL: Setting command has not been accepted.

6.2.5.12 Lpspi_Ip_UpdateLsb()

```
Lpspi_Ip_StatusType Lpspi_Ip_UpdateLsb (
    const Lpspi_Ip_ExternalDeviceType * ExternalDevice,
    boolean Lsb )
```

LPSPi change bit order.

This function updates bits order LSB or MSB of specific external device configuration for next transfer.

Parameters

in	<i>ExternalDevice</i>	- pointer to the external device configuration.
in	<i>Lsb</i>	- Data is transferred LSB first or not.

Returns

LPSPi_IP_STATUS_SUCCESS: Setting command has been accepted. LPSPi_IP_STATUS_FAIL: Setting command has not been accepted.

6.2.5.13 Lpspi_Ip_UpdateByteSwap()

```
Lpspi_Ip_StatusType Lpspi_Ip_UpdateByteSwap (
    const Lpspi_Ip_ExternalDeviceType * ExternalDevice,
    boolean ByteSwap )
```

LPSPi change byte swap.

This function updates to enable byte swap on each 32-bit word when transmitting and receiving data.

Parameters

in	<i>ExternalDevice</i>	- pointer to the external device configuration.
in	<i>ByteSwap</i>	- Data is transferred with byte swap feature or not.

Returns

LPSPI_IP_STATUS_SUCCESS: Setting command has been accepted. LPSPI_IP_STATUS_FAIL: Setting command has not been accepted.

6.2.5.14 Lpspi_Ip_UpdateDefaultTransmitData()

```
Lpspi_Ip_StatusType Lpspi_Ip_UpdateDefaultTransmitData (
    const Lpspi_Ip_ExternalDeviceType * ExternalDevice,
    uint32 DefaultData )
```

LPSPI change default transmit data.

This function updates default transmit data of specific external device configuration for next transfer.

Parameters

in	<i>ExternalDevice</i>	- pointer to the external device configuration.
in	<i>DefaultData</i>	- New default transmit data.

Returns

LPSPI_IP_STATUS_SUCCESS: Setting command has been accepted. LPSPI_IP_STATUS_FAIL: Setting command has not been accepted.

6.2.5.15 Lpspi_Ip_UpdateTransferMode()

```
Lpspi_Ip_StatusType Lpspi_Ip_UpdateTransferMode (
    uint8 Instance,
    Lpspi_Ip_ModeType Mode )
```

LPSPI change transfer mode.

This function updates the asynchronous mechanism mode for the specified SPI Hardware microcontroller peripheral.

Parameters

in	<i>Instance</i>	- SPI peripheral instance number.
in	<i>Mode</i>	- new mode (interrupt or polling).

Returns

LPSPi_IP_STATUS_SUCCESS: Setting command has been accepted. LPSPi_IP_STATUS_FAIL: Setting command has not been accepted.

6.2.5.16 Lpspi_Ip_Cancel()

```
void Lpspi_Ip_Cancel (
    uint8 Instance )
```

LPSPi cancel current asynchronous transmission.

This function cancels an asynchronous transmission in progress for the specified SPI Hardware microcontroller peripheral.

Parameters

in	<i>Instance</i>	- SPI peripheral instance number.
----	-----------------	-----------------------------------

Returns

void

6.2.6 Variable Documentation

6.2.6.1 Lpspi_Ip_DeviceAttributes_SpiExternalDevice_LPSPi0_Instance_0_VS_0

```
const Lpspi_Ip_ExternalDeviceType Lpspi_Ip_DeviceAttributes_SpiExternalDevice_LPSPi0_Instance_0_VS_0 ←
0 [extern]
```

Export Post-Build configurations.

6.3 Spi Driver

6.3.1 Detailed Description

Data Structures

- struct [Spi_SequenceConfigType](#)
This structure contains all the needed data to configure one SPI Sequence. [More...](#)
- struct [Spi_SeqsConfigType](#)
This structure contains Sequence configuration. [More...](#)
- struct [Spi_JobConfigType](#)
This is the structure containing all the parameters needed to completely define a Job. [More...](#)
- struct [Spi_JobsCfgType](#)
This is the structure containing Job configuration. [More...](#)
- struct [Spi_BufferDescriptorType](#)
The structure contains the pointers to the Tx/Rx memory locations for the given buffer (IB or EB). [More...](#)
- struct [Spi_HWUnitQueue](#)
This structure holds the HWUnit scheduling queue. [More...](#)
- struct [Spi_ChannelConfigType](#)
The structure contains the channel configuration parameters. [More...](#)
- struct [Spi_ChannelsCfgType](#)
The structure contains the channel configuration. [More...](#)
- struct [Spi_ConfigType](#)
This is the top level structure containing all the needed parameters for the SPI Handler Driver. [More...](#)
- struct [Spi_Ipw_ParamSyncTransmit](#)
Parameters for Spi_Ipw_SyncTransmit. [More...](#)
- struct [Spi_Ipw_ParamAsyncTransmit](#)
Parameters for Spi_Ipw_AsyncTransfer. [More...](#)
- struct [Spi_Ipw_ParamDMAFast](#)
Parameters for Spi_Ipw_PrepareDMAFastTransferCfg. [More...](#)
- struct [Spi_Ipw_IpConfigType](#)
- struct [Spi_Ipw_ExternalDeviceType](#)
- struct [Spi_HWUnitConfigType](#)
This structure holds the HWUnit configuration parameters. [More...](#)
- struct [Spi_PhyUnitsConfigType](#)
This structure holds the PhyUnit configuration. [More...](#)
- struct [Spi_ExternalDeviceConfigType](#)
This structure holds the configuration parameters for each ExternalDevice. [More...](#)
- struct [Spi_ExDevicesConfigType](#)
This structure holds the ExternalDevice configuration. [More...](#)

Macros

- `#define SPI_E_PARAM_CHANNEL`
API service called with wrong parameter of Channel.
- `#define SPI_E_PARAM_JOB`
API service called with wrong parameter of Job.
- `#define SPI_E_PARAM_SEQ`
API service called with wrong parameter of Sequence.
- `#define SPI_E_PARAM_LENGTH`
API service called with wrong parameter of external buffer length.
- `#define SPI_E_PARAM_UNIT`
API service called with wrong parameter of HWUnit.
- `#define SPI_E_PARAM_CONFIG`
API service called with wrong resource assigned.
- `#define SPI_E_UNINIT`
API service used without module initialization.
- `#define SPI_E_SEQ_PENDING`
Services called in a wrong sequence.
- `#define SPI_E_SEQ_IN_PROCESS`
Synchronous transmission service called at wrong time.
- `#define SPI_E_ALREADY_INITIALIZED`
API SPI_Init service called while the SPI driver has already been initialized.
- `#define SPI_E_CONFIG_OUT_OF_RANGE`
The number of sequences, jobs or channels exceeds precompile time sizes.
- `#define SPI_E_INIT_FAILED`
API Spi_Init was called with wrong configuration pointer.
- `#define SPI_E_PARAM_EB_UNIT`
When a sequence contains uninitialized external buffers.
- `#define SPI_E_SEQ_EMPTY`
No job in sequence.
- `#define SPI_E_JOB_EMPTY`
No channel in job.
- `#define SPI_E_PARAM_POINTER`
If the parameter versioninfo or Spi configuration is NULL_PTR.
- `#define SPI_INIT_ID`
API service ID for SPI Init function.
- `#define SPI_DEINIT_ID`
API service ID for SPI DeInit function.
- `#define SPI_WRITEIB_ID`
API service ID for SPI write IB function.
- `#define SPI_ASYNCTRANSMIT_ID`

- API service ID for SPI async transmit function.*

 - #define [SPI_READIB_ID](#)
- API service ID for SPI read IB function.*

 - #define [SPI_SETUPEB_ID](#)
- API service ID for SPI setup EB function.*

 - #define [SPI_GETSTATUS_ID](#)
- API service ID for SPI get status function.*

 - #define [SPI_GETJOBRESULT_ID](#)
- API service ID for SPI get job result function.*

 - #define [SPI_GETSEQUENCERESULT_ID](#)
- API service ID for SPI get sequence result function.*

 - #define [SPI_GETVERSIONINFO_ID](#)
- API service ID for SPI get version info function.*

 - #define [SPI_SYNCTRANSMIT_ID](#)
- API service ID for SPI sync transmit function.*

 - #define [SPI_GETHWUNITSTATUS_ID](#)
- API service ID for SPI get hwunit status function.*

 - #define [SPI_CANCEL_ID](#)
- API service ID for SPI cancel function.*

 - #define [SPI_SETASYNCMODE_ID](#)
- API service ID for SPI set async mode function.*

 - #define [SPI_MAINFUNCTION_HANDLING_ID](#)
- API service ID for SPI main function.*

 - #define [SPI_SETHWUNITASYNCMODE_ID](#)
- API service ID for SPI set HW Unit async mode.*

 - #define [SPI_SETCLOCKMODE_ID](#)
- API service ID for SPI Set Clock Mode.*

 - #define [SPI_JOB_PRIORITY_LEVELS_COUNT](#)
- The number of allowed job priority levels (0..3).*

 - #define [NUMBER_OF_DATA_TYPE_MAX](#)
- Maximum value for number of data elements of the type Spi_DataBufferType.*

Types Reference

- typedef uint8 [Spi_DataBufferType](#)

Type of application data buffer elements.
- typedef uint16 [Spi_NumberOfDataType](#)

Type for defining the number of data elements of the type Spi_DataBufferType.
- typedef uint8 [Spi_ChannelType](#)

Specifies the identification (ID) for a Channel.
- typedef uint16 [Spi_JobType](#)

Specifies the identification (ID) for a Job.

- typedef uint8 [Spi_SequenceType](#)
Specifies the identification (ID) for a sequence of jobs.
- typedef uint8 [Spi_HWUnitType](#)
Specifies the ID for a SPI Hardware microcontroller peripheral unit.
- typedef uint8 [Spi_ExternalDeviceType](#)
Contains the ID of an external device.

Enum Reference

- enum [Spi_StatusType](#)
This type defines a range of specific status for SPI Driver.
- enum [Spi_JobResultType](#)
This type defines a range of specific Jobs status for SPI Driver.
- enum [Spi_SeqResultType](#)
This type defines a range of specific Sequences status for SPI Driver.
- enum [Spi_BufferType](#)
The enumeration containing the designated values for buffer types (internal or external).
- enum [Spi_AsyncModeType](#)
Specifies the asynchronous mechanism mode for SPI buses handled asynchronously in Level 2.
- enum [Spi_HalfDuplexModeType](#)
Half duplex mode.
- enum [Spi_Ipw_SupportedIpsType](#)
This enum contains all IPs which can integrate SPI functionalities.

Function Reference

- void [Spi_GetVersionInfo](#) (Std_VersionInfoType *versioninfo)
This function returns the version information for the SPI driver.
- void [Spi_Init](#) (const [Spi_ConfigType](#) *ConfigPtr)
This function initializes the SPI driver.
- Std_ReturnType [Spi_DeInit](#) (void)
This function de-initializes the SPI driver.
- Std_ReturnType [Spi_WriteIB](#) ([Spi_ChannelType](#) Channel, const [Spi_DataBufferType](#) *DataBufferPtr)
This function writes the given data into the buffer of a specific channel.
- Std_ReturnType [Spi_ReadIB](#) ([Spi_ChannelType](#) Channel, [Spi_DataBufferType](#) *DataBufferPointer)
This function reads the data from the buffer of a channel and puts at the memory location.
- Std_ReturnType [Spi_AsyncTransmit](#) ([Spi_SequenceType](#) Sequence)
This function triggers the asynchronous transmission for the given sequence.
- Std_ReturnType [Spi_SetupEB](#) ([Spi_ChannelType](#) Channel, const [Spi_DataBufferType](#) *SrcDataBufferPtr, [Spi_DataBufferType](#) *DesDataBufferPtr, [Spi_NumberOfDataType](#) Length)
This function setup an external buffer to be used by a specific channel.
- [Spi_StatusType](#) [Spi_GetStatus](#) (void)
This function returns the status of the SPI driver.

- [Spi_JobResultType Spi_GetJobResult \(Spi_JobType Job\)](#)
This function is used to request the status of a specific job.
- [Spi_SeqResultType Spi_GetSequenceResult \(Spi_SequenceType Sequence\)](#)
This function is used to request the status of a specific sequence.
- Std_ReturnType [Spi_SyncTransmit \(Spi_SequenceType Sequence\)](#)
This function is used for synchronous transmission of a given sequence.
- [Spi_StatusType Spi_GetHWUnitStatus \(Spi_HWUnitType HWUnit\)](#)
This function is used to request the status of a specific SPI peripheral unit.
- void [Spi_Cancel \(Spi_SequenceType Sequence\)](#)
This function is used to request the cancelation of the given sequence.
- Std_ReturnType [Spi_SetAsyncMode \(Spi_AsyncModeType Mode\)](#)
This function specifies the asynchronous mode for the SPI busses handled asynchronously.
- Std_ReturnType [Spi_SetHWUnitAsyncMode \(Spi_HWUnitType HWUnit, Spi_AsyncModeType AsyncMode\)](#)
This function specifies the asynchronous mode for a given HWUnit.
- void [Spi_MainFunction_Handling \(void\)](#)
This function shall asynchronously poll SPI interrupts and call ISR if appropriate.

6.3.2 Data Structure Documentation

6.3.2.1 struct Spi_SequenceConfigType

This structure contains all the needed data to configure one SPI Sequence.

Definition at line 567 of file Spi.h.

Data Fields

- [Spi_JobType NumJobs](#)
Number of jobs in the sequence.
- uint32 [SpiCorePartitionUse](#)
UserID used.
- const [Spi_JobType * JobIndexList](#)
Job index list.
- [Spi_NotifyType * EndNotification](#)
Job notification handler.
- uint8 [Interruptible](#)
Boolean indicating if the Sequence is interruptible or not.
- boolean [EnableDmaFastTransfer](#)
Boolean indicating if the Sequence is transferred in Dma fast mode or not.

Field Documentation

NumJobs

`Spi_JobType` NumJobs

Number of jobs in the sequence.

Definition at line 570 of file Spi.h.

SpiCorePartitionUse

`uint32` SpiCorePartitionUse

UserID used.

Definition at line 572 of file Spi.h.

JobIndexList

`const Spi_JobType*` JobIndexList

Job index list.

Definition at line 574 of file Spi.h.

EndNotification

`Spi_NotifyType*` EndNotification

Job notification handler.

Definition at line 576 of file Spi.h.

Interruptible

`uint8` Interruptible

Boolean indicating if the Sequence is interruptible or not.

Definition at line 578 of file Spi.h.

EnableDmaFastTransfer

`boolean` EnableDmaFastTransfer

Boolean indicating if the Sequence is transferred in Dma fast mode or not.

Definition at line 581 of file Spi.h.

6.3.2.2 struct Spi_SeqsConfigType

This structure contains Sequence configuration.

Definition at line 593 of file Spi.h.

Data Fields

const Spi_SequenceConfigType *	SeqConfig	Point to Sequence configuration.
--	-----------	----------------------------------

6.3.2.3 struct Spi_JobConfigType

This is the structure containing all the parameters needed to completely define a Job.

Definition at line 604 of file Spi.h.

Data Fields

- [Spi_ChannelType](#) NumChannels
Number of channels in the job.
- const [Spi_ChannelType](#) * [ChannelIndexList](#)
Channel index list.
- [Spi_NotifyType](#) * [EndNotification](#)
Job end notification.
- [Spi_NotifyType](#) * [StartNotification](#)
Job start notification.
- sint8 [Priority](#)
Priority.
- uint32 [SpiCorePartitionUse](#)
UserID used.
- [Spi_HWUnitType](#) HWUnit
HWUnit.
- [Spi_ExternalDeviceType](#) ExternalDevice
ExternalDevice.
- const [Spi_ExDevicesConfigType](#) * [ExternalDeviceConfig](#)
Implementation specific field: cached LLD device attributes.

Field Documentation**NumChannels**

[Spi_ChannelType](#) NumChannels

Number of channels in the job.

Definition at line 607 of file Spi.h.

ChannelIndexList

```
const Spi_ChannelType* ChannelIndexList
```

Channel index list.

Definition at line 609 of file Spi.h.

EndNotification

```
Spi_NotifyType* EndNotification
```

Job end notification.

Definition at line 611 of file Spi.h.

StartNotification

```
Spi_NotifyType* StartNotification
```

Job start notification.

Definition at line 613 of file Spi.h.

Priority

```
sint8 Priority
```

Priority.

Definition at line 615 of file Spi.h.

SpiCorePartitionUse

```
uint32 SpiCorePartitionUse
```

UserID used.

Definition at line 617 of file Spi.h.

HWUnit

```
Spi_HWUnitType HWUnit
```

HWUnit.

Definition at line 619 of file Spi.h.

ExternalDevice

`Spi_ExternalDeviceType` ExternalDevice

ExternalDevice.

Definition at line 621 of file Spi.h.

ExternalDeviceConfig

`const Spi_ExDevicesConfigType*` ExternalDeviceConfig

Implementation specific field: cached LLD device attributes.

Definition at line 623 of file Spi.h.

6.3.2.4 struct Spi_JobsCfgType

This is the structure containing Job configuration.

Definition at line 629 of file Spi.h.

Data Fields

<code>const Spi_JobConfigType *</code>	JobCfg	Point to Job configuration.
--	--------	-----------------------------

6.3.2.5 struct Spi_BufferDescriptorType

The structure contains the pointers to the Tx/Rx memory locations for the given buffer (IB or EB).

Definition at line 639 of file Spi.h.

Data Fields

<code>const Spi_DataBufferType *</code>	ExternalBufferTX	Transmit buffer pointer.
<code>Spi_DataBufferType *</code>	InternalBufferTX	
<code>Spi_DataBufferType *</code>	BufferRX	Receive buffer pointer.

6.3.2.6 struct Spi_HWUnitQueue

This structure holds the HWUnit scheduling queue.

For async transmissions, this structure holds the HWUnit scheduling queue . For sync transmissions, only HWUnit Status is managed.

Definition at line 654 of file Spi.h.

Data Fields

<code>Spi_JobType</code>	ScheduledJobsListHead[(4)]	Array of the IDs of jobs to be scheduled, for each priority level.
<code>Spi_JobType</code>	ScheduledJobsListTail[(4)]	Array of the IDs of last jobs in queues, for each priority level.
<code>sint8</code>	MaxScheduledPriority	Array of the IDs of last jobs in queues, for each priority level.
<code>Spi_StatusType</code>	Status	Init, idle or busy state.
<code>Spi_ChannelType</code>	Channel	Current channel index in Job.

Data Fields

Spi_JobType	Job	Current job index.
-----------------------------	-----	--------------------

6.3.2.7 struct Spi_ChannelConfigType

The structure contains the channel configuration parameters.

Definition at line 672 of file Spi.h.

Data Fields

Spi_BufferType	BufferType	Buffer Type IB/EB.
uint8	FrameSize	Data frame size.
boolean	Lsb	Bite order (MSB/LSB).
boolean	ByteSwap	Byte Swap.
Spi_HalfDuplexModeType	HalfDuplexMode	Half duplex mode.
uint32	DefaultTransmitValue	Default Transmit Value.
Spi_NumberOfDataType	Length	Data length.
Spi_BufferDescriptorType *	BufferDescriptor	Buffer Descriptor.
uint32	SpiCorePartitionUse	UserID assigned.

6.3.2.8 struct Spi_ChannelsCfgType

The structure contains the channel configuration.

Definition at line 701 of file Spi.h.

Data Fields

const Spi_ChannelConfigType *	ChannelCfg	Point to Channel configuration.
---	------------	---------------------------------

6.3.2.9 struct Spi_ConfigType

This is the top level structure containing all the needed parameters for the SPI Handler Driver.

Definition at line 721 of file Spi.h.

Data Fields

- uint16 [MaxExternalDevice](#)
Number of external devices defined in the configuration.
- [Spi_ChannelType](#) [SpiMaxChannel](#)
Number of channels defined in the configuration.
- [Spi_JobType](#) [SpiMaxJob](#)
Number of jobs defined in the configuration.
- [Spi_SequenceType](#) [SpiMaxSequence](#)
Number of sequences defined in the configuration.

- uint32 [SpiCorePartitionUse](#)
UserID used.
- const [Spi_ChannelsCfgType](#) * [ChannelConfig](#)
Pointer to Array of channels defined in the configuration.
- const [Spi_JobsCfgType](#) * [JobConfig](#)
Pointer to Array of jobs defined in the configuration.
- const [Spi_SeqsConfigType](#) * [SequenceConfig](#)
Pointer to Array of sequences defined in the configuration.
- const [Spi_ExDevicesConfigType](#) * [ExternalDeviceConfig](#)
External device unit attributes.
- const [Spi_PhyUnitsConfigType](#) * [HWUnitConfig](#)
Pointer to Array of LLD DSPI device instances.
- const Mcal_DemErrorType [SpiErrorHardwareCfg](#)
SPI Driver DEM Error: SPI_E_HARDWARE_ERROR.

Field Documentation

MaxExternalDevice

uint16 MaxExternalDevice

Number of external devices defined in the configuration.

Definition at line 724 of file Spi.h.

SpiMaxChannel

[Spi_ChannelType](#) SpiMaxChannel

Number of channels defined in the configuration.

Definition at line 726 of file Spi.h.

SpiMaxJob

[Spi_JobType](#) SpiMaxJob

Number of jobs defined in the configuration.

Definition at line 728 of file Spi.h.

SpiMaxSequence

`Spi_SequenceType` SpiMaxSequence

Number of sequences defined in the configuration.

Definition at line 730 of file Spi.h.

SpiCorePartitionUse

`uint32` SpiCorePartitionUse

UserID used.

Definition at line 732 of file Spi.h.

ChannelConfig

`const Spi_ChannelsCfgType*` ChannelConfig

Pointer to Array of channels defined in the configuration.

Definition at line 734 of file Spi.h.

JobConfig

`const Spi_JobsCfgType*` JobConfig

Pointer to Array of jobs defined in the configuration.

Definition at line 736 of file Spi.h.

SequenceConfig

`const Spi_SeqsConfigType*` SequenceConfig

Pointer to Array of sequences defined in the configuration.

Definition at line 738 of file Spi.h.

ExternalDeviceConfig

`const Spi_ExDevicesConfigType*` ExternalDeviceConfig

External device unit attributes.

Definition at line 740 of file Spi.h.

HWUnitConfig

```
const Spi_PhyUnitsConfigType* HWUnitConfig
```

Pointer to Array of LLD DSPI device instances.

Definition at line 742 of file Spi.h.

SpiErrorHardwareCfg

```
const Mcal_DemErrorType SpiErrorHardwareCfg
```

SPI Driver DEM Error: SPI_E_HARDWARE_ERROR.

Definition at line 745 of file Spi.h.

6.3.2.10 struct Spi_Ipw_ParamSyncTransmit

Parameters for Spi_Ipw_SyncTransmit.

Definition at line 150 of file Spi_Ipw.h.

6.3.2.11 struct Spi_Ipw_ParamAsyncTransmit

Parameters for Spi_Ipw_AsyncTransfer.

Definition at line 162 of file Spi_Ipw.h.

6.3.2.12 struct Spi_Ipw_ParamDMAFast

Parameters for Spi_Ipw_PrepareDMAFastTransferCfg.

Definition at line 174 of file Spi_Ipw.h.

6.3.2.13 struct Spi_Ipw_IpConfigType

@bried This union contains config structure for all IPs available.

Definition at line 146 of file Spi_Ipw_Types.h.

6.3.2.14 struct Spi_Ipw_ExternalDeviceType

@bried This union contains config structure for all external device available.

Definition at line 162 of file Spi_Ipw_Types.h.

6.3.2.15 struct Spi_HWUnitConfigType

This structure holds the HWUnit configuration parameters.

Definition at line 175 of file Spi_Ipw_Types.h.

6.3.2.16 struct Spi_PhyUnitsConfigType

This structure holds the PhyUnit configuration.

Definition at line 186 of file Spi_Ipw_Types.h.

Data Fields

const Spi_HWUnitConfigType *	PhyUnitConfig	Point to PhyUnit configuration.
--	---------------	---------------------------------

6.3.2.17 struct Spi_ExternalDeviceConfigType

This structure holds the configuration parameters for each ExternalDevice.

Definition at line 195 of file Spi_Ipw_Types.h.

6.3.2.18 struct Spi_ExDevicesConfigType

This structure holds the ExternalDevice configuration.

Definition at line 206 of file Spi_Ipw_Types.h.

Data Fields

const Spi_ExternalDeviceConfigType *	ExDeviceConfig	Point to ExternalDevice configuration.
--	----------------	--

6.3.3 Macro Definition Documentation

6.3.3.1 SPI_E_PARAM_CHANNEL

```
#define SPI_E_PARAM_CHANNEL
```

API service called with wrong parameter of Channel.

Definition at line 132 of file Spi.h.

6.3.3.2 SPI_E_PARAM_JOB

```
#define SPI_E_PARAM_JOB
```

API service called with wrong parameter of Job.

Definition at line 138 of file Spi.h.

6.3.3.3 SPI_E_PARAM_SEQ

```
#define SPI_E_PARAM_SEQ
```

API service called with wrong parameter of Sequence.

Definition at line 144 of file Spi.h.

6.3.3.4 SPI_E_PARAM_LENGTH

```
#define SPI_E_PARAM_LENGTH
```

API service called with wrong parameter of external buffer length.

Definition at line 150 of file Spi.h.

6.3.3.5 SPI_E_PARAM_UNIT

```
#define SPI_E_PARAM_UNIT
```

API service called with wrong parameter of HWUnit.

Definition at line 156 of file Spi.h.

6.3.3.6 SPI_E_PARAM_CONFIG

```
#define SPI_E_PARAM_CONFIG
```

API service called with wrong resource assigned.

Definition at line 162 of file Spi.h.

6.3.3.7 SPI_E_UNINIT

```
#define SPI_E_UNINIT
```

API service used without module initialization.

Definition at line 168 of file Spi.h.

6.3.3.8 SPI_E_SEQ_PENDING

```
#define SPI_E_SEQ_PENDING
```

Services called in a wrong sequence.

Definition at line 174 of file Spi.h.

6.3.3.9 SPI_E_SEQ_IN_PROCESS

```
#define SPI_E_SEQ_IN_PROCESS
```

Synchronous transmission service called at wrong time.

Definition at line 180 of file Spi.h.

6.3.3.10 SPI_E_ALREADY_INITIALIZED

```
#define SPI_E_ALREADY_INITIALIZED
```

API SPI_Init service called while the SPI driver has already been initialized.

Definition at line 186 of file Spi.h.

6.3.3.11 SPI_E_CONFIG_OUT_OF_RANGE

```
#define SPI_E_CONFIG_OUT_OF_RANGE
```

The number of sequences, jobs or channels exceeds precompile time sizes.

The number of sequences, jobs or channels in the configuration exceeds precompile time related sizes: SPI_MAX←SEQUENCE, SPI_MAX_JOB or SPI_MAX_CHANNEL.

Definition at line 196 of file Spi.h.

6.3.3.12 SPI_E_INIT_FAILED

```
#define SPI_E_INIT_FAILED
```

API Spi_Init was called with wrong configuration pointer.

Definition at line 203 of file Spi.h.

6.3.3.13 SPI_E_PARAM_EB_UNIT

```
#define SPI_E_PARAM_EB_UNIT
```

When a sequence contains uninitialized external buffers.

Definition at line 210 of file Spi.h.

6.3.3.14 SPI_E_SEQ_EMPTY

```
#define SPI_E_SEQ_EMPTY
```

No job in sequence.

Definition at line 217 of file Spi.h.

6.3.3.15 SPI_E_JOB_EMPTY

```
#define SPI_E_JOB_EMPTY
```

No channel in job.

Definition at line 224 of file Spi.h.

6.3.3.16 SPI_E_PARAM_POINTER

```
#define SPI_E_PARAM_POINTER
```

If the parameter versioninfo or Spi configuration is NULL_PTR.

Definition at line 230 of file Spi.h.

6.3.3.17 SPI_INIT_ID

```
#define SPI_INIT_ID
```

API service ID for SPI Init function.

Parameters used when raising an error or exception.

Definition at line 247 of file Spi.h.

6.3.3.18 SPI_DEINIT_ID

```
#define SPI_DEINIT_ID
```

API service ID for SPI DeInit function.

Parameters used when raising an error or exception.

Definition at line 254 of file Spi.h.

6.3.3.19 SPI_WRITEIB_ID

```
#define SPI_WRITEIB_ID
```

API service ID for SPI write IB function.

Parameters used when raising an error or exception.

Definition at line 261 of file Spi.h.

6.3.3.20 SPI_ASYNCTRANSMIT_ID

```
#define SPI_ASYNCTRANSMIT_ID
```

API service ID for SPI async transmit function.

Parameters used when raising an error or exception.

Definition at line 268 of file Spi.h.

6.3.3.21 SPI_READIB_ID

```
#define SPI_READIB_ID
```

API service ID for SPI read IB function.

Parameters used when raising an error or exception.

Definition at line 275 of file Spi.h.

6.3.3.22 SPI_SETUPEB_ID

```
#define SPI_SETUPEB_ID
```

API service ID for SPI setup EB function.

Parameters used when raising an error or exception.

Definition at line 282 of file Spi.h.

6.3.3.23 SPI_GETSTATUS_ID

```
#define SPI_GETSTATUS_ID
```

API service ID for SPI get status function.

Parameters used when raising an error or exception.

Definition at line 289 of file Spi.h.

6.3.3.24 SPI_GETJOBRESULT_ID

```
#define SPI_GETJOBRESULT_ID
```

API service ID for SPI get job result function.

Parameters used when raising an error or exception.

Definition at line 296 of file Spi.h.

6.3.3.25 SPI_GETSEQUENCERESULT_ID

```
#define SPI_GETSEQUENCERESULT_ID
```

API service ID for SPI get sequence result function.

Parameters used when raising an error or exception.

Definition at line 303 of file Spi.h.

6.3.3.26 SPI_GETVERSIONINFO_ID

```
#define SPI_GETVERSIONINFO_ID
```

API service ID for SPI get version info function.

Parameters used when raising an error or exception.

Definition at line 310 of file Spi.h.

6.3.3.27 SPI_SYNCTRANSMIT_ID

```
#define SPI_SYNCTRANSMIT_ID
```

API service ID for SPI sync transmit function.

Parameters used when raising an error or exception.

Definition at line 317 of file Spi.h.

6.3.3.28 SPI_GETHWUNITSTATUS_ID

```
#define SPI_GETHWUNITSTATUS_ID
```

API service ID for SPI get hwunit status function.

Parameters used when raising an error or exception.

Definition at line 324 of file Spi.h.

6.3.3.29 SPI_CANCEL_ID

```
#define SPI_CANCEL_ID
```

API service ID for SPI cancel function.

Parameters used when raising an error or exception.

Definition at line 331 of file Spi.h.

6.3.3.30 SPI_SETASYNCMODE_ID

```
#define SPI_SETASYNCMODE_ID
```

API service ID for SPI set async mode function.

Parameters used when raising an error or exception.

Definition at line 338 of file Spi.h.

6.3.3.31 SPI_MAINFUNCTION_HANDLING_ID

```
#define SPI_MAINFUNCTION_HANDLING_ID
```

API service ID for SPI main function.

Parameters used when raising an error or exception

Definition at line 345 of file Spi.h.

6.3.3.32 SPI_SETHWUNITASYNCMODE_ID

```
#define SPI_SETHWUNITASYNCMODE_ID
```

API service ID for SPI set HW Unit async mode.

Parameters used when raising an error or exception.

Definition at line 352 of file Spi.h.

6.3.3.33 SPI_SETCLOCKMODE_ID

```
#define SPI_SETCLOCKMODE_ID
```

API service ID for SPI Set Clock Mode.

Parameters used when raising an error or exception.

Definition at line 359 of file Spi.h.

6.3.3.34 SPI_JOB_PRIORITY_LEVELS_COUNT

```
#define SPI_JOB_PRIORITY_LEVELS_COUNT
```

The number of allowed job priority levels (0..3).

The Priority has to be sint8.

Definition at line 367 of file Spi.h.

6.3.3.35 NUMBER_OF_DATA_TYPE_MAX

```
#define NUMBER_OF_DATA_TYPE_MAX
```

Maximum value for number of data elements of the type Spi_DataBufferType.

Definition at line 495 of file Spi.h.

6.3.4 Types Reference

6.3.4.1 Spi_DataBufferType

```
typedef uint8 Spi_DataBufferType
```

Type of application data buffer elements.

Definition at line 481 of file Spi.h.

6.3.4.2 Spi_NumberOfDataType

```
typedef uint16 Spi_NumberOfDataType
```

Type for defining the number of data elements of the type Spi_DataBufferType.

Type for defining the number of data elements of the type Spi_DataBufferType to send or receive by Channel.

Definition at line 490 of file Spi.h.

6.3.4.3 Spi_ChannelType

```
typedef uint8 Spi_ChannelType
```

Specifies the identification (ID) for a Channel.

Definition at line 526 of file Spi.h.

6.3.4.4 Spi_JobType

```
typedef uint16 Spi_JobType
```

Specifies the identification (ID) for a Job.

Definition at line 533 of file Spi.h.

6.3.4.5 Spi_SequenceType

```
typedef uint8 Spi_SequenceType
```

Specifies the identification (ID) for a sequence of jobs.

Definition at line 540 of file Spi.h.

6.3.4.6 Spi_HWUnitType

```
typedef uint8 Spi_HWUnitType
```

Specifies the ID for a SPI Hardware microcontroller peripheral unit.

This type is used for specifying the identification (ID) for a SPI Hardware microcontroller peripheral unit.

Definition at line 550 of file Spi.h.

6.3.4.7 Spi_ExternalDeviceType

```
typedef uint8 Spi_ExternalDeviceType
```

Contains the ID of an external device.

This contains the identification (ID) of the external device for which there's a collection of particular settings

Definition at line 558 of file Spi.h.

6.3.5 Enum Reference

6.3.5.1 Spi_StatusType

```
enum Spi_StatusType
```

This type defines a range of specific status for SPI Driver.

Enumerator

SPI_UNINIT	Not initialized or not usable.
SPI_IDLE	Not currently transmitting any jobs.
SPI_BUSY	Is performing a SPI Job(transmit).

Definition at line 378 of file Spi.h.

6.3.5.2 Spi_JobResultType

```
enum Spi_JobResultType
```

This type defines a range of specific Jobs status for SPI Driver.

Enumerator

SPI_JOB_OK	The last transmission of the Job has been finished successfully.
SPI_JOB_PENDING	The SPI handler/Driver is performing a SPI Job.
SPI_JOB_FAILED	The last transmission of the Job has failed.
SPI_JOB_QUEUED	An asynchronous transmit Job has been accepted, while actual transmission for this Job has not started yet.

Definition at line 390 of file Spi.h.

6.3.5.3 Spi_SeqResultType

```
enum Spi_SeqResultType
```

This type defines a range of specific Sequences status for SPI Driver.

Enumerator

SPI_SEQ_OK	The last transmission of the Sequence has been finished successfully.
------------	---

Enumerator

SPI_SEQ_PENDING	The SPI handler/Driver is performing a SPI Sequence.
SPI_SEQ_FAILED	The last transmission of the Sequence has failed.
SPI_SEQ_CANCELLED	The last transmission of the Sequence has been cancelled by the user.

Definition at line 404 of file Spi.h.

6.3.5.4 Spi_BufferType

enum [Spi_BufferType](#)

The enumeration containing the designated values for buffer types (internal or external).

Enumerator

IB	The Channel is configured using Internal Buffer.
EB	The Channel is configured using External Buffer.

Definition at line 416 of file Spi.h.

6.3.5.5 Spi_AsyncModeType

enum [Spi_AsyncModeType](#)

Specifies the asynchronous mechanism mode for SPI buses handled asynchronously in Level 2.

`#if (SPI_LEVEL2 == SPI_LEVEL_DELIVERED)` Specifies the asynchronous mechanism mode for SPI buses handled asynchronously in LEVEL 2. SPI150: This type is available or not according to the pre compile time parameter: SPI_LEVEL_DELIVERED. This is only relevant for LEVEL 2.

Enumerator

SPI_POLLING_MODE	The asynchronous mechanism is ensured by polling, so interrupts related to SPI buses handled asynchronously are disabled.
SPI_INTERRUPT_MODE	The asynchronous mechanism is ensured by interrupt, so interrupts related to SPI buses handled asynchronously are enabled.

Definition at line 432 of file Spi.h.

6.3.5.6 Spi_HalfDuplexModeType

enum [Spi_HalfDuplexModeType](#)

Half duplex mode.

Enumerator

SPI_HALF_DUPLEX_TRANSMIT	Transmit only.
--------------------------	----------------

Enumerator

SPI_HALF_DUPLEX_RECEIVE	Receive only.
SPI_FULL_DUPLEX	Full duplex mode.

Definition at line 460 of file Spi.h.

6.3.5.7 Spi_Ipw_SupportedIpsType

enum [Spi_Ipw_SupportedIpsType](#)

This enum contains all IPs which can integrate SPI functionalities.

Definition at line 135 of file Spi_Ipw_Types.h.

6.3.6 Function Reference

6.3.6.1 Spi_GetVersionInfo()

```
void Spi_GetVersionInfo (
    Std_VersionInfoType * versioninfo )
```

This function returns the version information for the SPI driver.

This function returns the version information for the SPI driver.

- Service ID: 0x09
- Sync or Async: Synchronous
- Reentrancy: Non-Reentrant

Parameters

in, out	<i>versioninfo</i>	Pointer to where to store the version information of this module.
---------	--------------------	---

Precondition

Pre-compile parameter SPI_VERSION_INFO_API shall be STD_ON.

6.3.6.2 Spi_Init()

```
void Spi_Init (
    const Spi\_ConfigType * ConfigPtr )
```

This function initializes the SPI driver.

This function initializes the SPI driver using the pre-established configurations

- Service ID: 0x00
- Sync or Async: Synchronous
- Reentrancy: Non-Reentrant

Parameters

in	<i>ConfigPtr</i>	Specifies the pointer to the configuration set
----	------------------	--

6.3.6.3 Spi_DeInit()

```
Std_ReturnType Spi_DeInit (
    void )
```

This function de-initializes the SPI driver.

This function de-initializes the SPI driver using the pre-established configurations

- Service ID: 0x01
- Sync or Async: Synchronous
- Reentrancy: Non-Reentrant

Returns

Std_ReturnType

Return values

<i>E_OK</i>	de-initialisation command has been accepted
<i>E_NOT_OK</i>	de-initialisation command has not been accepted

Precondition

The driver needs to be initialized before calling [Spi_DeInit\(\)](#) otherwise, the function [Spi_DeInit\(\)](#) shall raise the development error if SPI_DEV_ERROR_DETECT is STD_ON.

6.3.6.4 Spi_WriteIB()

```
Std_ReturnType Spi_WriteIB (
    Spi_ChannelType Channel,
    const Spi_DataBufferType * DataBufferPtr )
```

This function writes the given data into the buffer of a specific channel.

This function writes the given data into the buffer of a specific channel.

- Service ID: 0x02
- Sync or Async: Synchronous
- Reentrancy: Reentrant

Parameters

in	<i>Channel</i>	Channel ID
in	<i>DataBufferPtr</i>	Pointer to source data buffer

Returns

Std_ReturnType

Return values

<i>E_OK</i>	Command has been accepted
<i>E_NOT_OK</i>	Command has not been accepted

Precondition

The driver needs to be initialized before calling [Spi_WriteIB\(\)](#) otherwise, the function [Spi_WriteIB\(\)](#) shall raise the development error if SPI_DEV_ERROR_DETECT is STD_ON.

Pre-compile parameter SPI_CHANNEL_BUFFERS_ALLOWED shall be SPI_USAGE0 or SPI_USAGE2.

6.3.6.5 Spi_ReadIB()

```
Std_ReturnType Spi_ReadIB (
    Spi_ChannelType Channel,
    Spi_DataBufferType * DataBufferPointer )
```

This function reads the data from the buffer of a channel and puts at the memory location.

This function reads the data from the buffer of a specific channel and puts at the specified memory location.

- Service ID: 0x04
- Sync or Async: Synchronous
- Reentrancy: Reentrant

Parameters

in	<i>Channel</i>	Channel ID
in, out	<i>DataBufferPointer</i>	Pointer to the memory location that will be written with the data in the internal buffer

Returns

Std_ReturnType

Return values

<i>E_OK</i>	read command has been accepted
<i>E_NOT_OK</i>	read command has not been accepted

Precondition

The driver needs to be initialized before calling [Spi_ReadIB\(\)](#) otherwise, the function [Spi_ReadIB\(\)](#) shall raise the development error if `SPI_DEV_ERROR_DETECT` is `STD_ON`.

Pre-compile parameter `SPI_CHANNEL_BUFFERS_ALLOWED` shall be `SPI_USAGE0` or `SPI_USAGE2`.

6.3.6.6 Spi_AsyncTransmit()

```
Std_ReturnType Spi_AsyncTransmit (
    Spi_SequenceType Sequence )
```

This function triggers the asynchronous transmission for the given sequence.

This function triggers the asynchronous transmission for the given sequence.

- Service ID: 0x03
- Sync or Async: Asynchronous
- Reentrancy: Reentrant

Parameters

in	<i>Sequence</i>	Sequence ID
----	-----------------	-------------

Returns

Std_ReturnType

Return values

<i>E_OK</i>	Transmission command has been accepted
<i>E_NOT_OK</i>	Transmission command has not been accepted

Precondition

The driver needs to be initialized before calling [Spi_AsyncTransmit\(\)](#) otherwise, the function [Spi_AsyncTransmit\(\)](#) shall raise the development error if `SPI_DEV_ERROR_DETECT` is `STD_ON`.

Pre-compile parameter `SPI_LEVEL_DELIVERED` shall be `SPI_LEVEL1` or `SPI_LEVEL2`.

6.3.6.7 Spi_SetupEB()

```
Std_ReturnType Spi_SetupEB (
    Spi_ChannelType Channel,
    const Spi_DataBufferType * SrcDataBufferPtr,
    Spi_DataBufferType * DesDataBufferPtr,
    Spi_NumberOfDataType Length )
```

This function setup an external buffer to be used by a specific channel.

This function setup an external buffer to be used by a specific channel.

- Service ID: 0x05
- Sync or Async: Synchronous

- Reentrancy: Reentrant

Parameters

in	<i>Channel</i>	Channel ID
in	<i>SrcDataBufferPtr</i>	Pointer to the memory location that will hold the transmitted data
in	<i>Length</i>	Number of data elements in the external buffer
out	<i>DesDataBufferPtr</i>	Pointer to the memory location that will hold the received data

Returns

Std_ReturnType

Return values

<i>E_OK</i>	Setup command has been accepted
<i>E_NOT_OK</i>	Setup command has not been accepted

Precondition

The driver needs to be initialized before calling [Spi_SetupEB\(\)](#) otherwise, the function [Spi_SetupEB\(\)](#) shall raise the development error if SPI_DEV_ERROR_DETECT is STD_ON.

Pre-compile parameter SPI_CHANNEL_BUFFERS_ALLOWED shall be SPI_USAGE1 or SPI_USAGE2.

6.3.6.8 Spi_GetStatus()

```
Spi_StatusType Spi_GetStatus (
    void )
```

This function returns the status of the SPI driver.

This function returns the status of the SPI driver.

- Service ID: 0x06
- Sync or Async: Synchronous
- Reentrancy: Reentrant

Returns

Spi_StatusType

Return values

<i>SPI_UNINIT</i>	The driver is un-initialized
<i>SPI_IDLE</i>	The driver has no pending transfers
<i>SPI_BUSY</i>	The driver is busy

Precondition

The driver needs to be initialized before calling [Spi_GetStatus\(\)](#) otherwise, the function [Spi_GetStatus\(\)](#) shall raise the development error if SPI_DEV_ERROR_DETECT is STD_ON.

6.3.6.9 Spi_GetJobResult()

```
Spi_JobResultType Spi_GetJobResult (  
    Spi_JobType Job )
```

This function is used to request the status of a specific job.
This function is used to request the status of a specific job.

- Service ID: 0x07
- Sync or Async: Synchronous
- Reentrancy: Reentrant

Parameters

in	<i>Job</i>	Job ID
----	------------	--------

Returns

Spi_JobResultType

Return values

<i>SPI_JOB_OK</i>	The job ended successfully
<i>SPI_JOB_PENDING</i>	The job is pending
<i>SPI_JOB_FAILED</i>	The job has failed

Precondition

The driver needs to be initialized before calling Spi_GetJobResult() otherwise, the function Spi_GetJobResult() shall raise the development error if SPI_DEV_ERROR_DETECT is STD_ON.

6.3.6.10 Spi_GetSequenceResult()

```
Spi_SeqResultType Spi_GetSequenceResult (  
    Spi_SequenceType Sequence )
```

This function is used to request the status of a specific sequence.
This function is used to request the status of a specific sequence.

- Service ID: 0x08
- Sync or Async: Synchronous
- Reentrancy: Reentrant

Parameters

in	<i>Sequence</i>	Sequence ID
----	-----------------	-------------

Returns

Spi_SeqResultType

Return values

<i>SPI_SEQ_OK</i>	The sequence ended successfully
<i>SPI_SEQ_PENDING</i>	The sequence is pending
<i>SPI_SEQ_FAILED</i>	The sequence has failed

Precondition

The driver needs to be initialized before calling [Spi_GetSequenceResult\(\)](#) otherwise, the function [Spi_GetSequenceResult\(\)](#) shall raise the development error if SPI_DEV_ERROR_DETECT is STD_ON.

6.3.6.11 Spi_SyncTransmit()

```
Std_ReturnType Spi_SyncTransmit (
    Spi_SequenceType Sequence )
```

This function is used for synchronous transmission of a given sequence.

This function is used for synchronous transmission of a given sequence.

- Service ID: 0x0a
- Sync or Async: Synchronous
- Reentrancy: Reentrant

Parameters

in	<i>Sequence</i>	Sequence ID
----	-----------------	-------------

Returns

Std_ReturnType

Return values

<i>E_OK</i>	Transmission command has been completed successfully
<i>E_NOT_OK</i>	Transmission command has not been accepted

Precondition

The driver needs to be initialized before calling [Spi_SyncTransmit\(\)](#). otherwise, the function [Spi_SyncTransmit\(\)](#) shall raise the development error if SPI_DEV_ERROR_DETECT is STD_ON.

Pre-compile parameter SPI_LEVEL_DELIVERED shall be SPI_LEVEL0 or SPI_LEVEL2

6.3.6.12 Spi_GetHWUnitStatus()

```
Spi_StatusType Spi_GetHWUnitStatus (
    Spi_HWUnitType HWUnit )
```

This function is used to request the status of a specific SPI peripheral unit.

This function is used to request the status of a specific SPI peripheral unit.

Module Documentation

- Service ID: 0x0b
- Sync or Async: Synchronous
- Reentrancy: Reentrant

Parameters

in	<i>HWUnit</i>	The HW peripheral for which we need the status
----	---------------	--

Returns

`Spi_StatusType`

Return values

<i>SPI_UNINIT</i>	The peripheral is un-initialized
<i>SPI_IDLE</i>	The peripheral is in idle state
<i>SPI_BUSY</i>	The peripheral is busy

Precondition

The driver needs to be initialized before calling `Spi_GetHWUnitStatus()` otherwise, the function `Spi_GetHWUnitStatus()` shall raise the development error if `SPI_DEV_ERROR_DETECT` is `STD_ON`.

`SPI_HW_STATUS_API == STD_ON`

6.3.6.13 Spi_Cancel()

```
void Spi_Cancel (
    Spi_SequenceType Sequence )
```

This function is used to request the cancelation of the given sequence.

This function is used to request the cancelation of the given sequence.

- Service ID: 0x0c
- Sync or Async: Asynchronous
- Reentrancy: Reentrant

Parameters

in	<i>Sequence</i>	Sequence ID
----	-----------------	-------------

Precondition

The driver needs to be initialized before calling `Spi_Cancel()` otherwise, the function `Spi_Cancel()` shall raise the development error if `SPI_DEV_ERROR_DETECT` is `STD_ON`.

Pre-compile parameter `SPI_CANCEL_API` shall be `STD_ON`

Postcondition

The SPI Handler Driver is not responsible on external devices damages or undefined state due to cancelling a sequence transmission.

6.3.6.14 Spi_SetAsyncMode()

```
Std_ReturnType Spi_SetAsyncMode (
    Spi_AsyncModeType Mode )
```

This function specifies the asynchronous mode for the SPI busses handled asynchronously.

This function specifies the asynchronous mode for the SPI busses handled asynchronously.

- Service ID: 0x0d
- Sync or Async: Synchronous
- Reentrancy: Non-Reentrant

Parameters

in	<i>Mode</i>	This parameter specifies the asynchronous operating mode (SPI_POLLING_MODE or SPI_INTERRUPT_MODE)
----	-------------	---

Returns

Std_ReturnType

Return values

<i>E_OK</i>	The command ended successfully
<i>E_NOT_OK</i>	The command has failed

Precondition

The driver needs to be initialized before calling [Spi_SetAsyncMode\(\)](#) otherwise, the function [Spi_SetAsyncMode\(\)](#) shall raise the development error if SPI_DEV_ERROR_DETECT is STD_ON.

Pre-compile parameter SPI_LEVEL_DELIVERED shall be SPI_LEVEL1 or SPI_LEVEL2

6.3.6.15 Spi_SetHWUnitAsyncMode()

```
Std_ReturnType Spi_SetHWUnitAsyncMode (
    Spi_HWUnitType HWUnit,
    Spi_AsyncModeType AsyncMode )
```

This function specifies the asynchronous mode for a given HWUnit.

This function specifies the asynchronous mode for the SPI busses handled asynchronously. For synchronous HW units, the function has no impact. The function will fail in two cases:

- driver not initialised (SPI_E_UNINIT reported by DET)
- a sequence transmission is pending the the asynchronous HW unit (SPI_E_SEQ_PENDING reported by DET)

Parameters

in	<i>HWUnit</i>	The ID of the HWUnit to be configured
----	---------------	---------------------------------------

Parameters

in	<i>AsyncMode</i>	This parameter specifies the asynchronous operating mode (SPI_POLLING_MODE or SPI_INTERRUPT_MODE)
----	------------------	---

Returns

Std_ReturnType

Return values

<i>E_OK</i>	The command ended successfully
<i>E_NOT_OK</i>	The command has failed

Precondition

Pre-compile parameter SPI_LEVEL_DELIVERED shall be SPI_LEVEL2 and SPI_HWUNIT_ASYNC_MODE should be on STD_ON

6.3.6.16 Spi_MainFunction_Handling()

```
void Spi_MainFunction_Handling (
    void )
```

This function shall asynchronously poll SPI interrupts and call ISR if appropriate.

This function shall asynchronously poll SPI interrupts and call ISR if appropriate.

- Service ID: 0x10

Precondition

Pre-compile parameter SPI_LEVEL_DELIVERED shall be SPI_LEVEL1 or SPI_LEVEL2.

How to Reach Us:

Home Page:

nxp.com

Web Support:

nxp.com/support

Limited warranty and liability - Information in this document is believed to be accurate and reliable. However, NXP Semiconductors does not give any representations or warranties, expressed or implied, as to the accuracy or completeness of such information and shall have no liability for the consequences of use of such information. NXP Semiconductors takes no responsibility for the content in this document if provided by an information source outside of NXP Semiconductors.

In no event shall NXP Semiconductors be liable for any indirect, incidental, punitive, special or consequential damages (including - without limitation - lost profits, lost savings, business interruption, costs related to the removal or replacement of any products or rework charges) whether or not such damages are based on tort (including negligence), warranty, breach of contract or any other legal theory.

Notwithstanding any damages that customer might incur for any reason whatsoever, NXP Semiconductors' aggregate and cumulative liability towards customer for the products described herein shall be limited in accordance with the Terms and conditions of commercial sale of NXP Semiconductors.

Right to make changes - NXP Semiconductors reserves the right to make changes to information published in this document, including without limitation specifications and product descriptions, at any time and without notice. This document supersedes and replaces all information supplied prior to the publication hereof.

Suitability for use in automotive applications - This NXP product has been qualified for use in automotive applications. If this product is used by customer in the development of, or for incorporation into, products or services (a) used in safety critical applications or (b) in which failure could lead to death, personal injury, or severe physical or environmental damage (such products and services hereinafter referred to as "Critical Applications"), then customer makes the ultimate design decisions regarding its products and is solely responsible for compliance with all legal, regulatory, safety, and security related requirements concerning its products, regardless of any information or support that may be provided by NXP. As such, customer assumes all risk related to use of any products in Critical Applications and NXP and its suppliers shall not be liable for any such use by customer. Accordingly, customer will indemnify and hold NXP harmless from any claims, liabilities, damages and associated costs and expenses (including attorneys' fees) that NXP may incur related to customer's incorporation of any product in a Critical Application.

Applications - Applications that are described herein for any of these products are for illustrative purposes only. NXP Semiconductors makes no representation or warranty that such applications will be suitable for the specified use without further testing or modification.

Customers are responsible for the design and operation of their applications and products using NXP Semiconductors products, and NXP Semiconductors accepts no liability for any assistance with applications or customer product design. It is customer's sole responsibility to determine whether the NXP Semiconductors product is suitable and fit for the customer's applications and products planned, as well as for the planned application and use of customer's third party customer(s). Customers should provide appropriate design and operating safeguards to minimize the risks associated with their applications and products.

NXP Semiconductors does not accept any liability related to any default, damage, costs or problem which is based on any weakness or default in the customer's applications or products, or the application or use by customer's third party customer(s). Customer is responsible for doing all necessary testing for the customer's applications and products using NXP Semiconductors products in order to avoid a default of the applications and the products or of the application or use by customer's third party customer(s). NXP does not accept any liability in this respect.

Terms and conditions of commercial sale - NXP Semiconductors products are sold subject to the general terms and conditions of commercial sale, as published at <http://www.nxp.com/profile/terms>, unless otherwise agreed in a valid written individual agreement. In case an individual agreement is concluded only the terms and conditions of the respective agreement shall apply. NXP Semiconductors hereby expressly objects to applying the customer's general terms and conditions with regard to the purchase of NXP Semiconductors products by customer.

Export control - This document as well as the item(s) described herein may be subject to export control regulations. Export might require a prior authorization from competent authorities.

Translations — A non-English (translated) version of a document, including the legal information in that document, is for reference only. The English version shall prevail in case of any discrepancy between the translated and English versions.

Security — Customer understands that all NXP products may be subject to unidentified vulnerabilities or may support established security standards or specifications with known limitations. Customer is responsible for the design and operation of its applications and products throughout their lifecycles to reduce the effect of these vulnerabilities on customer's applications and products. Customer's responsibility also extends to other open and/or proprietary technologies supported by NXP products for use in customer's applications. NXP accepts no liability for any vulnerability. Customer should regularly check security updates from NXP and follow up appropriately.

Customer shall select products with security features that best meet rules, regulations, and standards of the intended application and make the ultimate design decisions regarding its products and is solely responsible for compliance with all legal, regulatory, and security related requirements concerning its products, regardless of any information or support that may be provided by NXP.

NXP has a Product Security Incident Response Team (PSIRT) (reachable at <mailto:PSIRT@nxp.com>) that manages the investigation, reporting, and solution release to security vulnerabilities of NXP products.

NXP B.V. - NXP B.V. is not an operating company and it does not distribute or sell products.